



VEScript User Guide

VEScript Python editor & API

IES Virtual Environment

Copyright © 2026 Integrated Environmental Solutions Limited. All rights reserved.

No part of the manual is to be copied or reproduced in any form without the express agreement of Integrated Environmental Solutions Limited.

Contents

1	INTRODUCTION	4
1.1	INTRODUCTION TO PYTHON.....	5
1.2	ADDITIONAL RESOURCES	6
1.3	3 RD PARTY LIBRARIES.....	7
2	VESCRIPITS EDITOR.....	8
2.1	FILE LIST.....	8
2.2	SCRIPTING TOOLBAR	9
2.3	CODE EDITING WINDOW	9
2.4	OUTPUT WINDOW / OUTPUT WINDOW TOOLBAR	9
2.5	HELP WINDOW.....	10
2.6	WATCH WINDOW	10
2.7	BREAKPOINTS.....	10
3	RUNNING SCRIPTS	11
3.1	PROMOTE TO NAVIGATOR	11
3.2	INSTALLING A SCRIPT PACKAGE	11
3.3	CREATING A SCRIPT PACKAGE	11
4	CREATING A SAMPLE SCRIPT	12
5	TROUBLESHOOTING ERROR MESSAGES.....	15
5.1	SYNTAXERROR: INVALID SYNTAX	15
5.2	INDENTATIONERROR: UNEXPECTED INDENT	15
5.3	TYPEERROR: CAN'T CONVERT 'INT; OBJECT TO STR IMPLICITLY	16
5.4	KEYERROR: 'ROOM_NAME'	16
5.5	ATTRIBUTEERROR: 'ROOMGROUPS' OBJECT HAS NO ATTRIBUTE 'GET_ROOM_GROUPZ'	16
5.6	ARGUMENTERROR	17
5.7	OTHER RESOURCES.....	17
6	VE PYTHON API	18
6.1	CONVENTIONS USED IN THE VE API AND USER GUIDE	20
6.1.1	<i>Top Level Methods.....</i>	<i>21</i>
6.1.2	<i>AirExchange</i>	<i>22</i>
6.1.3	<i>ApacheSim</i>	<i>23</i>
6.1.4	<i>BpfCustom.....</i>	<i>26</i>
6.1.5	<i>CasualGain</i>	<i>27</i>
6.1.6	<i>EnergySources.....</i>	<i>29</i>
6.1.7	<i>HVAC Network</i>	<i>30</i>
6.1.8	<i>IECC.....</i>	<i>141</i>
6.1.9	<i>ImportGBXML</i>	<i>143</i>
6.1.10	<i>Mv2 (Model Viewer 2)</i>	<i>144</i>
6.1.11	<i>NECB</i>	<i>145</i>
6.1.12	<i>PRM</i>	<i>147</i>
6.1.13	<i>ProjectInfo</i>	<i>151</i>
6.1.14	<i>ResultsReader</i>	<i>152</i>
6.1.15	<i>RoomAirExchange</i>	<i>160</i>
6.1.16	<i>RoomInternalGain</i>	<i>162</i>
6.1.17	<i>RoomGroups.....</i>	<i>164</i>
6.1.18	<i>TariffsEngine.....</i>	<i>166</i>
6.1.19	<i>TDVCalculator.....</i>	<i>174</i>
6.1.20	<i>T24.....</i>	<i>174</i>
6.1.21	<i>T24EndUseData</i>	<i>175</i>
6.1.22	<i>T24ResponsiblePerson</i>	<i>177</i>
6.1.23	<i>TransformerLosses.....</i>	<i>178</i>

6.1.24	UMLH.....	179
6.1.25	VEAdjacency	180
6.1.26	VEApacheSystem	181
6.1.27	VEBody.....	195
6.1.28	VECdbConstruction	198
6.1.29	VECdbDatabase	201
6.1.30	VECdbLayer.....	202
6.1.31	VECdbMaterial.....	203
6.1.32	VECdbProject	204
6.1.33	VEComponentProcess.....	207
6.1.34	VEEnergyMeter.....	209
6.1.35	VEGeometry.....	210
6.1.36	VELocate.....	213
6.1.37	VEMacroFlo	215
6.1.38	VEModel	216
6.1.39	VEProfile	218
6.1.40	VEProject	222
6.1.41	VERenewables	225
6.1.42	VERoomData	229
6.1.43	VESankey	237
6.1.44	VESuncast	239
6.1.45	VESurface.....	240
6.1.46	VEThermalTemplate	242
6.1.47	WeatherFileReader.....	246
6.1.48	RefModelUserEdits	248

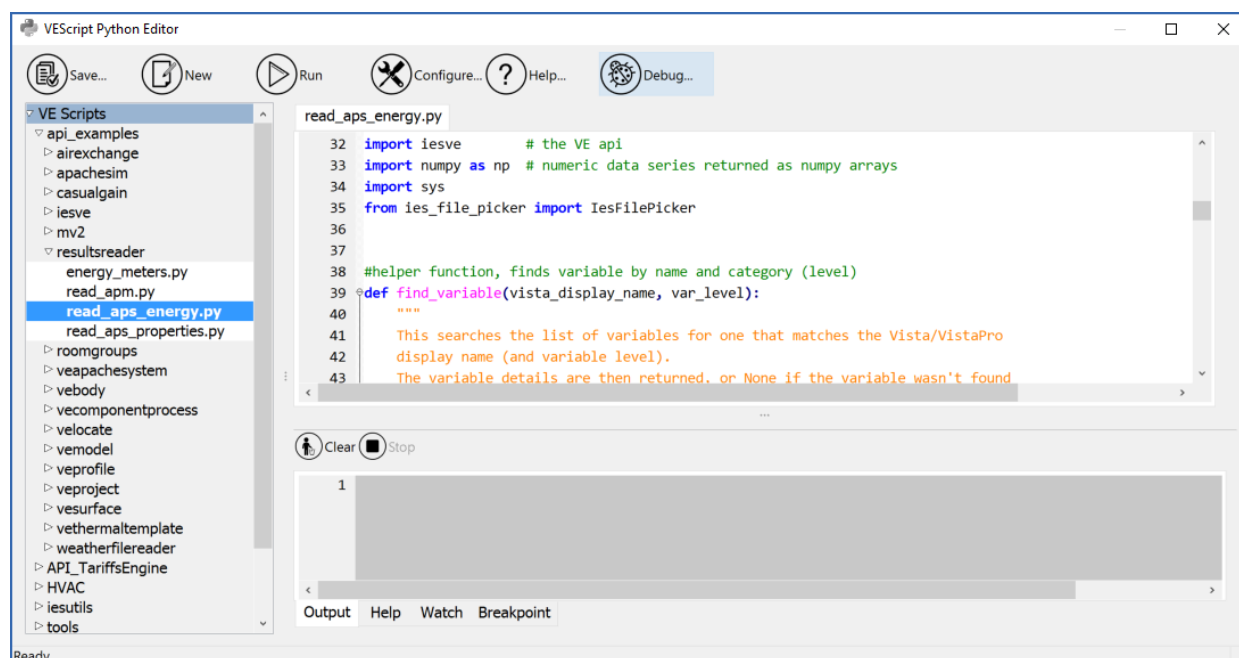
1 Introduction

The Python API (application programming interface) was introduced in VE 2017. An API in its simplest terms is a mechanism for one software program to communicate with another software program. The VE Python API allows users create simple programs using the Python Programming Language, to interact with a VE Model and/or Vista Results File. This gives users the option to automate many of the repetitive tasks, which up to now would have been relatively tedious. The VE Python API will allow users to create simple python programs, to automatically:

- Extract specific information from a VE Model or Vista Results file,
- Perform some calculations on the data if desired,
- Arrange the data into a standard format,
- Write the data to another program such as Microsoft Excel or Word.

Some simple examples of this would include:

- Extracting air temperatures for a group of rooms from the Vista Results File, performing a range test to determine the percentage of annual occupied hours that each room is above 28°C and then writing this data to an excel sheet.
- Extracting room data (internal gains, ventilation rates, etc.) for a group of rooms from the building template manager, collating this data into a table and then writing it to an excel sheet.
- Extracting location and climate data from the weather file that is assigned to a model and writing it to a word file.



1.1 Introduction to Python

Python is a high level interpreted programming language used for general purpose programming and scripting. The main advantage of using Python over other programming languages is its readability and user friendliness. Python has a syntax that allows programs to be expressed in fewer lines of code than would be possible in many other languages. Python has a large library of open source modules that can be easily imported and used in any project. An experienced programmer in any other language can pick up Python very quickly, however it is also quite straightforward for beginners to learn and use. Note that the VE Python API uses Python 3 not Python 2.

Before attempting to use the VE Python API, it is strongly advised that the user becomes familiar with the basics of the Python Programming Language. Specifically the user must be competent in the following areas:

- Variables and Types
 - How to declare variables
 - Types -> int, float, string, bool
- Lists
 - Accessing values in a list using []
 - Append and Delete values from list
 - Get max min and length of list
- Basic Operators
 - Arithmetic operators: '+', '-', '*', '/'
 - Comparison operators: '==', '!=', '>', '<', '>=', '<='
 - Assignment operators: '=', '+=', '-='
 - Bool operators: 'and', 'or'
 - Using operators with lists
- String Formatting
 - String substitution
- Basic String Operation
 - String slicing
 - '.split()', '.strip()', 'len()', 'join()'
 - Concatenating strings using + operator
- Conditionals
 - 'if', 'else', 'elif' statements
 - Using with Bool operators
- Loops
 - 'for' loop
 - 'while' loop
 - 'break' and 'continue' statements
- Functions
 - Defining a function
 - Calling a function
 - Arguments and Parameters
- Dictionaries
 - Initialising a dictionary object
 - Adding and removing items from a dictionary
 - Accessing values in a dictionary using a key
 - Iterating over dictionaries using a 'for' loop



- Modules and Packages
 - Importing built in modules
 - 'from', 'import', 'as' statements
- List Comprehensions
 - Creating a new list based on an existing list
- Classes and Objects
 - Defining a class
 - The '__init__():' method
 - Defining attributes of a class
 - Defining methods of a class
 - Declaring an object

1.2 Additional Resources

Many free online learning resources including:

- Basic Python Tutorials:
 - <https://www.learnpython.org/>
 - <https://docs.python.org/3/tutorial/>
 - <http://www.tutorialspoint.com/python3/>
 - <https://youtu.be/IX6mc9l6tY4?list=PLQVvva0QuDe8XSftW-RAxdo6OmaeL85M>
- Numpy Tutorial for Beginners:
 - <http://www.python-course.eu/numpy.php>
- Xlsx Writer Tutorial:
 - <http://xlsxwriter.readthedocs.io/>
- Tkinter Tutorial:
 - <http://effbot.org/tkinterbook/tkinter-index.htm>

Note that in addition to knowing the basics of Python, it is advisable that the user has experience in the VE before attempting to use the VE Python API.

1.3 3rd Party Libraries

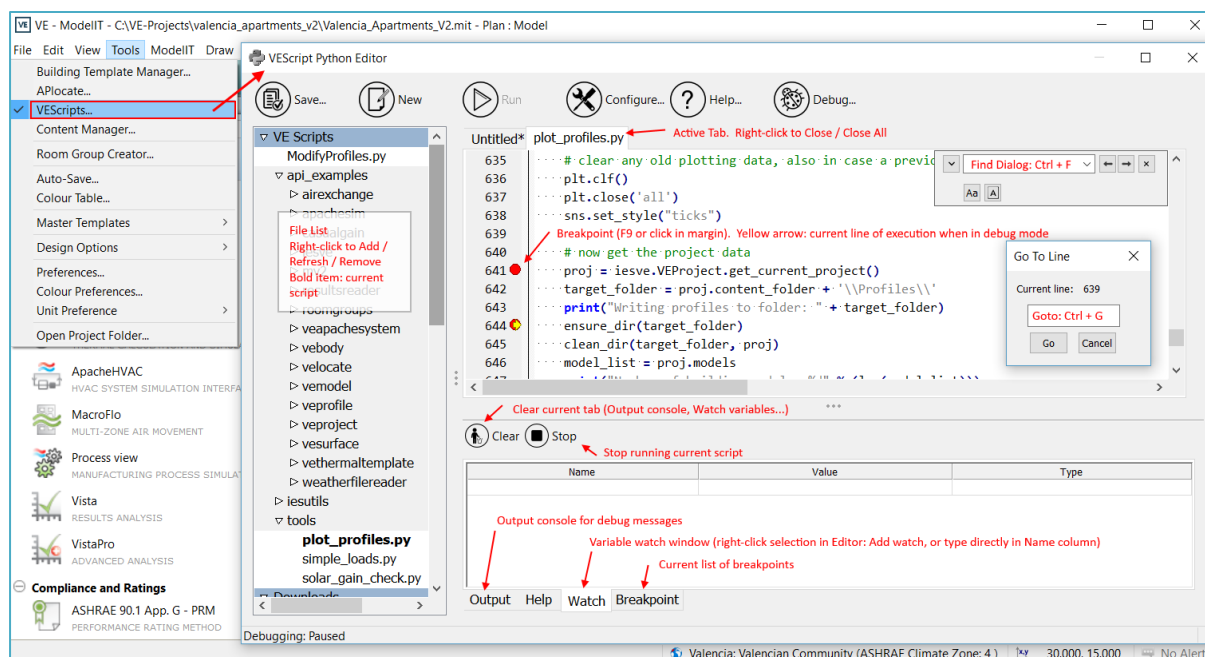
The following 3rd party python libraries are available in the VE Python environment:

- [Numpy 1.11.0](#)
- [Scipy 0.17.0](#)
- [Pandas 0.18.1](#)
- [Matplotlib 1.5.1](#) (+ the [Seaborn 0.7.0](#) formatting package)
- [Dateutil 2.0](#) (extensions to the standard python datetime module)
- [Docx 0.8.5](#) (create Word documents)
- [XlsxWriter 0.7.3](#) (create Excel spreadsheets)
- [Xlrd 0.9.4](#) (read Excel spreadsheets)
- [Lxml 3.4.4](#) (xml reader/writer)
- [Pywin32 219](#) (Windows extensions, including COM interoperability)
- [CoolProp 5.1.1](#) (thermodynamic and transport properties database)
- [NumExpr 2.4.6](#) (fast numerical expression evaluator for NumPy)
- [Jinja2-2.8](#) (templating language)
- [MarkupSafe-0.23](#) (create XML/HTML/XHTML Markup safe string)
- [Pillow-3.0.0](#) (Python imaging library)
- [bokeh-0.10.0](#) (HTML based plotting library, output to file only)
- [requests-2.9.1](#) (HTTP requests made easy)
- [PyTables-3.2.2](#) (large dataset support, including HDF5 support for Pandas)
- [Arrow-0.7.0](#) (better dates and times)
- [ReportLab 3.2](#) (PDF creation library)
- [Pint 0.7.1](#) (unit conversion)
- [Six 1.9.0](#) (Python 2/3 compatibility library)

2 VEScripts Editor

The VE includes an integrated editor / debugger / console solution for developing VEScripts from inside the <VE>. This Python editor is modeless so activating it will not stop you from working with the rest of the VE.

To launch the VEScripts editor, use the VE Tools menu, VEScripts... entry (appropriate licence required for entry to be visible).



The editor consists of the following elements:

2.1 File List

By default, the File List will show the scripts that are included with the VE. **Add your own scripts by right-clicking anywhere in the File List and selecting “Add script folder”**. To remove a scripting folder, right-click on the top-level node for that folder and select “Remove folder”. The VE scripts folder cannot be removed. To force a refresh of the File List content, right-click and select Refresh. **Only files with the “.py” extension will show up as scripts**.

The File List right-click menu also allows you to “Promote to Navigator”. This inserts the relevant script file into the VE Python Scripts navigator. See Chapter 3 for further information on running scripts.

Selecting a script in the file list will load its content into the editor in a new tab (or if already loaded, its tab will be brought to the foreground). The currently active script is shown in **bold**.

2.2 Scripting Toolbar

Allows access to basic features of the scripting dialog. It has the following entries:

Save: drop-down menu (Save current tab, Save As... current tab, or Save All tabs). When trying to Save the content of a tab that is Untitled (no file name), you will be prompted to select a filename/location.

New: create a new, empty, untitled script file tab

Run: run the script in the current editing window. If the script was edited, you will be prompted to save it first (shortcut key: F5).

Configure:

Show the configuration dialog with display options for the Editor

Help:

Python Documentation: takes you to the Python documentation page for Python 3.4.3
VE Automation API: This document

Debug:

Run Debugger (Shift + F5): start the current script in debug mode.
Pause Debugging (Ctrl + B): pause current running script
Continue (F5): if paused, continue running the script
Step In (F11): step execution, going into a function if possible
Step Over (F12): step execution, do not go into function (execute function in 1 step)
Step Out (Shift + F11): continue execution until we're out of the current function
Stop: if running or paused, stop executing the script

2.3 Code Editing Window

- Standard editing control with syntax highlighting for Python. Supports code block collapsing / expanding, cut/copy/paste.
- Bracket / brace matching highlighting
- Ctrl + F launches find/replace window. If anything is currently selected, it will be the default text in Find/Replace.
- Ctrl + G launches the "Go To Line" window. Allows quick navigation to a specific line in your file.
- Integrated help system: select something in the editor, right-click -> Help.
- Integrated debug support: select a variable in the editor, right-click -> Add Watch

2.4 Output Window / Output Window Toolbar

Any output from the Python "print" statement will be directed here. Any run-time errors, syntax errors etc. will also be reported here.

You can double-click on a line number in the output window (for example as part of an error message) and the editor will automatically move the cursor to that line. Similarly, if you double-click a function name, the editor will try and locate that function in the current file and place the cursor at that function.

The Clear button in the toolbar clears its contents. The Stop button will be enabled only whilst a script is executing, and can be used to abort the currently running script.

2.5 Help Window

Provides output from Python's integrated help function. Highlight an API function in the editor and right-click -> Help to invoke Help. Note that the Help system may need to be told which module a function is in, for best results a function call will need to be prefixed with its module / parent class. For instance, to see help on a profile function called `is_yearly`, run `help on iesve.GroupProfiles.is_yearly`.

2.6 Watch Window

To add a variable to the Watch window, type or paste its name into the first column, or select the variable in the editor and right-click-> Add. To remove a Watch variable, delete its name, or press the "Clear" button when the Watch tab is active.

To see detailed variable content, the Watch window supports copy to Clipboard (Ctrl + C, or right-click -> Copy). A watched variable is displayed red if its value has changed since last run. Greyed out variables are not recognized or not available.

When debugging and paused on a breakpoint, you can stop the current script from executing by pressing the Stop button (output toolbar).

2.7 Breakpoints

Click to the right of line numbers, or press F9 on a line, to set / remove breakpoints. Breakpoints are only active when running a script through the Debug command. (Shift + F5).

In Debug mode, the active line of execution is indicated with a yellow arrow. Use the **Watches** window to view variables / contents.

Editing a script whilst debugging will confuse everyone and cause line number mismatches with breakpoints.

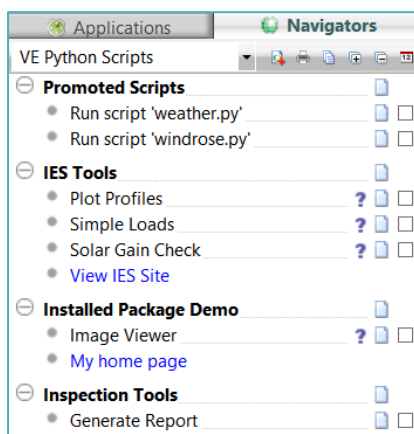
Keyboard shortcuts when in Breakpoint / debugging paused mode:

- F11 = step into Function (only inside the source code of main script being run)
- Shift + F11 = step out of Function
- F12 = Step to next line
- F5 = continue running to end or next breakpoint

3 Running Scripts

In addition to running scripts from the built-in editor, the VE Python Scripts navigator allows the user to run:

- Any script “Promoted to Navigator” from the editor
- Any script installed from an installable script package



3.1 Promote to Navigator

You can manually add scripts to the Python Scripts navigator using the built-in editor, File View. Right-click on the script you wish to add, and select Promote to Navigator. The script is now added to the navigator, and can be launched from there.

3.2 Installing a Script Package

Scripts can be added to the VE through installable packages. A package can contain any number of scripts, as well as help files, hyperlinks, and any other files required for a script to run (data files, data templates, etc.).

An installed package is automatically added to the VE Python Scripts navigator. To install a package, select Tools menu -> Manage Script(s) to open the script package manage. Click the install button and select your script package using the file browser. The package will then be installed.

3.3 Creating a Script Package

Please contact IES for more information on creating installable script packages.

Script packages can be subject to licensing controls if so desired (licensed to an individual user, organization, etc.).

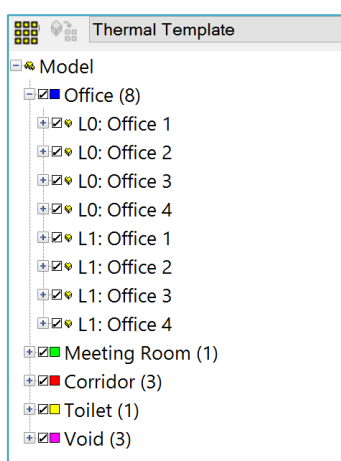
Scripts inside a package can optionally be encrypted for distribution.

4 Creating a Sample Script

A common task with VE Python scripting is to extract room level data results from an aps file. Many of the methods of the 'ResultsReader' class in the VE Python API, require 'room_id' to be passed in as an argument.

In most cases we would want to use room groups in the VE to specify which rooms we would like to include in our analysis. For example we might want to extract the room air temperatures from each space in our model in order to perform an overheating check. We would however, only be concerned with the temperature in certain space types i.e. 'Offices', 'Meeting Rooms', etc. We would want to exclude spaces types such as 'Voids' from our analysis. In this case we would use room groups to specify which room types we would like to be included in our analysis.

In this section we will walk-through the steps involved in creating a simple script, to interrogate the grouping schemes in our model. We will then use this to generate a list of all of the room ids in our model that's exclude the rooms in the 'Void' group. Before we begin, we must manually create a grouping scheme in our model called 'Thermal Template'. This grouping scheme contains the groups shown below.



Once we start a new script, the first task is to import the VE API into the project.

```
1 import iesve
```

Create an instance of the RoomGroups interface object using 'iesve.RoomGroups()' and assign it to a variable. The variable can be named anything. In this example we will assign it to the variable named 'rg'. The RoomGroups interface object will allow us to interact with the Grouping Schemes in the VE model.

```
1 import iesve
2
3 rg = iesve.RoomGroups()
```

We will now create a list of the grouping schemes contained in our model. Each individual grouping scheme will be represented as a Python Dictionary object. Each grouping scheme dictionary has the following entries:

Key	value type
handle	(int)
name	(string)

We will essentially be creating a list of dictionaries. To do this we will call the 'get_grouping_schemes()' method on the RoomGroups object. We will then assign the list of grouping schemes to another variable. In this example we will name this variable 'schemes'.

```
1 import iesve
2
3 rg = iesve.RoomGroups()
4 schemes = rg.get_grouping_schemes()
```

Using a 'for loop' we will iterate through our list of grouping schemes and print the name of each grouping scheme to the console.

```
1 import iesve
2
3 rg = iesve.RoomGroups()
4 schemes = rg.get_grouping_schemes()
5
6 for scheme in schemes:
7     print('~~~~~')
8     print(scheme['name'])
```

We will now create a list of the groups contained within each grouping scheme. Firstly we declare a new variable inside the loop and assign the scheme handle to it. We will call this variable 'scheme_handle'. We then call the method 'get_room_groups()' on our RoomGroups interface object and pass in the scheme handle as an argument. We assign this list of groups to a new variable which we have called 'groups'.

```
1 import iesve
2
3 rg = iesve.RoomGroups()
4 schemes = rg.get_grouping_schemes()
5
6 for scheme in schemes:
7     print('~~~~~')
8     print(scheme['name'])
9
10     scheme_handle = scheme['handle']
11     groups = rg.get_room_groups(scheme_handle)
```

Again each individual group in the list is represented as a Python Dictionary object which contains data the about the group. Each grouping scheme dictionary has the following entries:

Key	value type
colour	(tuple of R,G,B ints) each colour entry in range 0-255
handle	(int)
name	(string)
rooms	(list of strings) list of room IDs assigned to group

Finally we iterate through the list of groups using another for loop. We then print the name of each group and a list of the room ids for every room contained in each group.

```

1 import iesve
2
3 rg = iesve.RoomGroups()
4 schemes = rg.get_grouping_schemes()
5
6 for scheme in schemes:
7     print('~~~~~')
8     print(scheme['name'])
9
10    scheme_handle = scheme['handle']
11    groups = rg.get_room_groups(scheme_handle)
12
13    for group in groups:
14        print(group['name'])
15        print(group['rooms'])

```

We want to obtain a list of the room ids for all of the rooms in our model excluding the rooms in the 'Void' group. Firstly we declare a new variable before the first for loop. We call it 'room_ids' and assign it to an empty list. Next inside the second for loop we create an 'if' statement. The if statement below reads as follows: if the grouping scheme name is equal to 'Thermal Template' and the group is not equal to 'Void', then add the room ids for that group into the 'room_ids' list. This means that the rooms ids for all of the other groups will be added to the 'room_ids' list. Finally we print the 'room_ids' list to the console to manually verify that our script is working correctly.

```

1 import iesve
2
3 rg = iesve.RoomGroups()
4 schemes = rg.get_grouping_schemes()
5
6 room_ids = []
7 for scheme in schemes:
8     print('~~~~~')
9     print(scheme['name'])
10
11    scheme_handle = scheme['handle']
12    groups = rg.get_room_groups(scheme_handle)
13
14    for group in groups:
15        print(group['name'])
16        print(group['rooms'])
17        if scheme['name'] == 'Thermal Template' and group['name'] != 'Void':
18            room_ids += group['rooms']
19
20 print('Room Ids')
21 print(room_ids)

```

We have now created a list of room ids which could be used for many different tasks within the VE Python API. VE Scripts >> api_examples contains many other example scripts to demonstrate other useful functions of the VE Python API.

5 Troubleshooting Error Messages

Where a Python Script contains an error, it will typically be flagged to the user in the console at runtime. Python error messages are typically very helpful and make debugging Python code relatively straightforward. The console will typically display the line number of the problem code, as well as a '^' symbol showing the user where it thinks the problem has occurred. It is not always correct, however it will usually point the user in the right direction. In many cases the most effective approach to solving an error in Python code is to copy and paste the error message into a search engine such as Google. This section will detail some of the common Python errors that beginners may encounter.

5.1 SyntaxError: Invalid Syntax

The most common error a beginner will encounter is a syntax error. Syntax errors can have many different causes but are usually the result of a typo. They are typically very easy for the Python interpreter to find and easy for the user to correct.

Code	Error Message	Solution
<pre> 7 for scheme in schemes 8 print('~~~~~') 9 print(scheme['name']) </pre>	<pre> for scheme in schemes ^ SyntaxError: invalid syntax </pre>	<pre> 7 for scheme in schemes: 8 print('~~~~~') 9 print(scheme['name']) </pre>

We need a ':' (colon) at the end of any 'if', 'else', 'elif', 'for', 'while', 'class', or 'def' statement

Code	Error Message	Solution
<pre> 17 if scheme['name'] = 'Thermal Template': 18 room_ids += group['rooms'] </pre>	<pre> File "C:\Scripts\example.py", line 17 if scheme['name'] = 'Thermal Template': ^ SyntaxError: invalid syntax </pre>	<pre> 17 if scheme['name'] == 'Thermal Template': 18 room_ids += group['rooms'] </pre>

Used the assignment operator '=' instead of a comparison operator '=='

5.2 IndentationError: Unexpected Indent

Indentation should only take place after a statement ending with a colon ':'. The Python interpreter will return an indentation error if there is no indentation after a colon ':' or the code is indented where it should not be.

Code	Error Message	Solution
<pre> 1 import iesve 2 3 rg = iesve.roomGroups() 4 schemes = rg.get_grouping_schemes() </pre>	<pre> schemes = rg.get_grouping_schemes() ^ IndentationError: unexpected indent </pre>	<pre> 1 import iesve 2 3 rg = iesve.roomGroups() 4 schemes = rg.get_grouping_schemes() </pre>

Code on line 4 should not have been indented as there is no colon ':' at the end of line 3

5.3 TypeError: Can't convert 'int; object to str implicitly

A type error can occur if the type of an object is not what the Python interpreter expected to see for example if you try to concatenate (link) a non-string value to a string value.

Code	Error Message	Solution
<pre>1 number = 5 2 room_name = 'Office' 3 4 print(room_name + number)</pre>	<pre>Traceback (most recent call last): File "C:\Scripts\example.py", line 4, in <module> print(room_name + number) TypeError: Can't convert 'int' object to str implicitly</pre>	<pre>1 number = 5 2 room_name = 'Office' 3 4 print(room_name + str(number))</pre>

The int variable 'number' must be explicitly converted to a string before it can be concatenated to the string 'room_name'.

5.4 KeyError: 'room_name'

Python raises a key error whenever a dictionary object is requested using a key that does not exist in that dictionary.

Code	Error Message	Solution
<pre>14 for group in groups: 15 print(group['room_name'])</pre>	<pre>Traceback (most recent call last): File "C:\Scripts\example.py", line 15, in <module> print(group['room_name']) KeyError: 'room_name'</pre>	<pre>14 for group in groups: 15 print(group['name'])</pre>

The key 'room_name' does not exist in the dictionary group. The key 'name' does exist. The VE Scripts User Guide details the keys for every dictionary object in the VE Python API.

5.5 AttributeError: 'RoomGroups' object has no attribute 'get_room_groupz'

An attribute error will occur if a method that does not exist is called on an object. It can often be as a result of a simple typo in the method name. The VE Scripts User Guide details all of the methods for each object in the VE Python API.

Code	Error Message	Solution
<pre>7 for scheme in schemes: 8 print('~~~~~') 9 print(scheme['name']) 10 11 scheme_handle = scheme['handle'] 12 groups = rg.get_room_groupz(scheme_handle)</pre>	<pre>Traceback (most recent call last): File "C:\Scripts\example.py", line 12, in <module> groups = rg.get_room_groupz(scheme_handle) AttributeError: 'RoomGroups' object has no attribute 'get_room_groupz'</pre>	<pre>7 for scheme in schemes: 8 print('~~~~~') 9 print(scheme['name']) 10 11 scheme_handle = scheme['handle'] 12 groups = rg.get_room_groups(scheme_handle)</pre>

A typo in the method name. It should be get_room_groups() not get_room_groupz()'

5.6 ArgumentError

An argument error occurs if an incorrect number of arguments are passed into a method, the arguments are of the wrong type or the arguments are in the wrong order. In this example Python interpreter returns a more complex error message, but it still points the user to line 12 where the error has occurred.

Code	Error Message	Solution
<pre> 7 for scheme in schemes: 8 print('~~~~~') 9 print(scheme['name']) 10 11 scheme_handle = scheme['handle'] 12 groups = rg.get_room_groups() </pre>	<pre> Traceback (most recent call last): File "C:\Scripts\example.py", line 12, in <module> groups = rg.get_room_groups() Boost.Python.ArgumentError: Python argument types in roomGroups.get_room_groups(roomGroups) did not match C++ signature: get_room_groups(class RoomGroups (lvalue), int) </pre>	<pre> 7 for scheme in schemes: 8 print('~~~~~') 9 print(scheme['name']) 10 11 scheme_handle = scheme['handle'] 12 groups = rg.get_room_groups(scheme_handle) </pre>

'get_room_groups()' method takes one argument of the type int. The VE Scripts User Guide details all of the arguments for all methods for all objects in the VE Python API.

5.7 Other Resources

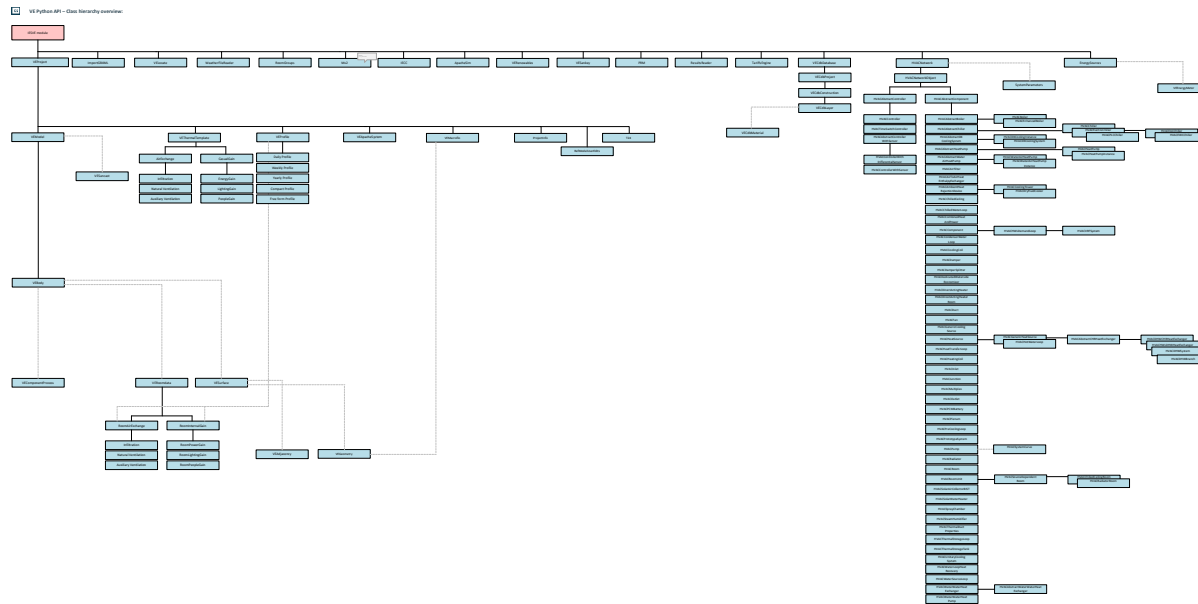
Many free online resources to help with troubleshooting:

- <https://inventwithpython.com/blog/2012/07/09/16-common-python-runtime-errors-beginners-find/>
- <https://www.toptal.com/python/top-10-mistakes-that-python-programmers-make>
- <http://pythonforbiologists.com/index.php/29-common-beginner-python-errors-on-one-page/>
- <http://www.dummies.com/programming/python/the-8-most-common-python-programming-errors/>
- http://www.onlamp.com/pub/a/python/2004/02/05/learn_python.html

6 VE Python API

When running inside the VE, the python environment includes the <Virtual Environment> API. To use the VE API, import the module to your script like this:

```
import iesve
```



High res PDF version: <https://www.iesve.com/support/faq/pdf/vscriptsguide/iiesve-class-structure-summary.pdf>



6.1 Conventions Used in the VE API and User Guide

- Units
- All API methods return their data in Metric units, regardless of the VE Units display setting.
- The VE Python includes a unit conversion library, Pint, that can be used to assist with unit conversions to IP units where desired.
- The VEProject class exports an API to retrieve the VE display settings, facilitating conversion to the VE display convention.
- General

Exported functions are shown in **bold** (call them using () and supply the required parameters as documented in the API details below).

Exported attributes are shown in *italic* – these can be accessed directly (no brackets required).

6.1.1 Top Level Methods

6.1.1.1 Methods Defined Here

iesve.get_application_folder() -> string
Gets the location of the VE instance.

Basic usage:

```
import iesve
iesve.get_application_folder()
```

6.1.2 AirExchange

Represents Air Exchange template data for the following:

- Infiltration
- Natural Ventilation
- Auxiliary Ventilation

Used by the [VEThermalTemplate](#) class.

6.1.2.1 Methods Defined Here

get() -> dictionary

Returns a dictionary containing the data for an air exchange. The following fields are returned:

<i>adjacent_condition_string</i>	(string)
<i>adjacent_condition_val</i>	(int) see below for values
<i>max_flow</i>	(float)
<i>name</i>	(string)
<i>offset_temperature</i>	(float)
<i>temperature_profile</i>	(string) profile ID
<i>type_str</i>	(string) 'Infiltration', 'Natural Ventilation', or 'Auxiliary ventilation'
<i>type_val</i>	(int) see below for values
<i>units_str</i>	(string) 'ach', 'l/s', 'l/s/m²', 'l/s/person', 'l/(s.m² fac)'
<i>units_val</i>	(int) see below for values
<i>variation_profile</i>	(string) profile ID

Valid values for *adjacent_condition_val*:

- 1 = External air
- 2 = External air + Offset Temperature
- 3 = From adjacent room
- 5 = Temperature from profile

Valid values for *type_val*:

- 0 = Infiltration
- 1 = Natural Ventilation
- 2 = Auxiliary Ventilation

Valid values for *units_val*:

- 0 = ach
- 1 = l/s
- 2 = l/s/m²
- 3 = l/s/person
- 4 = l/(s.m² façade)

6.1.2.2 Attributes Defined Here

<i>name</i>	(string) Descriptive string
-------------	-----------------------------

6.1.2.3 Enums Defined Here

<code>AdjacentCondition_type</code>	<code>external_air</code> , <code>external_air_and_offset_temp</code> , <code>from_adjacent_room</code> , <code>temperature_from_profile</code>
<code>AirExchange_type</code>	<code>infiltration</code> , <code>mechanical_ventilation</code> , <code>natural_ventilation</code>

6.1.3 ApacheSim

Interface for running Apache Simulations.

Basic usage:

```
s = iesve.ApacheSim()
s.show_simulation_dialog()
```

or

```
s = iesve.ApacheSim()
s.save_options({'results_filename': 'abc.aps'})
s.run_simulation()
```

6.1.3.1 Methods Defined Here

get_options() -> dictionary

Get the simulation options currently in effect. Return type is a dictionary with the current options.
Available options are:

<code>aux_ventilation</code>	(Bool) defaults to True
<code>cooling_end_month</code>	(int) 1-12
<code>cooling_start_month</code>	(int) 1-12
<code>end_day</code>	(int) 1-31
<code>end_month</code>	(int) 1-12
<code>HVAC</code>	(Bool) include HVAC
<code>HVAC_filename</code>	(string) HVAC network filename, used if HVAC is True
<code>macroflo</code>	(Bool) setting this to True requires a macroflo input file (.mfo) to be present. Setting this to True is required for opening-level results
<code>nat_ventilation</code>	(Bool) defaults to True
<code>preconditioning_days</code>	(int) 0-365
<code>radiance</code>	(Bool) setting this to True will automatically run Radiance if <code>queue_to_tasks</code> is True when running simulations. It will not run automatically if a blocking simulation is run, in which case an existing Radiance results file (.ill) is required
<code>reporting_interval</code>	(int) 0=6, 1=10, 2=30, 3=60 minutes
<code>results_filename</code>	(string) file will be located in Vista folder
<code>simulation_timestep</code>	(int) 0=1, 1=2, 2=6, 3=10, 4=30 minutes
<code>start_day</code>	(int) 1-31
<code>start_month</code>	(int) 1-12
<code>suncast</code>	(Bool) setting this to True will run Suncast if required
<code>suncast_filename</code>	(string) shading file filename

Output options. These correspond to the Apache Simulation Output Options dialog and apply to all rooms:

<code>output_conduction_gains</code>	(Bool) defaults to False
<code>output_HVAC_components</code>	(Bool) defaults to False
<code>output_HVAC_systems</code>	(Bool) defaults to True
<code>output_latent_internal_gains</code>	(Bool) defaults to False
<code>output_latent_ventilation_gains</code>	(Bool) defaults to False
<code>output_sensible_internal_gains</code>	(Bool) defaults to True
<code>output_standard_outputs</code>	(Bool) defaults to True

Detailed output options. These correspond to the Apache Simulation Output Options dialog and apply only to the rooms indicated in `detailed_rooms`. Note that detailed output per room can significantly increase APS file size if a large number of rooms are included.

<code>detailed_conduction_gains</code>	(Bool) defaults to False
<code>detailed_convective_gains</code>	(Bool) defaults to True
<code>detailed_external_solar</code>	(Bool) defaults to False. Setting this for a room enables Surface level results for that room
<code>detailed_HVAC_systems</code>	(Bool) defaults to True
<code>detailed_internal_solar</code>	(Bool) defaults to False. Setting this for a room enables Surface level results for that room
<code>detailed_latent_internal_gains</code>	(Bool) defaults to True
<code>detailed_latent_ventilation_gains</code>	(Bool) defaults to True
<code>detailed_microflo</code>	(Bool) defaults to True
<code>detailed_rooms</code>	(list of strings) each string holds a room ID
<code>detailed_sensible_internal_gains</code>	(Bool) defaults to True
<code>detailed_standard_outputs</code>	(Bool) defaults to True
<code>detailed_surface_temps</code>	(Bool) defaults to True. Setting this for a room enables Surface level results for that room

Note that setting Suncast and Radiance to True will only auto-run suncast when using batch operation. When using the blocking run, suncast and radiance will not be automatically run when set to True, but if corresponding data files are available (shd and ill files) they will be used in simulation. Absence of data files may cause simulation to fail.

reset_options() -> Bool

Reset the simulation options file to defaults. Returns True if the options file was saved successfully.

run_compliance_simulation() -> Bool

Run compliance simulation with the currently active settings. Parallel Simulation Manager (BETA) will be used. A return value of true indicates that simulation was run / queued, false indicates error.

run_room_zone_loads()

Run a Room loads simulation OR

Run a Room and HVAC Zone loads simulation

run_loads_sizing()

Run a HVAC system loads & sizing simulation

run_simulation(queue_to_tasks = False) -> Bool

Run Apache thermal simulation with the currently active settings. If the queue_to_tasks parameter is True, Parallel Simulation Manager (BETA) will be used (if licensed and activated). The task will be queued and script operation will continue immediately. When not using queueing, the simulation will run and the VE is blocked until simulation is complete. A return value of true indicates that simulation was run / queued, false indicates error.

set_hvac_network(network_name)

Set the HVAC network.

set_options({options}) -> Bool

Set and save simulation options. These options will be used for the subsequent simulation run. See get_options() for details on available options (keywords). Returns True if the options file was saved successfully.

Note: the options to be set can be supplied either as a dict or as keyword parameters.

show_simulation_dialog() -> Bool

Show the Apache Simulation dialog. Returns True if dialog was shown and simulation was run.

stitch(output_file_path, files_to_stitch)

Stitch together a list of APS files into a single APS file.

Parameters:

<i>output_file_path</i> (string)	Path of the output APS file
<i>files_to_stitch</i> (list)	Ordered list of APS file to stitch

6.1.3.2 Enums Defined Here

<i>time_step</i>	one_minute, two_minutes, six_minutes, ten_minutes, thirty_minutes
<i>reporting_interval</i>	six_minutes, ten_minutes, thirty_minutes, sixty_minutes

6.1.4 BpfCustom

Interface for custom building performance factors data.

6.1.4.1 Methods Defined Here

get() -> dictionary

Get custom building performance factors.

Returns a dictionary with the following keys:

MutltiFamily	(list) of floats ordered by climate zone: see table below.
Healthcare/hospital	"
Hotel	"
Office	"
Restaurant	"
Retail	"
School	"
Warehouse	"
Other	"

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Climate Zone	0A	1A	0B	1B	2A	2B	3A	3B	3C	4A	4B	4C	5A	5B	5C	6A	6B	7	8

6.1.4.2 Attributes Defined Here

<i>name</i>	(str) Name of the current BPF preset.
-------------	---------------------------------------

6.1.5 CasualGain

Internal Gain data. Returned from [VEThermalTemplate](#) function `get_casual_gains()`. Three categories of internal gains are defined (represented by individual classes):

- EnergyGain
- LightingGain
- PeopleGain

6.1.5.1 Methods Defined Here

get () -> dictionary

Returns a dictionary containing the data for the internal gain.

EnergyGain:

<i>energy_source</i>	(string) Fuel name
<i>max_latent_gain</i>	(float) see <i>units_str</i> / <i>units_val</i> for units
<i>max_power_consumption</i>	(float) see <i>units_str</i> / <i>units_val</i> for units
<i>max_sensible_gain</i>	(float) see <i>units_str</i> / <i>units_val</i> for units
<i>name</i>	(string)
<i>radiant_fraction</i>	(float)
<i>type_str</i>	(string) Gain type: 'Machinery', 'Miscellaneous', 'Cooking', 'Computers'
<i>type_val</i>	(enum: <i>EnergyGain_type</i>) see below for valid enum values
<i>units_str</i>	(string) 'W/m²' or 'W'
<i>units_val</i>	(int) 0 = W/m², 1 = W
<i>variation_profile</i>	(string) profile ID

LightingGain:

<i>ballast</i>	(float)
<i>design_illuminance</i>	(float) Design Illuminance
<i>dimming_profile</i>	(string) profile ID
<i>diversity_factor</i>	(float)
<i>energy_source</i>	(string) Fuel name
<i>maximum_illuminance</i>	(float) Maximum Illuminance
<i>max_power_consumption</i>	(float) see <i>units_str</i> / <i>units_val</i> for units
<i>max_sensible_gain</i>	(float) see <i>units_str</i> / <i>units_val</i> for units
<i>minimum_illuminance</i>	(float) Minimum Illuminance
<i>name</i>	(string)
<i>radiant_fraction</i>	(float)
<i>type_str</i>	(string) 'Fluorescent Lighting' or 'Tungsten Lighting'
<i>type_val</i>	(enum: <i>LightingGain_type</i>) see below for valid enum values
<i>units_str</i>	(string) 'W/m²', 'W', 'lux'
<i>units_val</i>	(int) 0 = W/m², 1 = W, 2 = lux
<i>variation_profile</i>	(string) profile ID

PeopleGain:

<i>diversity_factor</i>	(float)
<i>max_latent_gain</i>	(float, W)
<i>max_sensible_gain</i>	(float)
<i>name</i>	(string)
<i>number_of_people</i>	(float) only valid/present if <i>units_str</i> / <i>units_val</i> are: people
<i>occupancy_density</i>	(float) only valid/present if <i>units_str</i> / <i>units_val</i> are: m ² /person
<i>type_str</i>	(string) 'People'
<i>type_val</i>	(enum: PeopleGain_type) see below for valid enum values
<i>units_str</i>	(string) 'm ² /person', 'people'
<i>units_val</i>	(int) 0 = m ² /person, 1 = people
<i>variation_profile</i>	(string) profile ID

6.1.5.2 Attributes Defined Here

<i>name</i>	(string) Descriptive string
-------------	-----------------------------

6.1.5.3 Enums Defined Here

<i>EnergyGain_type</i>	machinery, miscellaneous, cooking, computers, data_centre_equipment, refrigeration, process_equipment, local_fans, transformers, motors
<i>LightingGain_type</i>	fluorescent, tungsten, general, task, display, process, unknown
<i>PeopleGain_type</i>	people

6.1.6 EnergySources

Interface for energy sources

6.1.6.1 Static Methods Defined Here

get_all_energy_source_data() -> list

Get all energy source data as a list of dictionaries using averaged source and carbon factors if using monthly or hourly modes.

Dictionary entries are:

carbon_emissions_factor

fuel_code

id

name

source_energy_factor

get_all_energy_source_data_non_averaged(aps_results_data_points (optional)) -> list

Get all energy source data as a list of dictionaries using non-averaged source and carbon factors (Assumes 8760 timesteps as a default for `aps_results_data_points`).

Dictionary entries are:

carbon_emissions_factor

fuel_code

id

name

source_energy_factor

get_energy_source_data_by_id(id) -> dictionary

Get energy source data as a dictionary by ID.

Dictionary entries are:

carbon_emissions_factor

fuel_code

id

name

source_energy_factor

get_energy_meter(source_id, meter_id) -> [VEEnergyMeter](#)

Get energy meter ([VEEnergyMeter](#)).

6.1.7 HVAC Network

6.1.7.1 Enums

Ashrae621Version	ashrae621_2004, ashrae621_2007, ashrae621_2010, ashrae621_2013
CoilLocationCircuit	primary, secondary
CoolingCoilModelType	simple_cooling_coil, advanced_cooling_coil
DehumidificationControlOption	per_system_level_dry_bulb_or_dew_point_control_only, system_supply_air_conditioned_per_zone_relative_humidity_sensors
DesignZoneTemperatureSource	design_zone_air_temperature, temperature_setpoint
ExhaustMakeUpAirOption	make_up_air_is_hundred_percent_primary_air, make_up_air_is_hundred_percent_transfer_air, transfer_air_makes_up_difference, make_up_air_includes_fixed_percentage_transfer_air
GarageFanControlScheme	on_occupied, on_occupied_plus_extension, on_continuously, reduce_airflow_unoccupied, modulate_airflow
HeatingCoilModelType	simple_heating_coil, advanced_heating_coil
HeatingSourceSystemType	heating_system_generic, heating_system_hot_water_loop, heating_system_aa_heat_pump, heating_system_wa_heat_pump, heating_system_heat_transfer_loop, heating_system_electric, heating_system_vrf, heating_system_aa_heat_pump_advanced
HumidificationControlOption	per_system_level_relative_humidity_control_only, system_supply_air_humidified_per_zone_relative_humidity_sensors
HVACACCCalculationStatus	autocalc_ready, autocalc_complete, notposs_contactfactor, notposs_error1, notposs_error2, notposs_error3, notposs_error4, notposs_error5, notposs_error6, notposs_error7, autocalc_warning8, autocalc_notposs_error9, autocalc_notposs_error_unknown, mancalc_ready, mancalc_complete, mancalc_notposs_error1, mancalc_notposs_error2, mancalc_notposs_error3, mancalc_notposs_error4, mancalc_notposs_error5, mancalc_notposs_error6, mancalc_notposs_error7, mancalc_notposs_error8, mancalc_notposs_error9, mancalc_notposs_error10, mancalc_notposs_error_unknown
HVACACCSizingMode	autosizing, manual, both
HVACBoilerType	hot_water_boiler, part_load_curve_boiler
HVACChillerType	CoolingChiller, DischargeChiller, ElectricAirCooled, ElectricWaterCooled, PartLoadCurve
HVACComponentTypes	undefined, room, simple_heating_coil, simple_cooling_coil, spray_humidifier, steam_humidifier, temp_flow_control, junction, duct, equip_heat_gain, heat_recov, terminal_reheat, terminal_recool, boiler, chiller, fan, damper, versattemp, unitary_cooling, waterside_econ, ewc_chiller, hwb_boiler, adiabatic_duct, eac_chiller, dx_cooling, cooling_coil, chilled_water_loop, advanced_heating_coil, hot_water_loop, generic_heat_source, awhp, aahp, solar_water_heater, heat_transfer_loop, wahp, generic_cooling_source, room_radiator, room_chilled_ceiling, condenser_water_loop,

	cooling_tower, fluid_cooler, pre_cooling, water_source_loop, water_water_hx, pcm_thermal_battery, pump, vrf_indoor_heat, vrf_indoor_cool, vrf_indoor_type, vrf_outdoor, solar_aircollector_bist, air_filter, plenum, thermal_storage_loop, thermal_storage_tank, tsd_virtual_chiller, tsc_virtual_chiller, hwl_demand_loop, zone, dhw_system_loop, dhw_branch, dhw_zone, eahp, eahp_heating_instance, eahp_cooling_instance, heat_recovery_loop, heat_recovery_hx, central_plant_heat_pump
HVACCoolingSourceSystemType	legacy, chilled_water_loop, unitary_cooling, waterside_economizer, direct_expansion, water_air_heat_pump, generic_cooling_source, variable_refrigerant_flow, aa_heat_pump_advanced
HVACDefrostControl	on_demand, timed
HVACDefrostStrategy	resistive, reverse_cycle
HVACDemandLoopType	primary_demand, secondary_demand
HVACDHWModel	explicit, simple
HVACDHWTankDetails	insulation_details, heat_loss
HVACDHWTankInsulationType	uninsulated, loose_jacket, factory_insulated
HVACDHWTemperatureSetPointType	constant, profile
HVACDryFluidCoolerMode	dual_mode, dry_mode
HVACDXCoolingCondenserType	air_cooled, evaporative_cooled, number_of_types
HVACDXFanControl	one_speed_fan, two_speed_fan, vsd_fan
HVACHeatPumpType	air_source_heat_pump, air_water_heat_pump, air_air_heat_pump
HVACHeatRejectionDevice	cooling_tower, dry_fluid_cooler, no_ambient_hr
HVACHeatRecoveryModel	basic, percentage_hr, explicit
HVACLocation	duct_ext_to_building, duct_contained_within_room, duct_contained_within_void, duct_contained_within_ra_plenum, duct_contained_within_sa_plenum, duct_contained_within_zone
HVACLoopConfiguration	primary, primary_secondary
HVACMultiplexLayerEditingMode	current, all_selected
HVACNetworkObjectType	not_set, generic, component, controller, multiplex, prototype_system, waterside_container
HVACOutdoorUnitControl	load_priority, load_priority_with_compressor_power, zone_priority, master_thermostat_priority, thermostat_offset_priority, profile
HVACPrototypeSystemType	prm_system_1, prm_system_2, prm_system_3, prm_system_4, prm_system_5, prm_system_6, prm_system_7, prm_system_8, prm_system_9, prm_system_10, actimass_system, tata_renew_sc_system, cool_phase_system
HVACRecirculationScheduleType	space_occupancy, time_switch_profile
HVACSHWPanelType	flat_panel, parabolic_panel
HVACSysClassification	no_classification, standard_design_system, t24_design_system, t24_standard_design_system, ncm_design_system, prm_2004_compliance_system, prm_2007_compliance_system, prm_2010_compliance_system, prm_2013_compliance_system, prm_2016_compliance_system,

	prm_2019_compliance_system, ecb_2010_compliance_system, ecb_2013_compliance_system, necb_2011_compliance_system, necb_2017_compliance_system, iecc_2012_compliance_system
HVACSystemCurveRole	CHILL_FCAPT, CHILL_FEIRT, CHILL_FEIRPT, PUMP_PWRV, BOIL_FEPT, EAC_FCAPT, EAC_FEIRT, EAC_FEIRPT, DX_FCAPT, DX_FEIRT, DX_FEIRP, WAHP_FCAPT_HEAT, WAHP_FEIRT_HEAT, WAHP_FEIRP_HEAT, WAHP_FCAPT_COOL, WAHP_FEIRT_COOL, WAHP_FEIRP_COOL, AAHPH_FCAPT, AAHPH_FEIRT, AAHPH_FEIRP, AAHPH_FDEIRT, AAHPC_FCAPT, AAHPC_FEIRT, AAHPC_FEIRP, CPHPH_FCAPT, CPHPH_FEIRT, CPHPH_FCOPT, CPHPH_FEIRP, CPHPH_FCOPP, CPHPH_FDEIRT, CPHPC_FCAPT, CPHPC_FEIRT, CPHPC_FCOPT, CPHPC_FEIRP, CPHPC_FCOPP
HVACSystemCurveType	centrifugal, reciprocating, screw, scroll, other
HVACSystemLink	ACT_BEAM_IU_INDUCED_AIRFLOW, ACT_BM_IU_PRIMARY_AIR_CAV_VAV_COOL, ACT_BM_IU_PRIMARY_AIR_CAV_VAV_HEAT, ADJACENT_SPACE_WITH_EA_FAN, AHU_COOLING_COIL, AHU_HEATING_COIL, AHU_PRE_HEAT_COIL, AIRFLOW_ON_IF_NV_INSUF_COOL, AIRFLOW_ON_IF_NV_INSUF_VENT, CAV_AIRFLOW_NIGHTTIME_SETBACK, CAV_AIRFLOW_OCCUPIED_HOURS, COOLING_AIRFLOW_VAV_CAV, COOLING_COIL_AHU_COOL_LAT, COOLING_COIL_AHU_DEHUM_LAT, COOLING_COIL_AHU_EXTEND_DEHUM_LAT, COOLING_COIL_AHU_LOWTEMP_LAT, COOLING_COIL_AHU_SUPPLY_AIR_DPT, COOLING_COIL_COIL_LEAVING_DPT, COOLING_COIL_DOAS_DEHUM, COOLING_COIL_DOAS_FIXED_TEMP, COOLING_COIL_DOAS_TEMPERING, COOLING_COIL_FULL_COOLING_LAT_BAND, COOLING_COIL_FULL_DEHUM_LAT_BAND, COOLING_COIL_FULL_H_C_DEHUM_LAT, COOLING_COIL_FULL_H_C_LAT_BAND, COOLING_COIL_ZONE_FIXED_LAT, COOLING_COIL_ZONE_VARIABLE_LAT, COOLING_COIL_ZONE_VARIABLE_LAT_NO_OFFSET, COOLING_ONLY_SYS_AIRFLOW, COOLING_THERMOSTAT, DCV_MIN_OUTSIDE_AIR_ECON, DEDICATED_OA_VENT_COOLING_COIL, DEDICATED_OA_VENT_HEATING_COIL, DELIMITER, DIRECT_ACTING_HEATER_COOLER, DIRECT_EVAP_COOLING, DIRECT_EVAP_SPRAY, DOAS_AHU_COOLING_DEVICE, DOAS_PTAC_PTHP_COOLING_AIRFLOW_CAV_2SP_VAV, DOAS_PTAC_PTHP_HEATING_AIRFLOW_CAV_2SP_VAV, DOAS_VENT_AIRFLOW, DOAS_VENT_AIRFLOW_CAV_DCV, DUAL_FAN_DUAL_DUCT_ZN_HEAT_AIRFLOW, DUAL_FAN_DUAL_DUCT_ZONE_COOL_AIRFLOW_AND_OA_VE NT, EA_PERCENT_AVAILABLE_TO_ER, EA_PERCENT_VS_RETURN_OR_TRANSFER, ELEC_HEAT_COIL_2ND_STAGE, ENERGY_RECOVERY_BYPASS_DAMPER, ENERGY_RECOVERY_BYPASS_LEGACY_DAMPER,

ENERGY_RECOVERY_HX_WHEEL, ER_BYPASS_TEMP_TARGET,
 ER_BYPASS_TEMP_TARGET_LEGACY_CONFIG,
 ER_BYPASS_TEMP_TARGET_PSZ,
 ER_BYPASS_TEMP_TARGET_PSZ_LEGACY,
 ER_TARGET_COOL_MODE, ER_TARGET_HEAT_MODE,
 ER_TARGET_PSZ_HEAT_COOL,
 EVAP_SPRAY_HUMID_PER_ZN_RH,
 EXHAUST_DRIVEN_VENTILATION_AIRFLOW, EXHAUST_FAN,
 FCU_ACT_BEAM_COOL_COIL, FCU_ACT_BEAM_HEAT_COIL,
 FCU_COOLING_AIRFLOW_CAV_2SP_VAV, FCU_FAN,
 FCU_HEATING_AIRFLOW_CAV_2SP_VAV,
 FILTER_PRESSURE_AND_CHANGE, FPB_FAN,
 FPB_PRIMARY_AIRFLOW_CAV_VAV,
 FPB_SECONDARY_AIRFLOW_CAV_2SP_VAV,
 HEATED_COOLED_SLAB_ZONE, HEATING_AIRFLOW_VAV_CAV,
 HEATING_COIL_AHU_MIN_SAT,
 HEATING_COIL_DOAS_TEMPERING,
 HEATING_COIL_FULL_HEATING_LAT_BAND,
 HEATING_COIL_FULL_H_C_LAT_BAND,
 HEATING_COIL_MIN_SAT_WITH_RESET,
 HEATING_COIL_ZN_FIX_LAT_STAGE_1,
 HEATING_COIL_ZN_FIX_LAT_STAGE_2,
 HEATING_COIL_ZONE_FIXED_LAT,
 HEATING_COIL_ZONE_VARIABLE_LAT,
 HEATING_COIL_ZONE_VARIABLE_LAT_NO_OFFSET,
 HEATING_ONLY_SYS_AIRFLOW, HEATING_THERMOSTAT,
 HEAT_PIPE_RR_WHL_BYPASS_DAMPER,
 HEAT_PIPE_RUN_RND_WHEEL,
 HP_RRcoil_WHL_BYPASS_TEMP,
 HP_RR_WHL_BYPASS_TEMP_DIFF, INDIRECT_EVAP_COOLING,
 INDIRECT_EVAP_SPRAY, MIN_FAN_AIRFLOW,
 MIXED_MODE_DISABLE, MIXED_MODE_TOUT_TZN_DELTA,
 MODULATION_BY_LOAD, NONE, OA_ECON_DBT_HIGH_LIMIT,
 OA_ECON_DIFF_ENTHALPY, OA_ECON_DPT_LIMIT,
 OA_ECON_TARGET_FULL_COOLING_LAT_BAND,
 OA_ECON_TARGET_PER_RA_TEMP,
 OA_ECON_TARGET_RESET, OA_ECON_ZN_TEMP_LOW_LIMIT,
 OA_FIXED_PERCENTAGE,
 OA_MIN_RESET_OCCUPIED_ZONE_CO2,
 OA_MIN_RESET_OCCUPIED_ZONE_VAV_FLOW_PERCENT,
 OA_MIN_RESET_UNOCCUPIED_ZN,
 OA_MIN_RESET_UNOCCUPIED_ZN_VAV_FLOW_PERCENT,
 OA_MIN_RESET_ZN_VAV_FLOW_PERCENT,
 OA_MIN_RESET_ZONE_CO2,
 OA_VARIABLE_MIN_PERCENTAGE, OCCUPIED_ZONE,
 ON_IF_TEMP_BELOW_MIN_SAT,
 OUTSIDE_AIR_FIXED_MIN_DAMPER,
 OUTSIDE_AIR_MIN_ECON_DAMPER,
 OUTSIDE_AIR_NO_MIN_ECON_DAMPER,
 OUTSIDE_AIR_VARIABLE_MIN_DAMPER,
 PACKAGE_TERMINAL_UNIT_FAN,
 PARKING_GARAGE_EXHAUST,
 PRE_HEAT_COIL_AHU_MIN_EAT,
 PSZ_COOLING_AIRFLOW_CAV_2SP_VAV,
 PSZ_HEATING_AIRFLOW_CAV_2SP_VAV,
 PTAC_PTHP_COOLING_COIL,

	PTAC_PTHP_COOL_AIRFLOW_CAV_2SP_VAV, PTAC_PTHP_COOL_AIRFLOW_CAV_2SP_VAV_NO_OFFSET, PTAC_PTHP_HEATING_COIL, PTAC_PTHP_HEAT_AIRFLOW_CAV_2SP_VAV, PTAC_PTHP_HEAT_AIRFLOW_CAV_2SP_VAV_NO_OFFSET, RAD_CONV_COOL_DEVICE_PANEL, RAD_CONV_HEAT_DEVICE_PANEL, RA_BYPASS_MIXED_AIR_TARGET_TEMP, RA_DUCT_HEAT_LOSS_GAIN, RA_PERCENT_TO_ALT_PATH_OR_ZONE, RA_PLENUM, RETURN_AIR_DAMPER, RETURN_RELIEF_FAN, SA_DUCT_HEAT_LOSS_GAIN, SA_UFAD_PLENUM, SERIES_FPB_DAMPER, STEAM_HUMIDIFIER, STEAM_HUMID_PER_ZN_RH, STEAM_HUMID_SA_RH, STRATIFIED_ZN_REMIX_AIRFLOW, SUPPLY_FAN, SUPPLY_FAN_TSP_ADDITION, THERMALLY_STRATIFIED_ZONE, UCS_COOLING_AIRFLOW, UCS_COOLING_COIL, UNIT_HEATER_FAN, VOID_SPACE, ZONE_AIRFLOW_DETECTION, ZONE_COOLING_DEVICE, ZONE_DCV_STAGE1_VAV_CTRL, ZONE_MIXING_BOX_DAMPER, ZONE_MIXING_FIXED_DBT_DAMPER_TARGET, ZONE_MIXING_PERCENT_PRIMARY_AIR, ZONE_MIXING_VARIABLE_DBT_TARGET, ZONE_RA_TRANSFER_AIRFLOW, ZONE_REHEAT_HEATING_COIL, ZONE_RH_HIGH_LIMIT_FOR_DEC
HVACVRFSOURCEType	air_source, water_source
HVACWaterWaterHeatExchangerData Type	cooling, heating
OccupantDiversity	room_data_diversity_factors, specify_independently, ashrae621_time_averaging
OutdoorAirRequirementLinkage	linked, maximum, sum
ProfileVariation	constant, profile, two_values, relative_to_zone_room
SpaceOutdoorAirRequirementBasis	ashrae_621, outdoor_requirement, ashrae_170, maximum, sum, title_24
StrategyOutsideAirVentilationMinReq	occupied_opening_hours_only, occupied_extension_hours, all_hours
StrategySystemFanOperation	on_continuously, cycling, off_continuously, any_strategy
SystemConfigurationOptions	package_thermal_unit, single_zone_system, multi_zone_doas, multi_zone_ahu
SystemOutdoorAirRequirementBasis	ashrae_621, sum_of_zone_outdoor_air_requirements
SystemSizingBypassOption	bypass_system_sizing, system_sizing_on
TerminalUnitFanControl	constant_fan_control, two_speed_fan_control, variable_fan_control
UnitType	no_conversion, distance_km, distance_m, distance_cm, distance_mm, volume_flow_rate_m3_per_person, volume_flow_rate_litres_per_person, temperature, heatflux_wm2, heatflux_w, coordinates, area_m2, volume_m3, heating_cooling_power_kw, heating_cooling_power_per_person, heating_cooling_power_per_person_short, volume_flow_rate_litres_per_person_m2, layer_thick, volume_litres, speed_metres_per_second, mass_kg, temperature_difference_positive, temperature_difference_positive_negative, energy_joules,

humidity_ratio, pressure, volume_flow, mass_flow, enthalpy,
 density, specific_heat_capacity, conductivity,
 heat_transfer_coefficient, conductance, resistance,
 electrical_power_area, heating_cooling_power_kw,
 heating_cooling_power_w, heating_cooling_power_person,
 heating_cooling_power_area, solar_flux, occupancy_density,
 volume_flow_area, condensation_rate, condensation, lumens,
 lux_lumens_m2, viscosity, luminous_intensity, luminance,
 enthalpy_kj, crack_flow, pressure_in_mercury,
 pressure_in_water, vapour_permeability, vapour_permeance,
 luminous_power_density, luminous_efficacy_watt,
 length_m_to_in, volume_flow_rate_m3_per_person_cfm,
 cost_area, formatting_only,
 heating_cooling_power_w_display_kw, specific_fan_power,
 dhw_consumption_hour_per_person_uk,
 dhw_consumption_hour_per_person_us, heatflux_watts,
 thermal_capacity_kjm2k, tank_loss, line_loss,
 second_order_heat_loss,
 volume_flow_rate_litres_per_hour_m2,
 temperature_coefficient, volume_litres_uk_gallons,
 volume_flow_rate_litres_per_hour_gallon, circulation_loss,
 storage_loss_us_gallon, volume_litres_per_us_gallons,
 hourly_volumetric_flow, parts_per_million, inverse_length,
 pressure_velocity_1, pressure_velocity_2, pressure_velocity_3,
 mass_flow_per_hour, energy_consumption_per_hour,
 flow_litres_per_second_gpm, flow_per_km, watts_per_flow,
 inverse_k, inverse_k2, enthalpy_difference,
 volume_m3_us_gallon, linear_heat_transfer_coefficient,
 flow_gpm_kbtu_per_hour, inverse_k3,
 sbem_heat_transfer_rate, flow_per_person,
 dhw_consumption_per_hour_us, electrical_power_length,
 volume_flow_rate_litres_per_second_m2_facade,
 kwh_m2_to_kbtu_ft2, inverse_area_m2,
 inverse_per_energy_consumption_kwh,
 length_m_display_mm, mass_tonne_short_ton, wind_speed,
 volume_flow_ach, occupancy_people, degrees, percent,
 volume_flow_air_changes_per_hour_short, land_area,
 land_density, mass_kg_per_m,
 volume_flow_m3_per_hour_per_m2_of_envelope,
 pressure_inches_of_water_column,
 condensation_rate_kg_m2_second, joules_ton_hours,
 gigajoules_ton_hours, gigajoules_ton_kwh,
 occupany_people_per_100_m2, volume_flow_per_unit,
 volume_flow_equals_supply_air,
 volume_flow_greater_than_supply_air,
 volume_flow_percentage_above_supply_air,
 heating_cooling_power_kw_area,
 area_heating_cooling_power, suspended_solid, area_cm2,
 temperature_half_ip, constant_fan_control,
 two_speed_fan_control, variable_fan_control, no_conversion,
 distance_km, distance_m, distance_cm, distance_mm,
 volume_flow_rate_m3_per_person,
 volume_flow_rate_litres_per_person, temperature,
 heatflux_wm2, heatflux_w, coordinates, area_m2, volume_m3,
 heating_cooling_power_kw,
 heating_cooling_power_per_person,
 heating_cooling_power_per_person_short,

volume_flow_rate_litres_per_person_m2, layer_thick,
 volume_litres, speed_metres_per_second, mass_kg,
 temperature_difference_positive,
 temperature_difference_positive_negative, energy_joules,
 humidity_ratio, pressure, volume_flow, mass_flow, enthalpy,
 density, specific_heat_capacity, conductivity,
 heat_transfer_coefficient, conductance, resistance,
 electrical_power_area, heating_cooling_power_kw,
 heating_cooling_power_w, heating_cooling_power_person,
 heating_cooling_power_area, solar_flux, occupancy_density,
 volume_flow_area, condensation_rate, condensation, lumens,
 lux_lumens_m2, viscosity, luminous_intensity, luminance,
 enthalpy_kj, crack_flow, pressure_in_mercury,
 pressure_in_water, vapour_permeability, vapour_permeance,
 luminous_power_density, luminous_efficacy_watt,
 length_m_to_in, volume_flow_rate_m3_per_person_cfm,
 cost_area, formatting_only,
 heating_cooling_power_w_display_kw, specific_fan_power,
 dhw_consumption_hour_per_person_uk,
 dhw_consumption_hour_per_person_us, heatflux_watts,
 thermal_capacity_kjm2k, tank_loss, line_loss,
 second_order_heat_loss,
 volume_flow_rate_litres_per_hour_m2,
 temperature_coefficient, volume_litres_uk_gallons,
 volume_flow_rate_litres_per_hour_gallon, circulation_loss,
 storage_loss_us_gallon, volume_litres_per_us_gallons,
 hourly_volumetric_flow, parts_per_million, inverse_length,
 pressure_velocity_1, pressure_velocity_2, pressure_velocity_3,
 mass_flow_per_hour, energy_consumption_per_hour,
 flow_litres_per_second_gpm, flow_per_km, watts_per_flow,
 inverse_k, inverse_k2, enthalpy_difference,
 volume_m3_us_gallon, linear_heat_transfer_coefficient,
 flow_gpm_kbtu_per_hour, inverse_k3,
 sbem_heat_transfer_rate, flow_per_person,
 dhw_consumption_per_hour_us, electrical_power_length,
 volume_flow_rate_litres_per_second_m2_facade,
 kwh_m2_to_kbtu_ft2, inverse_area_m2,
 inverse_per_energy_consumption_kwh,
 length_m_display_mm, mass_tonne_short_ton, wind_speed,
 volume_flow_ach, occupancy_people, degrees, percent,
 volume_flow_air_changes_per_hour_short, land_area,
 land_density, mass_kg_per_m,
 volume_flow_m3_per_hour_per_m2_of_envelope,
 pressure_inches_of_water_column,
 condensation_rate_kg_m2_second, joules_ton_hours,
 gigajoules_ton_hours, gigajoules_ton_kwh,
 occupany_people_per_100_m2, volume_flow_per_unit,
 volume_flow_equals_supply_air,
 volume_flow_greater_than_supply_air,
 volume_flow_percentage_above_supply_air,
 heating_cooling_power_kw_area,
 area_heating_cooling_power, suspended_solid, area_cm2,
 temperature_half_ip

6.1.7.2 Components

6.1.7.2.1 HVACAbstractBoiler

Interface for HVAC abstract boiler

6.1.7.2.1.1 Attributes Defined Here

<i>distribution_losses</i>	(float) Distribution losses
<i>electrical_circulation_power</i>	(float) Electrical circulation power
<i>heating_plant_type</i>	(int) Heating plant type
<i>is_dhw_boiler</i>	(bool) Whether it is a DHW boiler. True if so, False otherwise
<i>is_output_capacity_sized</i>	(int) Whether output capacity is sized. True if so, False otherwise.
<i>model_type</i>	(HVACBoilerType) Model type
<i>number_of_part_load_entries</i>	(int) Number of part load entries
<i>output_capacity</i>	(float) Output capacity
<i>oversizing_factor</i>	(float) Oversizing factor
<i>sequence_number</i>	(int) Sequence number
<i>uses_chp</i>	(bool) Whether CHP is used. True if so, False otherwise.
<i>uses_water_source_chp</i>	(bool) Whether water source CHP is used. True if so, False otherwise.
<i>is_parent_equipment</i>	(bool) Is parent equipment
<i>output_capacity_all_kw</i>	(float) Output capacity in kW. Design capacity for boilers, or the total design capacity for all HPs.
<i>reference_all</i>	(str) reference of equipment, for HPs will return number of identical HPs

6.1.7.2.2 HVACAbstractComponent

Interface for HVAC abstract component

6.1.7.2.2.1 Attributes Defined Here

<i>inlet_nodes</i>	(list) Inlet node numbers (int)
<i>is_duct</i>	(bool) Whether the component is a duct. True if so, False otherwise.
<i>outlet_nodes</i>	(list) Outlet node numbers (int)
<i>aps_component_type</i>	(HVACComponentTypes) APS component type

6.1.7.2.3 HVACAbstractChiller

Interface for HVAC abstract chiller

6.1.7.2.3.1 Attributes Defined Here

<i>chiller_type</i>	(HVACChillerType) Chiller type
<i>is_output_capacity_sized</i>	(bool) Whether the output capacity is sized. True if so, False otherwise.
<i>output_capacity</i>	(float) Output capacity
<i>is_parent_equipment</i>	(bool) Is Parent Equipment
<i>output_capacity_all_kw</i>	(float) Output capacity in kW. Design capacity for chillers, or the total design capacity for all HPs

6.1.7.2.4 HVACAbstractCHRHeatExchanger

Interface for HVAC heat exchanger

6.1.7.2.4.1 Attributes Defined Here

<i>capacity</i>	(float, W) Design capacity
<i>inlet_temperature_delta_t</i>	(float) Design inlet temperature delta T

6.1.7.2.5 HVACAbstractDXCoolingSystem

Interface for HVAC abstract DX cooling system

6.1.7.2.5.1 Attributes Defined Here

<i>coefficient</i>	(float) Coefficient
<i>coil_wet_bulb_temp</i>	(float) Coil wet bulb temperature
<i>condenser_type</i>	(HVACDXCoolingCondenserType) Condenser type
<i>fan_electric_input_ratio</i>	(float) Fan electric input ratio
<i>min_part_load_ratio</i>	(float) Minimum part load ratio
<i>outdoor_air_dry_bulb_temp</i>	(float) Outdoor air dry bulb temperature

6.1.7.2.6 HVACAbstractHeatPump

Interface for HVAC abstract heat pump

6.1.7.2.6.1 Attributes Defined Here

<i>heat_pump_type</i>	(HVACHeatPumpType) Heat pump type
<i>heat_source_id</i>	(string) Heat source ID
<i>min_source_temperature</i>	(float) Minimum source temperature
<i>number_of_cop_entries</i>	(int) Number of COP entries
<i>performance</i>	(dict) HVAC heat pump performance data. Entries are <i>source_temperature</i> , <i>cop</i> and <i>max_output</i>

6.1.7.2.7 HVACAbstractWaterAirHeatPump

Interface for HVAC abstract water air heat pump

6.1.7.2.7.1 Attributes Defined Here

<i>cop_cooling</i>	(float)
<i>cop_heating</i>	(float)
<i>entering_coil_wet_bulb_temperature_cooling</i>	(float)
<i>entering_coil_dry_bulb_temperature_heating</i>	(float)
<i>entering_water_temperature_cooling</i>	(float)
<i>entering_water_temperature_heating</i>	(float)

6.1.7.2.8 HVACAbstractWaterWaterHeatExchanger

Interface for abstract HVAC water-water heat exchanger

6.1.7.2.8.1 Methods Defined Here

get_approach([HVACWaterWaterHeatExchangerDataType](#)) -> float

Get the design approach

get_effectiveness([HVACWaterWaterHeatExchangerDataType](#)) -> float

Get the heat exchanger effectiveness

get_load_side_entering_temperature([HVACWaterWaterHeatExchangerDataType](#)) -> float

Get the design load side entering temperature

get_load_side_leaving_temperature([HVACWaterWaterHeatExchangerDataType](#)) -> float

Get the design load leaving temperature

get_source_flow_rate([HVACWaterWaterHeatExchangerDataType](#)) -> float

Get the design source flow rate

get_source_side_entering_temperature([HVACWaterWaterHeatExchangerDataType](#)) -> float

Get the design source side entering temperature

get_source_side_leaving_temperature ([HVACWaterWaterHeatExchangerDataType](#)) -> float

Get the design source side leaving temperature

get_supply_flow_rate([HVACWaterWaterHeatExchangerDataType](#)) -> float

Get the design supply flow rate

is_design_heat_rejection_autosized([HVACWaterWaterHeatExchangerDataType](#)) -> bool

Check whether the design heat rejection is autosized for the given [HVACWaterWaterHeatExchangerDataType](#). Returns True if so, False otherwise.

6.1.7.2.9 HVACAirFilter

Interface for HVAC air filter

6.1.7.2.10HVACAirToAirHeatEnthalpyExchanger

Interface for HVAC air to air heat enthalpy exchanger

6.1.7.2.10.1 Attributes Defined Here

<i>latent_heat_effectiveness</i>	(float) Latent heat effectiveness
<i>motor_or_pump_power</i>	(float) Motor or pump power
<i>sensible_heat_effectiveness</i>	(float) Sensible heat effectiveness

6.1.7.2.11 HVACAmbientHeatRejectionDevice

Interface for HVAC ambient heat rejection device

6.1.7.2.11.1 Methods Defined Here

get_num_heat_rejection_devices (cwl_id) -> int

Gets the number of heat rejection devices for the specified chilled water loop ID.

6.1.7.2.11.2 Attributes Defined Here

<i>design_approach</i>	(float) Design approach
<i>design_flow_rate</i>	(float) Design flow rate
<i>design_heat_rejection</i>	(float) Design heat rejection
<i>design_leaving_temperature</i>	(float) Design leaving temperature
<i>design_range</i>	(float) Design range
<i>design_supply_water_temperature</i>	(float) Supply water temperature
<i>design_wet_bulb_temperature</i>	(float) Design wet bulb temperature
<i>fan_control</i>	(HVACDXFanControl) Fan control
<i>fan_electric_input_ratio</i>	(float) Fan electric input ratio
<i>fan_power</i>	(float) Fan power
<i>hr_device_autosized</i>	(bool) Whether the heat rejection device is autosized. True if so, False otherwise.
<i>low_speed_fan_flow_fraction</i>	(float) Low speed fan flow fraction
<i>low_speed_fan_power_fraction</i>	(float) Low speed fan power fraction
<i>pump_delta_t</i>	(float) Pump delta T
<i>pump_efficiency</i>	(float) Pump efficiency
<i>pump_heat_gain</i>	(float) Pump heat gain
<i>specific_pump_power</i>	(float) Specific fan power

6.1.7.2.12 HVACBoiler

Interface for HVAC boiler

6.1.7.2.12.1 Attributes Defined Here

<i>part_load_performance</i>	(list) List of dictionaries of part load performance data. Dictionary keys are: efficiency, load, pump_usage
<i>primary_meter</i>	(VEEnergyMeter) Primary energy meter

6.1.7.2.13 HVACChilledCeiling

Interface for HVAC chilled ceiling

6.1.7.2.13.1 Attributes Defined Here

<i>distribution_pump_consumption</i>	(float) Distribution pump consumption
<i>material_index</i>	(int) Material index
<i>max_cooling_from_chiller</i>	(float) Maximum cooling from chiller
<i>orientation</i>	(int) Orientation
<i>output_at_ref_temp_difference</i>	(float) Output at reference temperature difference
<i>radiant_fraction</i>	(float) Radiant fraction
<i>reference_temp_difference</i>	(float) Reference temperature difference
<i>water_capacity</i>	(float) Water capacity
<i>weight</i>	(float) Weight

6.1.7.2.14 HVACChilledCeilingRoom

Interface for HVAC chilled ceiling room

6.1.7.2.14.1 Attributes Defined Here

<i>chilled_ceiling_id</i>	(string) Chilled ceiling ID
<i>chiller_id</i>	(string) Chiller ID
<i>cooling_source_id</i>	(string) Cooling source ID
<i>cooling_source_type</i>	(int) Cooling source type

6.1.7.2.15 HVACChilledWaterLoop

Interface for HVAC chilled water loop

6.1.7.2.15.1 Methods Defined Here

get_chiller_count_by_type(chiller type) -> int

Get the number of chillers of a provided HVACChillerType

6.1.7.2.15.2 Attributes Defined Here

<i>chillers</i>	(list) Chillers (HVACChiller)
<i>condenser_loop_design_pump_power_w</i>	(Optional[float]) condenser loop design pump power in W
<i>condenser_water_loop</i>	(HVACCondenserWaterLoop) Condenser water loop
<i>cooling_towers</i>	(list[HVACCoolingTower]) list of cooling towers
<i>design_cooling_capacity_kw</i>	(float) design cooling capacity in kW
<i>distribution_loss</i>	(float) Distribution loss
<i>dry_fluid_coolers</i>	(list[HVACDryFluidCooler]) list of dry fluid coolers
<i>independent_secondary_loop_enabled</i>	(bool) Whether an independent secondary loop is enabled
<i>is_condenser_water_loop_used</i>	(bool) Is Condenser water loop used
<i>is_thermal_storage_loop_used</i>	(bool) Is Thermal water loop used
<i>oversizing_factor</i>	(float) Oversizing factor
<i>pre_cooling_loop</i>	(HVACPreCoolingLoop) pre cooling loop
<i>primary_chilled_water_demand_side_load_fraction_high_threshold</i>	(float) Higher threshold for primary chilled water demand side load fraction
<i>primary_chilled_water_demand_side_load_fraction_low_threshold</i>	(float) Lower threshold for primary chilled water demand side load fraction
<i>primary_chilled_water_supply_temp_high_threshold</i>	(float) Higher threshold for primary chilled water supply temperature
<i>primary_chilled_water_supply_temp_low_threshold</i>	(float) Lower threshold for primary chilled water supply temperature
<i>primary_chilled_water_supply_temp_reset_type</i>	(int) Primary chilled water supply temperature reset type
<i>primary_chilled_water_supply_temperature</i>	(float) Primary chilled water supply temperature
<i>primary_chilled_water_temperature_difference</i>	(float) Primary chilled water temperature difference
<i>primary_circuit_using_dedicated_chiller_pumps</i>	(float) Primary circuit using dedicated chiller pumps
<i>primary_design_flow_rate</i>	(float) primary loop design flow

	rate in l/s
<code>primary_outdoor_dew_point_temp_high_threshold</code>	(float) Higher threshold for primary outdoor dew point temperature
<code>primary_outdoor_dew_point_temp_low_threshold</code>	(float) Lower threshold for primary outdoor dew point temperature
<code>primary_outdoor_dry_bulb_temp_high_threshold</code>	(float) Higher threshold for primary outdoor dry bulb temperature
<code>primary_outdoor_dry_bulb_temp_low_threshold</code>	(float) Lower threshold for primary outdoor dry bulb temperature
<code>primary_pump_power_w</code>	(float) primary loop pump power in w
<code>secondary_chilled_water_demand_side_load_fraction_high_threshold</code>	(float) Higher threshold for secondary chilled water demand side load fraction
<code>secondary_chilled_water_demand_side_load_fraction_low_threshold</code>	(float) Lower threshold for secondary chilled water demand side load fraction
<code>secondary_chilled_water_supply_temp_high_threshold</code>	(float) Secondary chilled water supply temp high threshold
<code>secondary_chilled_water_supply_temp_low_threshold</code>	(float) Lower threshold for secondary chilled water supply temperature
<code>secondary_chilled_water_supply_temp_reset_type</code>	(int) Secondary chilled water supply temperature reset type
<code>secondary_chilled_water_supply_temperature</code>	(float) Secondary chilled water supply temperature
<code>secondary_chilled_water_temperature_difference</code>	(float) Secondary chilled water temperature difference
<code>secondary_loop_design_flow_rate</code>	(Optional[float]) secondary loop design flow rate in l/s
<code>secondary_loop_pump_power_w</code>	(Optional[float]) secondary loop design pump power in watts
<code>secondary_outdoor_dew_point_temp_high_threshold</code>	(float) Higher threshold for secondary outdoor dew point temperature
<code>secondary_outdoor_dew_point_temp_low_threshold</code>	(float) Lower threshold for secondary outdoor dew point temperature
<code>secondary_outdoor_dry_bulb_temp_high_threshold</code>	(float) Higher threshold for secondary outdoor dry bulb temperature
<code>secondary_outdoor_dry_bulb_temp_low_threshold</code>	(float) Lower threshold for secondary outdoor dry bulb temperature
<code>thermal_storage_loop</code>	(list) Thermal storage loops (HVACThermalStorageLoop)

6.1.7.2.16 HVACChiller

Interface for HVAC chiller

6.1.7.2.17 HVACCombinedHeatAndPower

Interface for HVAC combined heat and power

6.1.7.2.18 HVACComponent

Interface for HVAC component

6.1.7.2.19 HVACCondenserWaterLoop

Interface for HVAC condenser water loop

6.1.7.2.19.1 Attributes Defined Here

<code>condenser_water_loop_used</code>	(bool) Whether a condenser water loop is used. True if so, False otherwise.
<code>cooling_tower</code>	(HVACCoolingTower) Cooling tower or None
<code>design_condenser_entering_water_temp</code>	(float) Condenser design entering water temperature
<code>design_condenser_loop_flow_rate</code>	(float) Design condenser loop flow rate
<code>design_condenser_water_loop_supply_temp</code>	(float) Condenser design water loop supply temperature
<code>design_outdoor_dry_bulb_temp</code>	(float) Design outdoor dry bulb temperature
<code>design_outdoor_wet_bulb_temp</code>	(float) Design outdoor wet bulb temperature
<code>design_temperature_diff</code>	(float) Design temperature difference
<code>dry_fluid_cooler</code>	(HVACDryFluidCooler) Dry fluid cooler or None
<code>entering_water_constant_value</code>	(float) Entering water constant value
<code>heat_rejection_device</code>	(HVACHeatRejectionDevice) heat rejection device enum
<code>motor_efficiency_factor</code>	(float) Motor efficiency factor
<code>specific_pump_power</code>	(float) Specific pump power

6.1.7.2.20 HVACCoolingCoil

Interface for HVAC cooling coil

6.1.7.2.20.1 Attributes Defined Here

<i>chilled_water_loop_circuit</i>	(CoilLocationCircuit) Chilled water loop circuit if the coil is a chilled water loop. Throws TypeError exception otherwise.
<i>contact_factor</i>	(float) Contact factor
<i>cool_source_ext_dry_bulb_temperature</i>	(float) Cooling source external dry bulb temperature
<i>cool_source_ext_dry_bulb_temperature_sized</i>	(bool) Whether the cooling source external dry bulb temperature is sized. True if so, False otherwise
<i>cool_source_ext_wet_bulb_temperature</i>	(float) Cooling source external wet bulb temperature
<i>cool_source_total_capacity</i>	(float) Cooling source total capacity
<i>cooling_source</i>	(Varies) Cooling source or None
<i>design_capacity</i>	(float) Design capacity
<i>design_cop</i>	(float) Design coefficient of performance (COP)
<i>design_entering_coil_temp</i>	(float) Design entering coil temperature
<i>design_outdoor_air_temp</i>	(float) Design outdoor air temperature
<i>design_parameters</i>	(dict) Design sizing parameters. Keys vary based on model and system type.
Possible keys are: oversizing_factor, contact_factor, sizing_mode, air_flow_rate, entering_air_db_temperature, entering_air_wb_temperature, leaving_air_db_temperature, leaving_air_wb_temperature, chilled_water_supply_temp, chilled_water_loop_delta_t, chilled_water_flow_rate, status, rated_capacity, design_capacity, design_cop, design_outdoor_air_db_temperature, design_outdoor_air_wb_temperature, design_entering_coil_wb_temperature, source_type, design_entering_water_temperature	
<i>model_type</i>	(CoolingCoilModelType) Cooling coil model type
<i>oversizing_factor</i>	(float) Oversizing factor
<i>system_type</i>	(HVACCoolingSourceSystemType) Cooling source system type

6.1.7.2.21 HVACCoolingTower

Interface for HVAC cooling tower

6.1.7.2.22 HVACDamper

Interface for HVAC damper

6.1.7.2.22.1 Attributes Defined Here

<i>mid_node</i>	(int) Mid node
<i>minimum_flow</i>	(float) Minimum flow
<i>modulating_profile_profile_id</i>	(string) The profile ID of the modulating profile

6.1.7.2.23 HVACDamperSplitter

Interface for HVAC damper splitter

6.1.7.2.24 HVACDedicatedWatersideEconomizer

Interface for HVAC dedicated waterside economizer

6.1.7.2.24.1 Attributes Defined Here

<i>back_up_chilled_water_loop_id</i>	(string) Back-up chilled water loop ID
<i>cooling_coil_design_water_delta_t</i>	(float) Cooling coil design water delta T
<i>cooling_tower_design_fan_power</i>	(float) Cooling tower design fan power
<i>design_cooling_tower_approach</i>	(float) Design cooling tower approach
<i>design_cooling_tower_load</i>	(float) Design cooling tower load
<i>design_cooling_tower_range</i>	(float) Design cooling tower range
<i>design_outside_wet_bulb_temperature</i>	(float) Design outside wet bulb temperature
<i>design_pump_power</i>	(float) Design pump power
<i>fan_control</i>	(int) Fan control
<i>heat_exchanger_effectiveness</i>	(float) Heat exchanger effectiveness
<i>low_speed_fan_flow_fraction</i>	(float) Low speed fan flow fraction
<i>low_speed_fan_power_fraction</i>	(float) Low speed fan power fraction
<i>operates_only_when_it_can_meet_the_coil_load_in_full</i>	(int) Whether it operates only when it can meet the coil load in full
<i>use_back_up_chilled_water_loop_cooling_tower_and_pump_params</i>	(bool) Whether back-up chilled water loop cooling tower and pump parameters are used. True if so, False otherwise.

6.1.7.2.25 HVACDHWBranch

Interface for DHW Branch

6.1.7.2.25.1 Attributes Defined Here

<i>delivery_efficiency</i>	(float) DHW delivery efficiency if simple mode, None otherwise
<i>dhw_design_supply_temperature_set_point_profile</i>	(string) DHW design supply temperature set point profile ID if simple mode and setpoint type is profile, None otherwise
<i>dhw_design_supply_temperature_set_point_type</i>	(HVACDHWTemperatureSetPointType) DHW design supply temperature set point type if simple mode, None otherwise
<i>dhw_design_supply_temperature_set_point_value</i>	(float) DHW design supply temperature set point value if simple mode and setpoint type is constant, None otherwise
<i>dhw_location_on_hwl</i>	(HVACHWLDemandLoop) DHW heat exchanger (HX) location on HWL
<i>hwl_dhw_heat_exchanger</i>	(HVACHWLDHWHeatExchanger) DHW HWL heat exchanger (HX)
<i>hx_approach</i>	(float) Heat exchanger approach if explicit mode, None otherwise
<i>hx_effectiveness</i>	(float) Heat exchanger effectiveness
<i>hx_load_hwl_hx_capacity</i>	(float, W) Heat exchanger load and HWL HX capacity if explicit mode, None otherwise
<i>hx_source_side_flow_rate</i>	(float) Heat exchanger source-side flow rate if explicit mode, None otherwise
<i>hx_supply_flow_rate</i>	(float) Heat exchanger DHW supply flow rate if explicit mode, None otherwise
<i>is_pre_heat_using_solar_water_heater</i>	(bool) Is DHW pre-heat using Solar Water Heater (SWH)? True or False if simple mode, None otherwise.
<i>is_pump_used</i>	(bool) Is a DHW pump used? True or False if explicit mode, None otherwise.
<i>is_recirculation_percent_of_flow</i>	(bool) Is this recirculation a percentage of the DHW design supply flow? True or False if recirculation is used, None otherwise.
<i>is_recirculation_scheduled</i>	(bool) Is the recirculation scheduled? True or False if recirculation is used, None otherwise.
<i>is_recirculation_used</i>	(bool) Is DHW recirculation used? True if so, False otherwise
<i>is_storage_tank_used</i>	(bool) Is a DHW storage tank used? True if so, False otherwise
<i>pump</i>	(HVACPump) Pump component if a DHW pump is used, None otherwise
<i>pump_heat_gain_fraction</i>	(float) Pump heat gain (fraction) if a DHW pump is used, None otherwise
<i>recirculation_flow_rate</i>	(float) Recirculation flow rate if recirculation is used, None otherwise
<i>recirculation_loop_length</i>	(float) Recirculation loop length if recirculation is used, None otherwise
<i>recirculation_losses</i>	(float) Recirculation losses if recirculation is used,

	None otherwise
<i>recirculation_percent_of_supply_flow_rate</i>	(float) % of DHW design supply flow rate if recirculation is used, None otherwise
<i>recirculation_profile_id</i>	(string) Recirculation profile ID if recirculation is used, None otherwise
<i>recirculation_pump_meter</i>	(VEEnergyMeter) Recirculation pump meter ID if recirculation is used, None otherwise
<i>recirculation_pump_power</i>	(float) Recirculation pump power if recirculation is used, None otherwise
<i>recirculation_schedule_type</i>	(HVACRecirculationScheduleType) Recirculation schedule type if recirculation is used, None otherwise
<i>solar_water_heater</i>	(HVACSolarWaterHeater) Solar water heater (SWH) if SWH is used for DHW pre-heat, None otherwise
<i>specific_pump_power_at Rated_speed</i>	(float) Specific pump power at rated speed if a DHW pump is used, None otherwise
<i>tank_heat_loss</i>	(float) Storage tank heat loss if storage tank is used, None otherwise
<i>tank_insulation_thickness</i>	(float) Tank insulation thickness if storage tank is used, None otherwise
<i>tank_insulation_type</i>	(HVACDHWTankInsulationType) Tank insulation type if storage tank is used, None otherwise
<i>tank_storage_volume</i>	(float) Tank storage volume if storage tank is used, None otherwise
<i>tank_type</i>	(HVACDHWTankDetails) Storage tank type if storage tank is used, None otherwise

6.1.7.2.26 HVACDHWCHRHeatExchanger

Interface for DHW CHR Heat Exchanger

6.1.7.2.27 HVACDHWSystem

Interface for HVAC DHW system

6.1.7.2.27.1 Attributes Defined Here

<i>branches</i>	(list) DHW branches
<i>cold_water_inlet_temperature_set_point_profile</i>	(string) Cold water inlet temperature set point profile ID if setpoint type is profile, None otherwise
<i>cold_water_inlet_temperature_set_point_type</i>	(HVACDHWTemperatureSetPointType) Cold water inlet temperature set point type
<i>cold_water_inlet_temperature_value</i>	(float) DHW cold water inlet temperature value if setpoint type is constant, None otherwise
<i>condenser_heat_recovery</i>	(HVACDHWCHRHeatExchanger) Condenser heat recovery if CWL is used for DHW pre-heat, None otherwise
<i>condenser_heat_recovery_pre_heat_used</i>	(bool) Whether a condenser heat recovery heat exchanger is used for DHW pre-heat. True or False if explicit mode, None otherwise.
<i>condenser_water_loop</i>	(HVACCondenserWaterLoop) Condenser water loop (CWL) if CWL is used for DHW pre-heat, None otherwise
<i>design_cold_water_inlet_temperature</i>	(float) DHW design cold water inlet temperature if explicit mode, None otherwise
<i>dhw_design_supply_temperature</i>	(float) DHW design supply temperature if explicit mode, None otherwise
<i>dhw_design_supply_temperature_set_point_profile</i>	(string) DHW design supply temperature set point profile ID if explicit mode and setpoint type is profile, None otherwise
<i>dhw_design_supply_temperature_set_point_type</i>	(HVACDHWTemperatureSetPointType) DHW design supply temperature set point type if explicit mode, None otherwise
<i>dhw_design_supply_temperature_set_point_value</i>	(float) DHW design supply temperature set point value if explicit mode and setpoint type is constant, None otherwise
<i>dhw_model</i>	(HVACDHWModel) DHW Model
<i>heat_source</i>	(HVACHeatSource) Heat source
<i>is_to_be_autosized</i>	(bool) Whether the DHW flow rate is to be autosized. True or False if explicit mode, None otherwise.
<i>solar_water_heater</i>	(HVACSolarWaterHeater) Solar water heater (SWH) if SWH is used for DHW pre-heat, None otherwise
<i>solar_water_heater_pre_heat_used</i>	(bool) Whether a solar water heater is used for DHW pre-heat. True or False if explicit mode, None otherwise.

6.1.7.2.27.2 Methods Defined Here

get_branch_by_id(id) -> [HVACDHWBranch](#)

Get a DHW branch by ID

6.1.7.2.28 HVACDirectActingHeater

Interface for HVAC direct acting heater

6.1.7.2.28.1 Attributes Defined Here

<i>efficiency</i>	(float) efficiency
<i>radiant_fraction</i>	(float) Radiant fraction
<i>sequence_number</i>	(int) Sequence number
<i>uses_chp</i>	(bool) Whether CHP is used. True if so, False otherwise.

6.1.7.2.29 HVACDirectActingHeaterRoom

Interface for direct acting heater room

6.1.7.2.29.1 Attributes Defined Here

<i>direct_acting_heater_id</i>	(string) Direct acting heater ID
<i>heat_output_for_max_control_signal</i>	(float) Heat output for maximum control signal
<i>heat_output_for_min_control_signal</i>	(float) Heat output for minimum control signal
<i>heat_output_max_change_per_time_step</i>	(float) Heat output maximum change per time step
<i>heat_output_midband</i>	(float) Heat output midband
<i>heat_output_midband_mode</i>	(int) Heat output midband mode
<i>heat_output_midband_profile_id</i>	(string) Heat output midband profile ID
<i>heat_output_proportional_bandwidth</i>	(float) Heat output proportional bandwidth
<i>heat_output_proportional_control</i>	(int) Whether there is heat output proportional control 1 if so, 0 otherwise.
<i>heat_output_proportional_controller_id</i>	(string) Heat output proportional controller ID
<i>heat_output_proportional_sensed_variable</i>	(int) Heat output proportional sensed variable
<i>heat_output_proportional_sensor_body_index</i>	(int) Heat output proportional sensor body index
<i>heat_output_proportional_sensor_location</i>	(int) Heat output proportional sensor location
<i>heat_output_proportional_sensor_room_id</i>	(string) Heat output proportional sensor room ID
<i>prop_heat_output_orientation</i>	(float) Proportional heat output orientation
<i>prop_heat_output_radiant_fraction</i>	(float) Proportional heat output radiant fraction
<i>prop_heat_output_slope</i>	(float) Proportional heat output slope

6.1.7.2.30 HVACDryFluidCooler

Interface for HVAC dry fluid cooler

6.1.7.2.30.1 Attributes Defined Here

<i>design_dry_bulb_temperature</i>	(float) Design dry bulb temperature
<i>design_wet_bulb_delta_t</i>	(float) Design wet bulb delta T
<i>fluid_cooler_mode</i>	(HVACDryFluidCoolerMode) Fluid cooler mode
<i>spray_pump_electric_input_ratio</i>	(float) Spray pump electric input ratio
<i>spray_pump_power</i>	(float) Spray pump power
<i>switch_temperature</i>	(float) Switch temperature

6.1.7.2.31 HVACDuct

Interface for HVAC Duct

6.1.7.2.32 HVACDXCoolingInstance

Interface for HVAC DX cooling instance

6.1.7.2.33 HVACDXCoolingSystem

Interface for HVAC DX cooling system

6.1.7.2.34 HVACEACChiller

Interface for HVAC EAC chiller

6.1.7.2.34.1 Attributes Defined Here

<i>condenser_fan_power</i>	(float) Condenser fan power
<i>condenser_fan_eir</i>	(float) Condenser fan EIR
<i>is_outdoor_dry_bulb_temperature_autosized</i>	(bool) Whether outdoor dry bulb temperature is autosized. True if so, False otherwise

6.1.7.2.35 HVACEnhancedBoiler

Interface for HVAC enhanced boiler

6.1.7.2.35.1 Attributes Defined Here

<i>hot_water_pump_factor</i>	(float) Hot water pump factor
<i>hot_water_pump_power</i>	(float) Hot water pump power
<i>is_two_identical_boiler</i>	(bool) Whether it forms a pair of identical boilers. True if so, False otherwise.
<i>max_parasitic_power</i>	(float) Maximum parasitic power
<i>parasitic_power</i>	(float) Parasitic power
<i>parasitic_ratio</i>	(float) Parasitic ratio
<i>percentage_of_combined_capacity</i>	(float) Percentage of combined capacity
<i>rated_condition</i>	(dict) Dictionary of rated condition data. Keys are: efficiency
<i>primary_meter</i>	(VEEnergyMeter) Primary energy meter

6.1.7.2.36 HVACCPHPBoiler

Interface for HVAC enhanced boiler

6.1.7.2.36.1 Attributes Defined Here

No Attributes beyond those defined in HVACAbstractBoiler.

6.1.7.2.37 HVACElectricChiller

Interface for HVAC electric chiller

6.1.7.2.37.1 Attributes Defined Here

<i>compressor_heat_gain_to_condenser_loop</i>	(float) Compressor heat gain to condenser loop fraction
<i>cop</i>	(float) Design coefficient of performance (COP)
<i>curve_type_names</i>	(list) Curve type names (string)
<i>iplv</i>	(float) Integrated part load value
<i>min_part_load_ratio</i>	(float) Minimum part load ratio

6.1.7.2.38 HVACEWCChiller

Interface for HVAC EWC chiller

6.1.7.2.39 HVACFan

Interface for HVAC fan

6.1.7.2.39.1 Attributes Defined Here

<i>design_fan_power</i>	(float) Design fan power
<i>design_flow_rate</i>	(float) Design flow rate
<i>design_total_pressure</i>	(float) Design total pressure
<i>fan_efficiency_at_design_flow_rate</i>	(float) Fan efficiency at the design flow rate
<i>is_autosizable</i>	(bool) Whether the fan is autosizable. True if so, False otherwise.
<i>is_variable_air_volume</i>	(bool) Whether the fan has variable air volume. True if so, False otherwise.
<i>motor_airstream_heat_pickup_factor</i>	(float) Motor airstream pickup factor
<i>motor_efficiency_at_design_flow_rate</i>	(float) Motor efficiency at the design flow rate
<i>oversizing_factor</i>	(float) Oversizing factor

6.1.7.2.40 HVACGenericCoolingSource

Interface for HVAC generic cooling source

6.1.7.2.40.1 Attributes Defined Here

<i>distribution_losses</i>	(float) Distribution losses
<i>is_output_capacity_sized</i>	(bool) Whether output capacity is sized. True if so, False otherwise
<i>oversizing_factor</i>	(float) Oversizing factor
<i>plc_chiller</i>	(HVACPLCChiller) PLC chiller

6.1.7.2.41 HVACGenericHeatSource

Interface for HVAC generic heat source

6.1.7.2.41.1 Attributes Defined Here

<i>autosize_air_source_heat_pump</i>	(bool) Whether the air source heat pump is autosized. True if so, false otherwise.
<i>percentage_of_heat_source_capacity</i>	(float) Percentage of heat source capacity
<i>use_air_source_heat_pump</i>	(bool) Whether an air source heat pump is used. True if so, False otherwise.

6.1.7.2.42 HVACHeatingCoil

Interface for HVAC heating coil

6.1.7.2.42.1 Attributes Defined Here

<i>design_parameters</i>	(dict) Design sizing parameters. Keys vary based on model and system type. Possible keys are: oversizing_factor, design_capacity, air_flow_rate, entering_air_db_temperature, leaving_air_db_temperature, hot_water_supply_temp, hot_water_loop_delta_t, hot_water_flow_rate, rated_capacity, design_cop, design_entering_coil_db_temperature, design_outdoor_air_wb_temperature, meter, design_entering_water_temperature
<i>heat_source</i>	(Varies) Heat source or 'None' if not applicable
<i>hot_water_loop_circuit</i>	(HVACHWLDemandLoop) Hot water loop circuit if system type is 'heating_system_hot_water_loop', 'None' otherwise
<i>htl</i>	(HVACHeatTransferLoop) Heat transfer loop or 'None' if not applicable
<i>model_type</i>	(HeatingCoilModelType) Model type
<i>system_type</i>	(HeatingSourceSystemType) System type

6.1.7.2.43 HVACHeatPump

Interface for HVAC heat pump

6.1.7.2.44 HVACHeatPumpInstance

Interface HVAC heat pump instance

6.1.7.2.45 HVACHeatSource

Interface for HVAC heat source

6.1.7.2.45.1 Attributes Defined Here

<i>design_heating_capacity</i>	(float) Capacity
<i>equipment</i>	(list) Equipment (HVACAbstractBoiler)

6.1.7.2.46 HVACHeatTransferLoop

Interface for HVAC heat transfer loop

6.1.7.2.46.1 Attributes Defined Here

<i>flow_rate</i>	(float) Flow rate
<i>oversizing_factor</i>	(float) Oversizing factor
<i>heating_capacity</i>	(float) Heating capacity
<i>cooling_capacity</i>	(float) Cooling capacity
<i>primary_pump_power</i>	(float) Primary pump power
<i>secondary_pump_power</i>	(float) Secondary pump power or None
<i>heat_acquisition_chr_capacity</i>	(float) Heat acquisition condenser heat recovery capacity or None
<i>heat_acquisition_chr_wwhp_capacity</i>	(float) Heat acquisition condenser heat recovery water-water heat pump capacity or None
<i>heat_acquisition_awhp_capacity</i>	(float) Heat acquisition condenser heat recovery air-water heat pump capacity or None
<i>water_source_heat_exchanger_heating_capacity</i>	(float) Heating capacity for water-source-heat-exchanger or None
<i>water_source_heat_exchanger_cooling_capacity</i>	(float) Cooling capacity for water-source-heat-exchanger or None
<i>heat_rejection_equipment_capacity_cooling_tower</i>	(float) Heat rejection cooling tower capacity or None
<i>heat_rejection_equipment_capacity_fluid_cooler</i>	(float) Heat rejection fluid cooler capacity or None
<i>boilers</i>	(list) List of boilers

6.1.7.2.47 HVACHotWaterLoop

Interface for HVAC hot water loop

6.1.7.2.47.1 Attributes Defined Here

aux_boiler_data	(dict) Aux Boilers Data. Will return empty dict if aux boiler is not used.
boilers	(Optional[float]) Pre-heating condenser heat recovery air-to-water heat pump capacity
demand_loop	(HVACHWLDemandLoop) Demand Loop
is_pre_heat_using_solar_water_heater	(bool) True if HWL uses SWH, otherwise False
loop_configuration	(HVACLoopConfiguration) Loop Configuration Enum
oversizing_factor	(float) Oversizing factor
pre_heating_awhp_capacity	(Optional[float]) Pre-heating condenser heat recovery air-to-water heat pump capacity
pre_heating_chr_capacity	(Optional[float]) Pre-heating condenser heat recovery capacity
pre_heating_hr_wwhp_capacity	(Optional[float]) Pre-heating heat recovery capacity for water-to-water heat pump
secondary_loops	(list[HVACHWLDemandLoop]) List of secondary loops
solar_water_heater	(HVACSolarWaterHeater) Solar water heater
supply_loop	(HVACHWLSupplyLoop) Supply Loop

6.1.7.2.48 HVACWaterLoop

Interface for HVAC Water Loop Component.

6.1.7.2.48.1 Attributes Defined Here

<i>flow_rate</i>	(float) loop flow rate in l/s
<i>is_capacity_sized</i>	(bool) Is capacity sized

6.1.7.2.49HVACHWLSupplyLoop

Interface for Hot Water Loop Supply Loop component.

6.1.7.2.49.1 Attributes Defined Here

Includes interface `_HVACWaterLoop`

<i>capacity_w</i>	(float) Capacity in watts
<i>is_distribution_branch_used_for_coupling_loads</i>	(bool) is distribution branch used for coupling loads
<i>pump_power</i>	(float) Pump power in watts

6.1.7.2.50 HVACHWLDemandLoop

Interface for HVAC HWL demand loop

6.1.7.2.50.1 Attributes Defined Here

Includes interface `_HVACWaterLoop`

<i>capacity_w</i>	(float) Capacity in watts
<i>demand_loop_type</i>	(enum HVACDemandLoopType) demand loop type
<i>pump_power</i>	(float) Pump power in watts

6.1.7.2.51 HVACWLDHWHeatExchanger

Interface for HVAC DHW HWL heat exchanger

6.1.7.2.51.1 Attributes Defined Here

<code>inlet_temperature_delta_t</code>	(float) Inlet temperature delta T
--	-----------------------------------

6.1.7.2.52 HVACInlet

Interface for HVAC inlet

6.1.7.2.53 HVACJunction

Interface for HVAC Junction

6.1.7.2.54 HVACOutlet

Interface for HVAC outlet

6.1.7.2.55 HVACPCMBattery

Interface for HVAC PCM battery

6.1.7.2.55.1 Attributes Defined Here

<i>attached_room_id</i>	(string) Room ID
<i>model_id</i>	(string) Model ID
<i>model_not_found</i>	(bool) Whether the model is not found. True if the model is not found, False otherwise.
<i>number_of_units</i>	(int) Number of units

6.1.7.2.56 HVACPlenum

Interface for HVAC plenum

6.1.7.2.56.1 Attributes Defined Here

<i>attached_room_id</i>	(string) Attached room ID
-------------------------	---------------------------

6.1.7.2.57 HVACPLCChiller

Interface for HVAC PLC chiller

6.1.7.2.57.1 Attributes Defined Here

<i>chiller_pumps_power</i>	(float) Power of the chiller pumps
<i>chr_recipient_id</i>	(string) CHR recipient ID
<i>condenser_water_pump_power</i>	(float) Condenser water pump power
<i>cooling_tower_fan_power</i>	(float) Cooling tower fan power
<i>deprecated_chr_percentage</i>	(float) Deprecated CHR percentage
<i>heat_source_id</i>	(string) Heat source ID
<i>is_absorption_chiller</i>	(bool) Whether it is an absorption chiller. True if so, False otherwise.
<i>is_chr_recipient_set</i>	(bool) Whether the CHR recipient is set. True if so, False otherwise.
<i>is_used</i>	(bool) Is pre cooling loop used.
<i>number_of_part_load_entries</i>	(int) Number of part load entries
<i>number_of_temperature_dependent_cops</i>	(int) Number of temperature dependent COPs
<i>outside_temp_for_cop_data</i>	(int) Outside temperature for COP data

6.1.7.2.58 HVACPreCoolingLoop

Interface for HVAC pre-cooling loop

6.1.7.2.58.1 Attributes Defined Here

<i>autosizable</i>	(bool) Whether it is autosizable. True if so, False otherwise.
<i>autosize_value</i>	(float) Autosize value
<i>autosized</i>	(bool) Whether it is autosized. True if so, False otherwise.
<i>capacity</i>	(float) Capacity
<i>cooling_tower_autosizable</i>	(bool) Whether the cooling tower is autosizable. True if so, False otherwise.
<i>deprecated_heat_recovery</i>	(float) Deprecated heat recovery
<i>dry_fluid_cooler_autosizable</i>	(bool) Whether the dry fluid cooler is autosizable. True if so, False otherwise.
<i>heat_recovery_recipient_id</i>	(string) Heat recovery recipient ID
<i>heat_recovery_recipient_set</i>	(bool) Whether a heat recovery recipient is set. True if so, False otherwise.
<i>heat_recovery_used</i>	(bool) Whether heat recovery is used. True if so, False otherwise.
<i>water_source_loop_autosizable</i>	(bool) Whether the water source loop is autosizable. True if so, False otherwise.
<i>water_source_loop_autosize_value</i>	(float) Water source loop autosize value.
<i>water_source_loop_used</i>	(bool) Whether a water source loop is used. True if so, False otherwise.

6.1.7.2.59 HVACPump

Interface for HVAC pump

6.1.7.2.59.1 Attributes Defined Here

<i>efficiency_factor</i>	(float) Efficiency factor
<i>meter</i>	(VEEnergyMeter) Pump meter
<i>power_curve</i>	(HVACSystemCurve) System (power) curve
<i>pump_power</i>	(float) Pump power

6.1.7.2.60 HVACRadiator

Interface for HVAC radiator

6.1.7.2.60.1 Attributes Defined Here

<i>distribution_pump_consumption</i>	(float) Distribution pump consumption
<i>material_index</i>	(int) Material index
<i>max_input_from_boiler</i>	(float) Maximum input from boiler
<i>orientation</i>	(int) Orientation
<i>output_at_ref_temp_difference</i>	(float) Output at reference temperature difference
<i>radiant_fraction</i>	(float) Radiant fraction
<i>reference_temp_difference</i>	(float) Reference temperature difference
<i>water_capacity</i>	(float) Water capacity
<i>weight</i>	(float) Weight

6.1.7.2.61 HVACRadiatorRoom

Interface for HVAC radiator room

6.1.7.2.61.1 Attributes Defined Here

<i>get_heat_source_type</i>	(int) Heat source type
<i>heat_source_id</i>	(string) Heat source ID
<i>radiator_id</i>	(string) Radiator ID

6.1.7.2.62 HVACRoom

Interface for HVAC room

6.1.7.2.62.1 Attributes Defined Here

<i>chilled_ceiling_room_units</i>	(list) Chilled ceiling room units (HVACChilledCeilingRoom)
<i>direct_acting_heater_room_units</i>	(list) Direct acting heater room units (HVACDirectActingHeaterRoom)
<i>is_disconnected_room</i>	(bool) Whether the room is disconnected. True if so, False otherwise.
<i>is_principal</i>	(bool) Whether it is a principal room. True if so, False otherwise.
<i>radiator_units</i>	(list) Radiator units (HVACRadiatorRoom)

6.1.7.2.63 HVACRoomUnit

Interface for HVAC room unit

6.1.7.2.63.1 Attributes Defined Here

<i>deadband_value</i>	(float) Deadband value
<i>high_sensor_input</i>	(int) High sensor input
<i>is_setpoint_sensor_enabled</i>	(int) Whether the set point sensor is enabled. 1 if so, 0 otherwise
<i>number_of_and_connections</i>	(int) Number of AND connections
<i>number_of_or_connections</i>	(int) Number of OR connections
<i>on_off_controller_id</i>	(string) On/off controller ID
<i>on_off_sensor_body_index</i>	(int) On/off sensor body index
<i>on_off_sensor_room_id</i>	(string) On/off sensor room ID
<i>orientation</i>	(float) Orientation
<i>radiant_fraction</i>	(float) Radiant fraction
<i>sensed_variable</i>	(int) Sensed variable
<i>sensor_location</i>	(int) Sensor location
<i>set_point_mode</i>	(int) Set point mode
<i>set_point_profile_id</i>	(string) Set point profile ID
<i>set_point_value</i>	(float) Set-point value
<i>slope</i>	(float) Slope
<i>time_switch_profile_id</i>	(string) Time switch profile ID

6.1.7.2.64 HVACSolarAirCollectorBIST

Interface for solar air collector BIST

6.1.7.2.64.1 Attributes Defined Here

<i>attached_room_id</i>	(string) Attached room ID
-------------------------	---------------------------

6.1.7.2.65 HVACSolarWaterHeater

Interface for HVAC solar water heater

6.1.7.2.65.1 Attributes Defined Here

<i>flat_solar_panel_data</i>	(dict) Flat PV data (float)
<i>is_waterside_component</i>	(bool) Whether it is a waterside component. True if so, False otherwise.
<i>panel_type</i>	(HVACSHWPanelType) Panel type
<i>parabolic_solar_panel_data</i>	(dict) Parabolic PV data (float)

6.1.7.2.66 HVACSourceDependentRoom

Interface for HVAC source dependent room component

6.1.7.2.66.1 Attributes Defined Here

<i>autosize_mode</i>	(int) Whether autosize mode is enabled. 1 if so, 0 otherwise.
<i>design_heating_cooling_load</i>	(float) Design heating and cooling load
<i>design_room_air_temp</i>	(float) Design room air temperature
<i>design_room_radiant_temp</i>	(float) Design room radiant temperature
<i>design_water_delta_t</i>	(float) Design water delta T
<i>error_code</i>	(int) Error code
<i>flow_for_max_control_signal</i>	(float) Flow for maximum control signal
<i>flow_for_min_control_signal</i>	(float) Flow for minimum control signal
<i>flow_max_change_per_time_step</i>	(float) Flow max change per time step
<i>flow_midband</i>	(float) Flow midband
<i>flow_midband_mode</i>	(int) Flow midband mode
<i>flow_midband_profile_id</i>	(string) Flow midband profile ID
<i>flow_proportional_bandwidth</i>	(float) Flow proportional bandwidth
<i>flow_proportional_control</i>	(int) Whether it is flow proportional control. 1 if so, 0 otherwise.
<i>flow_proportional_controller_id</i>	(string) Flow proportional controller ID
<i>flow_proportional_sensed_variable</i>	(int) Flow proportional sensed variable
<i>flow_proportional_sensor_body_index</i>	(int) Flow proportional sensor body index
<i>flow_proportional_sensor_location</i>	(int) Flow proportional sensor location
<i>flow_proportional_sensor_room_id</i>	(string) Flow proportional sensor room ID
<i>model_id</i>	(string) Model ID
<i>number_of_units</i>	(float) Number of units
<i>oversizing_factor</i>	(float) Oversizing factor
<i>prop_flow_orientation</i>	(float) Proportional flow orientation
<i>prop_flow_radiant_fraction</i>	(float) Proportional flow radiant fraction
<i>prop_flow_slope</i>	(float) Proportional flow slope
<i>prop_temp_orientation</i>	(float) Proportional temperature orientation
<i>prop_temp_radiant_fraction</i>	(float) Proportional temperature radiant fraction
<i>prop_temp_slope</i>	(float) Proportional temperature slope
<i>temp_for_max_control_signal</i>	(float) Temperature for maximum control signal
<i>temp_for_min_control_signal</i>	(float) Temperature for minimum control signal
<i>temp_max_change_per_time_step</i>	(float) Temperature maximum change per time step
<i>temp_midband</i>	(float) Temperature midband
<i>temp_midband_mode</i>	(int) Temperature midband mode
<i>temp_midband_profile_id</i>	(string) Temperature midband profile ID
<i>temp_proportional_bandwidth</i>	(float) Temperature proportional bandwidth
<i>temp_proportional_control</i>	(int) Whether it is temperature proportional control. 1 is so, 0 otherwise.
<i>temp_proportional_controller_id</i>	(string) Temperature proportional controller ID
<i>temp_proportional_sensed_variable</i>	(int) Temperature proportional sensed variable
<i>temp_proportional_sensor_body_index</i>	(int) Temperature proportional sensor body index
<i>temp_proportional_sensor_location</i>	(int) Temperature proportional sensor location
<i>temp_proportional_sensor_room_id</i>	(string) Temperature proportional sensor ID
<i>thermal_source_id</i>	(string) Thermal source ID

water_loop_supply_temperature_used (int) Whether water loop supply temperature is used. 1 is so, 0 otherwise.

6.1.7.2.67 HVACSprayChamber

Interface for HVAC spray chamber

6.1.7.2.67.1 Attributes Defined Here

<i>circulation_pump_power</i>	(float) Circulation pump power
<i>spray_efficiency</i>	(float) Spray efficiency

6.1.7.2.68 HVACSteamHumidifier

Interface for HVAC steam humidifier

6.1.7.2.68.1 Attributes Defined Here

<i>boiler_supply_flag</i>	(int) Boiler supply flag
<i>heat_source_id</i>	(string) Heat source ID
<i>maximum_output</i>	(float) Maximum output
<i>maximum_relative_humidity</i>	(float) Maximum relative humidity
<i>total_efficiency</i>	(float) Total efficiency

6.1.7.2.69 HVACThermalDuctProperties

HVAC thermal duct properties

6.1.7.2.69.1 Attributes Defined Here

<i>body_index</i>	(int) Duct location (HVACLocation)
<i>duct_location</i>	(int) Body index
<i>duct_surface_area</i>	(float) Duct surface area
<i>duct_u_value</i>	(float) Duct U-value
<i>space_id</i>	(string) Space ID

6.1.7.2.70 HVACThermalStorageLoop

Interface for HVAC thermal storage loop

6.1.7.2.70.1 Attributed Defined Here

<i>chillers</i>	(list) Chillers (HVACChiller)
<i>condenser_water_loop</i>	(HVACCondenserWaterLoop) Condenser water loop
<i>pump</i>	(HVACPump) Pump
<i>tank</i>	(HVACThermalStorageTank) Tank

6.1.7.2.71 HVACThermalStorageTank

Interface for HVAC thermal storage tank

6.1.7.2.72 HVACUnitaryCoolingSystem

Interface for HVAC Unitary cooling system

6.1.7.2.72.1 Attributes Defined Here

<i>design_air_flow_rate</i>	(float) Design air flow rate
<i>design_airflow_autosized</i>	(int) Design airflow autosized
<i>extrapolate_performance_data</i>	(int) Extrapolate performance data
<i>heat_rejection_fan_power</i>	(float) Heat rejection fan power
<i>low_load_copr_degradation_factor</i>	(float) Low-load COPR degradation factor
<i>scale_performance_parameters_with_design_air_flow_rate</i>	(int) Scale performance parameters with design air flow rate
<i>supply_fan_power</i>	(float) Supply fan power

6.1.7.2.73 HVACVRFSystem

Interface for HVAC VRF System

6.1.7.2.73.1 Attributes Defined Here

<i>design_condition_outdoor_unit_data</i>	(dict) Dictionary of design condition outdoor unit data. Possible dictionary entries are: cooling_capacity, cooling_cop, cooling_combination_ratio, cooling_average_indoor_wb_temperature, cooling_outdoor_db_temperature, heating_capacity, heating_cop, heating_combination_ratio, heating_average_indoor_db_temperature, heating_average_indoor_wb_temperature, heating_outdoor_wb_temperature, heating_outdoor_db_temperature, cooling_entering_source_loop_temperature, heating_entering_source_loop_temperature
<i>design_parameters</i>	(dict) Dictionary of design parameters. Possible dictionary entries are: performance_curve_set_description, performance_curve_set_name, is_autosize_outdoor_unit, oversizing_factor
<i>reference_condition_indoor_unit_data</i>	(dict) Dictionary of reference indoor unit data. Possible dictionary entries are: cooling_entering_coil_wb_temperature, cooling_outdoor_air_db_temperature, use_same_cooling_temperature_as_outdoor, heating_entering_coil_db_temperature, heating_outdoor_air_wb_temperature, use_same_heating_temperature_as_outdoor, cooling_entering_water_temperature, heating_entering_water_temperature
<i>reference_condition_outdoor_unit_data</i>	(dict) Dictionary of reference condition outdoor unit data. Possible dictionary entries are: cooling_capacity, cooling_cop, cooling_combination_ratio, cooling_average_indoor_wb_temperature, cooling_outdoor_db_temperature, use_same_cooling_temperature_as_indoor, heating_capacity, heating_cop, heating_combination_ratio, heating_average_indoor_db_temperature, heating_average_indoor_wb_temperature, heating_outdoor_wb_temperature, heating_outdoor_db_temperature, use_same_heating_temperature_as_indoor, cooling_entering_source_loop_temperature, heating_entering_source_loop_temperature
<i>system_data</i>	(dict) Dictionary of VRF System data. Possible dictionary entries are:

unit_control_profile, source_type, min_part_load_ratio,
 unit_control, source_water_loop_id,
 master_thermostat_location_space_id, heat_recovery_mode,
 is_separate_piping_values, equivalent_piping_length_cooling,
 equivalent_piping_length_heating,
 piping_level_difference_cooling,
 piping_level_difference_heating, use_crankcase_heater,
 crankcase_heater_power,
 max_outdoor_temp_for_crankcase_heater_operation,
 use_electric_resistance_backup_heat,
 backup_cooling_source_cop,
 backup_heating_source_efficiency,
 outdoor_temp_operation_range_cooling_minimum,
 outdoor_temp_operation_range_cooling_maximum,
 outdoor_temp_operation_range_heating_minimum,
 outdoor_temp_operation_range_heating_maximum, fan_eir,
 defrost_strategy, resistive_defrost_heater_capacity,
 defrost_control, defrost_time_fraction,
 max_outdoor_temp_for_defrost_operation,
 cooling_heating_meter, crankcase_heater_meter,
 electric_resistance_backup_heat_meter,
 backup_cooling_source_meter,
 backup_heating_source_meter, fan_meter, defrost_meter

6.1.7.2.74 HVACWaterAirHeatPump

Interface for HVAC water air heat pump

6.1.7.2.75 HVACWaterAirHeatPumpInstance

Interface for HVAC water air heat pump instance

6.1.7.2.76 HVACWaterSourceLoop

Interface for HVAC water source loop

6.1.7.2.76.1 Attributes Defined Here

<i>pump_delta_t</i>	(float) Pump delta T
<i>pump_efficiency</i>	(float) Pump efficiency
<i>pump_heat_gain</i>	(float) Pump heat gain
<i>specific_pump_power</i>	(float) Specific pump power

6.1.7.2.77 HVACWaterWaterHeatExchanger

Interface for HVAC water-water heat exchanger

6.1.7.2.78 HVACWaterWaterHeatPump

Interface for HVAC water-water heat pump

6.1.7.2.79 HVACWaterLoopHeatRecovery

Interface for HVAC water loop heat recovery

6.1.7.2.79.1 Attributes Defined Here

<i>model</i>	(HVACHeatRecoveryModel) Model
--------------	---

6.1.7.2.80 HVACZone

Interface for HVAC Zone

6.1.7.2.80.1 Attributes Define Here

<i>rooms</i>	(list[HVACRoom]) List of rooms
<i>room_ids</i>	(list) List of room IDs
<i>zone_id</i>	(string) Zone ID

6.1.7.3 Controllers

6.1.7.3.1 HVACAbstractController

Interface for HVAC abstract controller

6.1.7.3.1.1 Attributes Defined Here

<i>control_line_orientation</i>	(int) Control line orientation
<i>control_node</i>	(int) Control node
<i>independent_mode</i>	(int) Independent mode
<i>max_control_signal_value</i>	(float) Maximum control signal value
<i>number_of_and_connections</i>	(int) Number of AND connections
<i>reference_line_orientation</i>	(int) Reference line orientation
<i>reference_node</i>	(int) Reference node
<i>sensor_line_orientation</i>	(int) Sensor line orientation
<i>sensor_node</i>	(int) Sensor node
<i>time_switch_profile_id</i>	(string) Time switch profile ID

6.1.7.3.2 HVACController

Interface for HVAC controller

6.1.7.3.3 HVACTimeSwitchController

Interface for HVAC time switch controller

6.1.7.3.3.1 Attributes Defined Here

<i>max_signal_variation_mode</i>	(int) Maximum signal variation mode
<i>max_signal_variation_profile_id</i>	(string) Maximum signal variation profile ID

6.1.7.3.4 HVACAbstractControllerWithSensor

Interface for HVAC abstract controller with sensor

6.1.7.3.4.1 Attributes Defined Here

<i>deadband</i>	(float) Deadband
<i>high_sensor_input</i>	(int) High sensor input
<i>is_active_set_point</i>	(int) Whether there is an active set point. 1 if so, 0 otherwise.
<i>is_proportional_control</i>	(int) Whether there is a proportional controller. 1 if so, 0 otherwise.
<i>max_change_per_time_step</i>	(float) Maximum change per time step
<i>midband</i>	(float) Midband
<i>midband_mode</i>	(int) Midband mode
<i>midband_point_profile_id</i>	(string) Midband point profile ID
<i>min_control_signal_value</i>	(float) Minimum control signal value
<i>on_off_max_signal_mode</i>	(int) On/off maximum signal mode
<i>on_off_max_signal_profile_id</i>	(string) On/off max signal profile ID
<i>orientation</i>	(float) Orientation
<i>prop_min_signal_profile_id</i>	(string) Proportional minimum signal profile ID
<i>proportional_bandwidth</i>	(float) Proportional bandwidth
<i>proportional_controller_id</i>	(string) Proportional controller ID
<i>proportional_min_signal_mode</i>	(int) Proportional minimum signal mode
<i>radiant_fraction</i>	(float) Radiant fraction
<i>sensed_variable</i>	(int) Sensed variable
<i>set_point</i>	(float) Set point
<i>set_point_mode</i>	(int) Set point mode
<i>set_point_profile_id</i>	(string) Set point profile ID
<i>slope</i>	(float) Slope

6.1.7.3.5 HVACControllerWithSensor

Interface for HVAC controller with sensor

6.1.7.3.6 HVACControllerWithDifferentialSensor

Interface for HVAC controller with differential sensor

6.1.7.4 Multiplex

6.1.7.4.1 HVACMultiplex

Interface for HVAC multiplex

6.1.7.4.1.1 Attributes Defined Here

<code>layer_editing_mode</code>	(HVACMultiplexLayerEditingMode)	Multiplex layer editing mode
<code>number_of_layers</code>	(int)	Number of layers in the multiplex

6.1.7.5 Networks

6.1.7.5.1 HVACNetwork

Interface for HVAC network

6.1.7.5.1.1 Methods Defined Here

get_component_by_id(id) -> [HVACComponent](#)

Get component by ID

get_controller_by_id(id) -> [HVACController](#)

Get controller by ID

get_multiplex_by_id(multiplex id) -> [HVACMultiplex](#)

Get multiplex by its ID

get_multiplex_on_prototype_system(system id) -> [HVACMultiplex](#)

Get multiplex by system ID

get_system_by_id(system id) -> [HVACPrototypeSystem](#)

Get system by ID

is_updated_post_sizing (system id) -> bool

True if HVAC network was updated post-sizing, False otherwise

load_network(network name) -> HVACNetwork

Load the given HVAC network

has_t24_non_compliant_systems([HVACSysClassification](#) sys_classification) -> bool

Whether the network has Title 24 non-compliant systems. True if so, False otherwise.

6.1.7.5.1.2 Static Methods Defined Here

load_network(network name) -> HVACNetwork

Load the given HVAC network

6.1.7.5.1.3 Attributes Defined Here

<i>chilled_water_loop_ids</i>	(list) Chilled water loop IDs (string) in the network
<i>chilled_water_loops</i>	(list) Chilled water loops (HVACChilledWaterLoop) in the network
<i>components</i>	(list) Components (HVACComponent) in the network
<i>controllers</i>	(list) Controllers (HVACController) in the network
<i>dhw_systems</i>	(list) Domestic Hot Water (DHW) systems (HVACDHWSystem) in the network
<i>generic_heat_sources</i>	(list) Generic heat sources (HVACGenericHeatSource) in the network
<i>hot_water_loop_ids</i>	(list) Hot water loop IDs (string) in the network
<i>hot_water_loops</i>	(list) Hot water loops (HVACHotWaterLoop) in the network

<code>multiplexes</code>	(list) Multiplexes (HVACMultiplex) in the network
<code>name</code>	(string) Network name
<code>non_master_rooms</code>	(list[str]) A list of all non-master room IDs in HVAC zones
<code>path</code>	(string) Network path
<code>systems</code>	(list) Systems (HVACPrototypeSystem) in the network
<code>systems_dict</code>	(dict) Systems (HVACPrototypeSystem) in the network, keyed by system ID
<code>vrf_systems</code>	(list) VRF Systems (HVACVRFSystem) in the network

6.1.7.6 Network Objects

6.1.7.6.1 HVACNetworkObject

Interface for HVAC network object

6.1.7.6.1.1 Attributes Defined Here

<i>available_system_links</i>	(list) Available system links (HVACSystemLink)
<i>id</i>	(string) ID of the network
<i>is_selected</i>	(bool) Whether the network is selected. True if so, false otherwise.
<i>multiplex_id</i>	(string) Multiplex ID
<i>multiplex_layer_number</i>	(int) Multiplex layer number
<i>network</i>	(HVACNetwork) Network
<i>network_object_type</i>	(HVACNetworkObjectType) Network object type
<i>reference</i>	(string) Reference of the network
<i>system_id</i>	(string) System ID
<i>system_link</i>	(HVACSystemLink) System link

6.1.7.7 Prototype Systems

6.1.7.7.1 HVACPrototypeSystem

Interface for HVAC Prototype system

6.1.7.7.1.1 Methods Defined Here

get_cooling_design_load(layer number) -> float

Gets the cooling design load

get_cooling_design_load_per_area(layer number) -> float

Gets the cooling design load per area

get_cooling_design_load_per_area_inversed(layer number) -> float

Gets the cooling design load per area inversed

get_cooling_max_primary_airflow(layer number) -> float

Gets the cooling maximum primary airflow

get_heating_design_load(layer number) -> float

Gets the heating design load

get_heating_design_load_per_area(layer number) -> float

Gets the heating design load per area

get_heating_design_load_per_area_inversed(layer number) -> float

Gets the heating design load per area inversed

get_heating_max_primary_airflow(layer number) -> float

Gets the heating maximum primary airflow

get_node_data() -> dict

Gets a dictionary of node data. Possible dictionary keys are:

multiplex_inner_supply_node (list) Multiplex inner supply node list

multiplex_outer_supply_node (list) Multiplex outer supply node list

multiplex_return_node (list) Multiplex return node list

ptu_inlet_node (int) PTU inlet node

return_node (int) Return node

get_oa_percentage_of_max_cooling_primary_sa(layer number) -> float

Gets the outdoor airflow of maximum cooling primary supply air percentage

get_oa_percentage_of_max_heating_primary_sa(layer number) -> float

Gets the outdoor airflow of maximum heating primary supply air percentage

get_peak_data(layer number) -> dict

Gets a dictionary of peak data. Possible dictionary keys are:

<code>room_peak_device_type_cooling</code>	(int) Room peak device cooling
<code>room_peak_device_type_heating</code>	(int) Room peak device heating
<code>room_peak_devices_cooling</code>	(list) List of room peak device cooling values
<code>room_peak_devices_heating</code>	(list) List of room peak device heating values
<code>system_peak_device_type_cooling</code>	(int) System peak device cooling
<code>system_peak_device_type_heating</code>	(int) System peak device heating
<code>system_peak_devices_cooling</code>	(list) List of peak devices cooling
<code>system_peak_devices_heating</code>	(list) List of peak devices heating
<code>zone_peak_device_type_cooling</code>	(int) Zone peak device cooling
<code>zone_peak_device_type_heating</code>	(int) Zone peak device heating
<code>zone_peak_devices_cooling</code>	(list) List of zone peak devices cooling values
<code>zone_peak_devices_heating</code>	(list) List of zone peak devices heating values

get_room_cooling_peak_month(layer number) -> string

Gets the room cooling peak month

get_room_cooling_peak_time(layer number) -> string

Gets the room cooling peak time

get_room_heating_peak_month(layer number) -> string

Gets the room heating peak month

get_room_heating_peak_time(layer number) -> string

Gets the room heating peak time

get_room_id_for_umlh(layer number) -> string

Gets the ID of the assigned space for the specified multiplex layer. If this ID is an HVAC zone, it will instead return the master room ID.

get_room_occupancy_data() -> dict

Gets the room occupancy for each room associated with prototype system. Room ID used as key for dict

get_room_or_zone(layer number : int) -> Optional[HVACRoom | HVACZone]

Gets the assigned principle HVAC Zone or Room component for the prototype system if Zones or rooms is used.

get_space_id(layer number) -> string

Gets the ID of the assigned space for the specified multiplex layer. This is the ID of the assigned HVAC zone if the prototype system uses HVAC zones, or the room ID if the prototype system uses rooms.

get_space_ids(layer number) -> list

Gets a list of IDs for the assigned space for the specified multiplex layer. This is the IDs of the zoned rooms if the prototype system uses HVAC zones, or a single room ID if the prototype system uses rooms.

get_system_configuration_option() -> SystemConfigurationOptions (enum)

get_system_parameters() -> [SystemParameters](#)

Gets the system parameters for this system

6.1.7.7.1.2 Attributes Defined Here

<code>components</code>	(list) Components (HVACComponent) in the system
<code>controllers</code>	(list) Controllers (HVACController) in the system
<code>is_sizing_enabled</code>	(bool) Whether or not sizing is enabled. True if so, False otherwise
<code>is_valid</code>	(bool) Whether or not the system is valid. True if so, False otherwise
<code>multiplex</code>	(HVACMultiplex) The system multiplex
<code>number_of_layers</code>	(int) Number of layers in the system
<code>standard</code>	(string) Standard
<code>system_type</code>	(HVACPrototypeSystemType) System type

6.1.7.8 System Curves

6.1.7.8.1 HVACSystemCurve

Interface for HVAC system curve

6.1.7.8.1.1 Attributes Defined Here

<code>curve_description</code>	(str) Curve description
<code>curve_name</code>	(str) Curve name
<code>curve_role</code>	(HVACSystemCurveRole) Curve role
<code>curve_role_string</code>	(str) Curve role
<code>curve_type</code>	(HVACSystemCurveType) Curve type

6.1.7.9 SystemParameters

Interface for HVAC system parameters

6.1.7.9.1 Methods Defined Here

get_schedules() -> dictionary

Get a dictionary of schedules data.

Possible dictionary entries:

after_hours_operation
contaminant_schedule_profile
control_scheme
design_max_flow_rate
exhaust_fan_operation
hvac_system_fan_operation_closed_periods
hvac_system_fan_operation_occupied_hours_and_extension_periods
is_link_operation_and_availability_profiles
is_optimal_start
minimum_outside_air_ventilation
morning_startup_time
percent_flow_during_unoccupied_hours
system_availability
system_availability_schedule
system_occupancy_schedule
system_operation_schedule
zone_recirculation_fan_operation_closed_periods
zone_recirculation_fan_operation_occupied_hours_and_extension_periods

get_system_parameters(layer_number) -> dictionary

Get dictionary of system parameter data.

Possible dictionary entries:

autosized_system_air_fan_max_flow_rate
basis_for_system_outdoor_air_requirement
chilled_water_loop_circuit
cooling_coil_model_type
cooling_coil_system_type
direct_humidification_efficiency
energy_recovery_latent_heat
energy_recovery_option
energy_recovery_power
energy_recovery_sensible_heat
heat_source_or_loop
heating_coil_model_type
heating_coil_system_type
heating_mode_max
indirect_humidification_efficiency
is_direct_evaporative_cooling

is_energy_recovery_exhaust_air_available
 is_energy_recovery_used
 is_indirect_evaporative_cooling
 is_one_hundred_percent_outside_air_system
 is_outdoor_air_economizer_dewpoint_temperature_high_limit
 is_outdoor_air_economizer_differential_enthalpy_limit
 is_outdoor_air_economizer_drybulb_temperature_high_limit
 is_outdoor_air_preheat_coil
 is_system_supply_air_dewpoint_temperature_high_limit
 is_system_supply_air_relative_humidity_low_limit
 outdoor_air_economizer_dewpoint_temperature_high_limit
 outdoor_air_economizer_differential_enthalpy_limit
 outdoor_air_economizer_drybulb_temperature_high_limit
 outdoor_air_preheat_coil
 override_system_air_fan_autosizing
 prn_baseline_system_pressure_drop_adjustment
 reset_system_minimum_outside_air_per_zone
 system_average_outdoor_air_fraction_cooling
 system_average_outdoor_air_fraction_heating
 system_cooling_coil_minimum
 system_exhaust_airflow_per_zone_requirements
 system_exhaust_airflow_per_zone_requirements_per_area
 system_minimum_outdoor_air_intake
 system_minimum_outdoor_air_intake_make_up
 system_minimum_outdoor_air_intake_make_up_per_area
 system_minimum_outdoor_air_intake_per_area
 system_minimum_sat
 system_sat_reset
 system_supply_air_dewpoint_temperature_high_limit
 system_supply_air_relative_humidity_low_limit
 system_supply_fan_design_flow_rate
 system_supply_fan_design_flow_rate_per_area
 system_supply_fan_minimum_flow
 system_uncorrected_outdoor_air_requirement
 system_ventilation_efficiency
 total_zone_minimum_primary_outside_air_flow_rates
 ventilation_air_tampering_maximum
 ventilation_air_tampering_minimum
 ventilation_design_primary_airflow_cooling
 ventilation_design_primary_airflow_heating

get_zone_temperature_humidity_equipment(layer_number) -> dictionary

Get dictionary of zone temperature, humidity and equipment data.
Possible dictionary entries:

chilled_water_loop_circuit
 cooling_availability_schedule_profile_id
 cooling_coil_leaving_air_temperature
 cooling_coil_model_type

cooling_coil_system_type
 cooling_control_throttling_range
 cooling_design_supply_air_temperature
 cooling_design_zone_air_temperature
 cooling_setback_setpoint
 cooling_setpoint
 cooling_setpoint_operation_schedule_used
 cooling_setpoint_switching_schedule_profile_id
 cooling_setpoint_variation
 cooling_source
 cooling_system_operation_schedule_used
 cooling_thermostat_temp_setpoint_used
 dehumidification
 heating_availability_schedule_profile_id
 heating_coil_leaving_air_temperature
 heating_coil_model_type
 heating_coil_system_type
 heating_control_throttling_range
 heating_design_supply_air_temperature
 heating_design_zone_air_temperature
 heating_setback_setpoint
 heating_setpoint
 heating_setpoint_operation_schedule_used
 heating_setpoint_switching_schedule_profile_id
 heating_setpoint_variation
 heating_source
 heating_system_operation_schedule_used
 heating_thermostat_temp_setpoint_used
 humidification
 is_force_cooling_airflow
 is_high_limit_only_when_outside_air_ventilation_engaged
 is_update_space_data
 system_level_cooling_equipment_used
 system_level_heating_equipment_used
 thermal_template
 zone_relative_humidity_high_limit
 zone_relative_humidity_low_limit
 zone_rh_high_limit_evaporated_cooling

get_zone_ventilation_exhaust(layer_number) -> dictionary

Get dictionary of zone ventilation & exhaust data.

Possible dictionary entries:

621_default_occupancy_per_are
 621_default_occupancy_used
 621_occupancy
 ashrae_170_function_category
 ashrae_621_version
 basis_for_zone_outside_air_requirement

breathing_zone
 breathing_zone_per_area
 cooling_air_distribution_effectiveness
 cooling_discharge_airflow
 cooling_discharge_outdoor_air_fraction
 cooling_percent_max_primary_airflow
 cooling_system_fraction_zone_recirculated_air
 cooling_zone_outdoor_airflow
 cooling_zone_outdoor_airflow_per_area
 cooling_zone_primary_air_fraction_of_total_discharge
 cooling_zone_primary_airflow
 cooling_zone_primary_airflow_outdoor_air_fraction
 cooling_zone_ventilation_efficiency
 dcv_co2_band_maximum
 dcv_co2_band_minimum
 dcv_minimum_when_occupied_volume_per_second
 dcv_minimum_when_occupied_volume_per_second_per_area
 dcv_reduction_when_unoccupied_used
 dcv_via_sensors_used
 exhaust_air_change_requirement
 exhaust_air_change_requirement_air_changes_per_hour
 exhaust_air_change_requirement_data
 exhaust_air_change_requirement_unit
 exhaust_air_change_requirement_used
 exhaust_air_change_requirement_volume_per_second
 exhaust_air_change_requirement_volume_per_second_per_person
 exhaust_air_change_requirement_volume_per_second_per_square_area
 exhaust_air_change_requirement_volume_per_second_per_unit
 exhaust_air_occupancy_id
 exhaust_air_occupancy_units
 exhaust_air_occupancy_used
 exhaust_airflow
 exhaust_airflow_adjacent_space
 force_zone_exhaust_used
 force_zone_outside_air_used
 heating_air_distribution_effectiveness
 heating_discharge_airflow
 heating_discharge_outdoor_air_fraction
 heating_percent_max_primary_airflow
 heating_system_fraction_zone_recirculated_air
 heating_zone_outdoor_airflow
 heating_zone_outdoor_airflow_per_area
 heating_zone_primary_air_fraction_of_total_discharge
 heating_zone_primary_airflow
 heating_zone_primary_airflow_outdoor_air_fraction
 heating_zone_ventilation_efficiency
 is_dcv
 is_dcv_force_vav_fully_open
 is_percent_cooling_max_primary_airflow

is_percent_heating_max_primary_airflow
 is_zone_supply_one_hundred_percent_transfer_air
 make_up_air_option
 make_up_air_transfer_percentage
 occupancy_category
 occupant_diversity
 occupant_diversity_factor
 outdoor_air_change_percent_requirement_volume
 outdoor_air_change_requirement_air_changes_per_hour
 outdoor_air_change_requirement_link
 outdoor_air_change_requirement_total_volume_per_second
 outdoor_air_change_requirement_used
 outdoor_air_change_requirement_volume
 outdoor_air_change_requirement_volume_per_second_per_area
 outdoor_air_change_requirement_volume_per_second_per_person
 primary_air_change_requirement
 primary_air_change_requirement_air_changes_per_hour
 primary_air_change_requirement_used
 primary_air_change_requirement_volume_per_second
 primary_air_change_requirement_volume_per_second_per_area
 primary_air_change_requirement_volume_per_second_per_person
 supply_air_transfer
 supply_air_transfer_used
 transfer_airflow_rate
 ventilation_coefficient_ra
 ventilation_coefficient_rp
 ventilation_increase
 ventilation_max_requirement
 ventilation_requirement

get_zone_loads_airflows() -> dictionary

Get dictionary of zone loads & supply airflows data.

Possible dictionary entries:

airside_cooling_load_after_room_contribution
 airside_heating_load_after_room_contribution
 cooling_adjusted_sat
 cooling_airflow_required_latent
 cooling_airflow_required_latent_per_area
 cooling_design_load_oversizing_factor
 cooling_design_loads_with_oversizing_latent
 cooling_design_loads_with_oversizing_sensible
 cooling_design_max_airflow
 cooling_design_max_airflow_per_area
 cooling_design_supply_air_temperature
 cooling_design_supply_delta_t
 cooling_design_temperature_for_airflow
 cooling_design_temperature_gain_for_airflow_sizing_used
 cooling_design_temperature_source

cooling_design_zone_load_latent
 cooling_design_zone_load_sensible
 cooling_load_for_terminal_unit
 cooling_max_airflow_for_terminal_unit
 cooling_max_airflow_for_terminal_unit_per_area
 cooling_system_minimum_sat_used
 dual_maximum_vav
 heating_adjusted_sat
 heating_design_load_oversizing_factor
 heating_design_loads_with_oversizing
 heating_design_max_airflow
 heating_design_max_airflow_per_area
 heating_design_supply_air_temperature
 heating_design_supply_delta_t
 heating_design_temperature_for_airflow
 heating_design_temperature_gain_for_airflow_sizing_used
 heating_design_temperature_source
 heating_design_zone_load
 heating_load_for_terminal_unit
 heating_max_airflow_for_terminal_unit
 heating_max_airflow_for_terminal_unit_per_area
 heating_system_minimum_sat_used
 is_adjust_cooling_airflow
 is_adjust_heating_airflow
 is_allow_cooling_undersized
 is_allow_heating_undersized
 is_cooling_design_max_airflow_derived
 is_force_cooling_at_least_equal_to_ventilation_rate
 is_force_cooling_to_maintain_relative_humidity_setpoint
 is_force_heating_at_least_equal_to_ventilation_rate
 is_heating_design_max_airflow_derived
 percent_of_sat
 percent_of_sat_for_airflow_sizing_used
 percent_sensible_cooling_load
 percent_sensible_heating_load
 transfer_air_contribution_cooling
 transfer_air_contribution_heating

get_zone_airflows_turndown_engineering(layer_number) -> dictionary

Get dictionary of zone airflows, turndown & engineering checks data.

Possible dictionary entries are:

active_beam_induced_air_ratio
 active_beam_or_induction_unit
 ashrae_621_voz_air_changes_per_hour
 ashrae_621_voz_volume_per_area
 ashrae_621_voz_volume_per_person
 ashrae_621_voz_volume_per_second
 constant_volume_cooling_airflow

constant_volume_cooling_airflow_per_area
 constant_volume_heating_airflow
 constant_volume_heating_airflow_per_area
 constant_volume_primary_airflow
 constant_volume_primary_airflow_per_area
 cooling_design_load
 cooling_design_load_per_area
 cooling_design_load_per_area_inversed
 dcv_co2
 dcv_min
 doas_primary_airflow
 doas_primary_airflow_per_area
 dynamic_reset_of_system_minimum
 exhaust_airflow
 exhaust_airflow_per_area
 fpb_recirculated_airflow_percentage
 fpb_recirculated_airflow_percentage_used
 heating_design_load
 heating_design_load_per_area
 heating_design_load_per_area_inversed
 is_active_beam_layer
 is_dual_maximum_vav
 is_force_design_minimum_airflow_at_least_equal_to_required_ventilation_rate
 is_occupants_served_by_system_derived
 is_terminal_unit_fan_control
 is_ventilation_make_up_for_ea
 manually_specified_target_air_changes_per_hour
 manually_specified_target_volume_per_second
 manually_specified_target_volume_per_second_per_area
 manually_specified_target_volume_per_second_per_person
 min_primary_supply_airflow
 min_primary_supply_airflow_per_area
 minimum_outside_air
 minimum_outside_air_per_area
 minimum_outside_air_per_person
 minimum_primary_airflow
 minimum_primary_airflow_or_vav_turndown
 minimum_primary_airflow_per_area
 minimum_primary_airflow_turndown
 minimum_primary_airflow_turndown_per_area
 occupant_density
 occupant_number
 occupants_served_by_system
 outdoor_air_requirement_at_max_supply_airflow
 outdoor_air_requirement_at_minimum_supply_airflow
 recirculated_secondary_airflow_max
 recirculated_secondary_airflow_max_per_area
 recirculated_secondary_airflow_minimum
 recirculated_secondary_airflow_minimum_per_area

```

required_make_up_air_changes_per_hour
required_make_up_volume_per_second
required_make_up_volume_per_second_per_area
required_make_up_volume_per_second_per_person
required_min_air_changes_per_hour
required_min_volume_per_person
required_min_volume_per_second
required_min_volume_per_second_per_area
requirement_as_percent_max_cooling_airflow
requirement_as_percent_max_heating_airflow
requirement_as_percent_minimum_airflow
return_at_max_ea_min_airflow
return_at_max_ea_min_airflow_per_area
system_exhaust_airflow
system_exhaust_airflow_per_area
system_ventilation_efficiency
terminal_fan_unit_control
total_floor_area_served
total_volume_served
transfer_at_max_ea_min_airflow
transfer_at_max_ea_min_airflow_per_area
zone_floor_area
zone_ventilation_max_total_required_air_changes_per_hour
zone_ventilation_max_total_required_air_changes_per_second_per_person
zone_ventilation_max_total_required_volume_per_second
zone_ventilation_max_total_required_volume_per_second_per_area
zone_volume

```

get_zone_airflow_distribution (layer_number) -> dictionary

Get dictionary of zone airflows, turndown & engineering checks data.

Possible dictionary entries are:

```

cooling_load_sensible
cooling_load_latent
heating_load
ventilation_pct
cooling_airflow_pct
cooling_airflow
heating_airflow
ventilation_airflow

```

6.1.8 IECC

Interface for IECC

6.1.8.1 Methods Defined Here

get_site_permit_software() -> Dictionary

Get a dictionary of site, permit and software data from IECC project details.

The dictionary contains the following entries:

```
title
address_1
address_2
city
state
zip_code
permit_no
permit_date
program_1
program_2
```

get_owner_agent() -> Dictionary

Get a dictionary of owner and agent data from IECC project details.

The dictionary contains the following entries:

```
first_name
last_name
company
address_1
address_2
city
state
zip_code
phone_number
email
```

get_designer_contractor() -> Dictionary

Get a dictionary of designer and contractor data from IECC project details.

The dictionary contains the following entries:

```
first_name
last_name
company
address_1
address_2
city
state
zip_code
phone_number
email
```

set_owner_agent(data)

Set owner and agent entries for IECC project details from a dictionary.
The dictionary can contain the following entries:

first_name
last_name
company
address_1
address_2
city
state
zip_code
phone_number
email

set_site_permit_software(data)

Set site, permit and software entries for IECC project details from a dictionary.
The dictionary can contain the following entries:

title
address_1
address_2
city
state
zip_code
permit_no
permit_date
program_1
program_2

set_designer_contractor(data)

Set designer and contractor entries for IECC project details from a dictionary
The dictionary can contain the following entries:

first_name
last_name
company
address_1
address_2
city
state
zip_code
phone_number
email

6.1.9 ImportGBXML

Interface for GBXML import

6.1.9.1 Methods Defined Here

Import_file(file_name, heal_geometry, cap_mode, cap_height)

Import a gbXML file

6.1.9.2 Enums Defined Here

<i>VolumeCapMode</i>	none, height_of_largest_ceiling, fixed_height
----------------------	---

6.1.10 Mv2 (Model Viewer 2)

Interface for Model Viewer 2.

6.1.10.1 Methods Defined Here

take_snapshot(file_name = 'Snapshot', path = "", view_mode = shaded, components = False)

Takes a snapshot of the model, using model viewer 2, and saves it to disk as a png image.

Parameters:

<i>file_name</i>	(string) output filename. Extension .png will be added automatically
<i>path</i>	(string) location where to save file. Empty string (") means current project folder
<i>view_mode</i>	(mv2_viewmode enum) rendering mode
<i>components</i>	(Bool) True to include components, else False. Component viewmode will force components on.

If path or file_name are specified, the caller is responsible for making sure they are valid.

take_snapshot_or_create_blank_image(file_name = 'Snapshot', path = "", view_mode = shaded, components = False)

Checks user preference for report images. If Active Model is selected, this function takes a snapshot of the model, using model viewer 2, and saves it to disk as a png image. If None is selected, this function creates a blank placeholder image to be used in reports.

Parameters:

<i>file_name</i>	(string) output filename. Extension .png will be added automatically
<i>path</i>	(string) location where to save file. Empty string (") means current project folder
<i>view_mode</i>	(mv2_viewmode enum) rendering mode
<i>components</i>	(Bool) True to include components, else False. Component viewmode will force components on.

If path or file_name are specified, the caller is responsible for making sure they are valid.

6.1.10.2 Enums Defined Here

mv2_viewmode	shaded, textured, hidden_line, xray, component
--------------	--

6.1.11 NECB

Interface for NECB

6.1.11.1 Methods Defined Here

get_designer_contractor() -> Dictionary

Get a dictionary of designer and contractor data from NECB project details.

The dictionary contains the following entries:

first_name

last_name

company

address_1

address_2

city

state

zip_code

phone_number

email

get_owner_agent() -> Dictionary

Get a dictionary of owner and agent data from NECB project details.

The dictionary contains the following entries:

first_name

last_name

company

address_1

address_2

city

state

zip_code

phone_number

email

get_site_permit_software() -> Dictionary

Get a dictionary of site, permit and software data from NECB project details.

The dictionary contains the following entries:

title

address_1

address_2

city

state

zip_code

permit_no

permit_date

program_1

program_2

set_designer_contractor(data)

Set designer and contractor entries for NECB project details from a dictionary

The dictionary can contain the following entries:

first_name

last_name

company

address_1

address_2

city

state

zip_code

phone_number

email

set_owner_agent(data)

Set owner and agent entries for NECB project details from a dictionary.

The dictionary can contain the following entries:

first_name

last_name

company

address_1

address_2

city

state

zip_code

phone_number

email

set_site_permit_software(data)

Set site, permit and software entries for NECB project details from a dictionary.

The dictionary can contain the following entries:

title

address_1

address_2

city

state

zip_code

permit_no

permit_date

program_1

program_2

6.1.12 PRM

Interface for PRM

6.1.12.1 Methods Defined Here

get_draw_pattern() -> draw_pattern

Gets the draw pattern

get_thermal_comfort_room_categories_data() -> list

Returns a list of dictionaries for each space.

The dictionary contains the following entries:

activity

analyse

id

name

set_thermal_comfort_room_categories_data(data)

Set thermal comfort room categories data from a list of dictionaries.

The number of entries in the list must match the number of spaces.

The dictionary must contain the following entries:

activity

analyse

id

name

get_unmet_load_hour_parameters() -> dictionary

Get unmet load hour parameters.

Returns a dictionary containing 'cooling_tolerance', 'heating_tolerance' and 'override_hvac_throttling_ranges'.

set_unmet_load_hour_parameters(data)

Set unmet load hour parameters with the dictionary of data.

The dictionary must contain 'cooling_tolerance', 'heating_tolerance' and 'override_hvac_throttling_ranges'.

get_site_permit_software() -> Dictionary

Get a dictionary of site, permit and software data from PRM project details.

The dictionary contains the following entries:

address_1

address_2

city

permit_date

permit_no

state

title

zip_code

get_owner_agent() -> Dictionary

Get a dictionary of owner and agent data from PRM project details.

The dictionary contains the following entries:

address_1
address_2
city
company
email
first_name
last_name
phone_number
state
zip_code

get_designer_contractor() -> Dictionary

Get a dictionary of designer and contractor data from PRM project details.

The dictionary contains the following entries:

address_1
address_2
city
company
email
first_name
last_name
phone_number
state
zip_code

get_florida() -> Dictionary

Get a dictionary of Florida data

The dictionary contains the following entries:

description
jurisdiction
class
building_floor_area
num_stories

set_florida(data)

Set Florida data from a dictionary.

The dictionary can contain the following entries:

description
jurisdiction
class
building_floor_area
num_stories

get_florida_certifications() -> Dictionary

Get a dictionary of Florida certifications data

The dictionary contains the following entries:

architect
architect_reg
building_official
building_official_date
electrical_designer
electrical_designer_reg
lighting_designer
lighting_designer_reg
mechanical_designer
mechanical_designer_reg
owner_agent
owner_agent_date
plumbing_designer
plumbing_designer_reg
preparer
preparer_date

set_florida_certifications(data)

Set Florida certifications data from a dictionary.

The dictionary can contain the following entries:

architect
architect_reg
building_official
building_official_date
electrical_designer
electrical_designer_reg
lighting_designer
lighting_designer_reg
mechanical_designer
mechanical_designer_reg
owner_agent
owner_agent_date
plumbing_designer
plumbing_designer_reg
preparer
preparer_date

set_owner_agent(data)

Set owner and agent entries for PRM project details from a dictionary.

The dictionary can contain the following entries:

address_1
address_2
city
company
email
first_name
last_name

phone_number

state

zip_code

set_site_permit_software(data)

Set site, permit and software entries for PRM project details from a dictionary.

The dictionary can contain the following entries:

address_1

address_2

city

permit_date

permit_no

state

title

zip_code

set_designer_contractor(data)

Set designer and contractor entries for PRM project details from a dictionary

The dictionary can contain the following entries:

address_1

address_2

city

company

email

first_name

last_name

phone_number

state

zip_code

6.1.12.2 Enums Defined Here

<i>prm_mode</i>	none, prm_2004_2007, prm_2010, prm_2013, iecc, necb, prm_2016, t24
<i>draw_pattern</i>	unset, very_small, low, medium, high

6.1.13 ProjectInfo

Interface for project information

6.1.13.1 Methods Defined Here

get() -> dictionary

Get project information.

Returns a dictionary with the following keys:

address
building_owner
conditioned_floor_area
design_team
energy_analyst
project_name

set(data)

Set project information using a dictionary of data.

Dictionary can contain the following keys:

address
building_owner
conditioned_floor_area
design_team
energy_analyst
project_name

6.1.14 ResultsReader

Support for reading simulation result files (APS files).

Basic usage:

```
f = iesve.ResultsReader.open(filename)
x = f.get_results('Total electricity', 'Total electricity', 'e')
f.close()
```

6.1.14.1 Notes on Reading APS files

The ResultsReader API has many references to variables and variable levels. This section will explain Variables and their details a bit further.

Variables have 2 names:

- an internal name which is used in the APS file. Pass this name to the various reading APIs that are part of the ResultsReader class
- a display name, which is shown in Vista / VistaPro. Attempting to read this from an APS will result in an error.

The variable level (also known as model level flag) is represented by a single character as shown in the table below, together with the corresponding API for reading the data.

Surface level results are available only for those rooms that have detailed output switched on for a surface-level result (Apache Simulation output options: Surface temperatures, External / internal incident solar flux)

Opening level results are available only if Macroflo was included in simulation.

For more information on reading APS data, and information on how to map a Vista / VistaPro display string to a variable, see the sample on reading APS data.

Valid Model Level flags (character)	API for reading data at this level
w = weather	get_weather_results
z = room level (zone)	get_room_results
v = apache systems misc	get_apache_system_results
j = apache systems energy	get_apache_system_results
r = apache systems carbon	get_apache_system_results
l = building loads	get_results
e = building energy	get_results
c = building carbon	get_results
s = surface level	get_surface_results
o = opening level	get_opening_results
n = HVAC node level	get_hvac_node_results
h = HVAC component level	get_hvac_component_results

6.1.14.2 Methods Defined Here

open(filename) -> `iesve.ResultsReader`

Open an APS file and return a ResultsReader instance. Throws exception on error.

open_aps_data(filename) -> `int`

Open an APS file, returns 1 on success, throws exception on file not found.

close() -> `None`

Close the results file, and releases all resources associated with the file. It is recommended to close files as soon as possible to save resources.

get_all_apache_system_results(system_id, aps_var, variable level, [start_day], [end_day]) ->

Dictionary of numpy array of floats, keyed by variable display name

Get the results for specified system + variable. See units spreadsheet for available variables and matching level. See `get_results` for `start_day` and `end_day` details.

get_all_component_process_results(room_id, index_in_room, aps_var, var_level, [start_day], [end_day])

-> Dictionary of numpy array of floats, keyed by variable display name

Read the results for specified process variable. See `get_results` for `start_day` and `end_day` details.

get_all_opening_results(room_id, surface_index, opening_index, aps_var, [start_day], [end_day])->

Dictionary of numpy array of floats, keyed by variable display name

Get the results for specified opening + variable. Obtain `room_ID` from `VEBody.get_index()`, `surface_index` from `VESurface.index` and `opening_index` from `VEGeometry.get_properties()`. See units spreadsheet for available variables for `aps_var`. See `get_results()` for `start_day` and `end_day` details.

get_all_process_results(process_name, aps_var [start_day], [end_day]) ->

Dictionary of numpy array of floats, keyed by variable display name

Read the results for specified process variable. See `get_results` for `start_day` and `end_day` details.

get_all_results(aps_var, var_level, [start_day], [end_day]) ->

Dictionary of numpy array of floats, keyed by variable display name

Get the model-level results for specified variable.

See units spreadsheet for information on variable + level

If `start_day` and/or `end_day` are omitted, all available results will be returned. `start_day = 1` means January 1, `end_day = 365` is December 31st.

Check APS file `firstday` and `lastday` properties for valid range for file.

Note: results are always returned in metric units

get_all_room_results(room_id, aps_var, var_level, [start_day], [end_day]) ->

Dictionary of numpy array of floats, keyed by variable display name

Get the results for specified room + variable. See units spreadsheet for available variables and matching level. See `get_results` for `start_day` and `end_day` details.

get_all_surface_results(room_id, aps_handle, aps_var, [start_day], [end_day]) -> Dictionary of numpy array of floats, keyed by variable display name

Get the results for specified surface + variable. See units spreadsheet for available variables for aps_var. Obtain room_id from [VEBody](#) and aps_handle from [VESurface.get_properties\(\)](#). See [get_results\(\)](#) for start_day and end_day details.

get_all_weather_results(aps_var, [start_day], [end_day]) ->

Dictionary of numpy array of floats, keyed by variable display name

Get the results for a weather variable. See units spreadsheet for available variables. See [get_results](#) for start_day and end_day details.

get_apache_system_results(system_id, aps_var, vista_var, var_level, start_day = -1, end_day = -1)

-> Numpy array of floats

Get the results for specified system + variable. See variables list for available variables and matching level. See [get_results](#) for start_day and end_day details.

get_apache_systems() -> list

Get list of Apache Systems in the results file. Each list entry holds a tuple of (system name, system ID).

get_component_objects()

-> [(room ID, index in room, component name, [(variable name, variable unit, variable level)])]

Return the list of process component objects for this results file. The variable unit can be used as an index into the units list returned from [get_units\(\)](#).

get_component_process_results(room_id, index_in_room, aps_var, vista_var, var_level,

start_day = -1, end_day = -1)

-> Numpy array of floats

Read the results for specified process variable. See [get_results](#) for start_day and end_day details.

get_conditioned_sizes() -> (area, volume, number of rooms)

Returns a tuple containing the conditioned area, volume and number of rooms as recorded in the results file.

get_data_file_details(filename, date_time_format) -> dictionary

Get a dictionary of file details.

Dictionary keys are:

<i>name</i>	Filename (str)
<i>calculated</i>	File date/time (str)
<i>profile_month_start</i>	Name of start month (str)
<i>profile_month_end</i>	Name of end month (str)

get_hvac_component_results(component_id, component_type, var_name, start_day = -1, end_day = -1)

-> Numpy array of floats

Get the results for HVAC Component ID + variable. See variables list for available variables and matching level. See [get_results](#) for start_day and end_day details.

The follow component types are currently available:

1 : iesve.HVACRoom	
2 : iesve.HVACHeatingCoil	iesve.HeatingCoilModelType.simple_heating_coil
3 : iesve.HVACCoolingCoil	iesve.HeatingCoilModelType.simple_cooling_coil
4 : iesve.HVACSprayChamber	
5 : iesve.HVACSteamHumidifier	
7 : iesve.HVACJunction	
8 : iesve.HVACDuct	
10 : iesve.HVACAirToAirHeatEnthalpyExchanger	
13 : iesve.HVACBoiler	part load boiler
14 : iesve.HVACPLCChiller	
15 : iesve.HVACFan	
16 : iesve.HVACDamper	
18 : iesve.HVACUnitaryCoolingSystem	
19 : iesve.HVACDedicatedWatersideEconomizer	
20 : iesve.HVACEWCChiller	
21 : iesve.HVACEnhancedBoiler	hot water boiler
23 : iesve.HVACEACChiller	
24 : iesve.HVACDXCoolingInstance	
25 : iesve.HVACCoolingCoil	iesve.HeatingCoilModelType.advanced_cooling_coil
26 : iesve.HVACChilledWaterLoop	
27 : iesve.HVACHeatingCoil	iesve.HeatingCoilModelType.advanced_heating_coil
28 : iesve.HVACHotWaterLoop	
29 : iesve.HVACGenericHeatSource	
30 : iesve.HVACHeatPump	iesve.HVACHeatPumpType.air_water_heat_pump
31 : iesve.HVACHeatPump	iesve.HVACHeatPumpType.air_air_heat_pump
32 : iesve.HVACSolarWaterHeater	
33 : iesve.HVACHeatTransferLoop	
34 : iesve.HVACWaterAirHeatPump	
35 : iesve.HVACGenericCoolingSource	
36 : iesve.HVACRadiatorRoom	
37 : iesve.HVACChilledCeiling	
38 : iesve.HVACCondenserWaterLoop	
39 : iesve.HVACCoolingTower	
40 : iesve.HVACDryFluidCooler	
41 : iesve.HVACPreCoolingLoop	
42 : iesve.HVACWaterSourceLoop	
43 : iesve.HVACWaterWaterHeatExchanger	
44 : iesve.HVACPCMBattery	
45 : iesve.HVACPump	
50 : iesve.HVACSolarAirCollectorBIST	
51 : iesve.HVACAirFilter	
52 : iesve.HVACPlenum	
53 : iesve.HVACThermalStorageLoop	
54 : iesve.HVACThermalStorageTank	

get_hvac_node_results(node_nr, layer_nr = -1, var_name, start_day = -1, end_day = -1)

-> Numpy array of floats

Get the results for HVAC Node. Use layer_nr to specify multiplex layer, or -1 for plant-side node (outside of multiplex). See variables list for available variables and matching level. See get_results for start_day and end_day details.

get_opening_results(room_id, surface_index, opening_index, aps_var, vista_var, start_day = -1, end_day = -1) -> Numpy array of floats

Get the results for specified opening + variable. Obtain room_ID from [VEBody.get_index\(\)](#), surface_index from [VESurface.index](#) and opening_index from [VEGeometry.get_properties\(\)](#). See units spreadsheet for available variables for aps_var and vista_var. See [get_results\(\)](#) for start_day and end_day details.

get_peak_results(roomID, [variables]) -> dictionary

Get the peak results of a series of variables at room-level, for the specified room. Variables are specified by APS variable name (string). Returns the peak value for each variable, using the variable name as dictionary key.

get_process_list() -> list

Return the list of process categories (strings) for this results file.

get_process_results(process_name, aps_var, vista_var, start_day = -1, end_day = -1)

-> Numpy array of floats

Read the results for specified process variable. See [get_results](#) for start_day and end_day details.

get_process_variables(process) -> list

For the process specified (one of the return values from [GetProcessList](#)), return the available process variables. The returned list contains tuples of: (name, units). The units can be used as an index into the units dictionary returned by [get_units\(\)](#).

get_results(aps_var, vista_var, var_level, start_day = -1, end_day = -1) -> Numpy array of floats

Get the model-level results for specified variable.

See variables list for information on variable + level details, as well as footnotes at the end of this section.

If start_day and/or end_day are omitted, all available results will be returned. start_day = 1 means January 1, end_day = 365 is December 31st. start_day and end_day value of -1 means first and last available day respectively. Check first_day and last_day properties for valid range for file.

Note: results are always returned in metric units

get_room_geometry_details([room ID]) -> list of dictionaries

get_room_geometry_details(roomID) -> dictionary

Returns a list of dictionaries for all requested room IDs. You can also pass a single room ID string, in which case a single dictionary is returned. All units are in m².

Dictionary entries are:

opaque, glazed, internal_wall, external_wall, internal_glazed, num_open, num_surf
external_glazed, rooflight, door, floor, ground_floor, ceiling, roof.

Note: glazing areas have construction frame dimensions applied (net glazed area, not gross opening area).

get_room_ids() -> [roomID]

Returns a list of room IDs for all rooms in the results file.

get_room_list() -> list

Returns a list of (room name, room ID, room area, room volume) tuples for all rooms in the results file.

get_room_results(room_id, aps_var, vista_var, var_level, start_day = -1, end_day = -1)

-> Numpy array of floats

Get the results for specified room + variable. See variables list for available variables and matching level.
See `get_results` for `start_day` and `end_day` details.

`get_surface_results`(room_id, aps_handle, aps_var, vista_var, start_day = -1, end_day = -1) -> Numpy array of floats

See units spreadsheet for available variables for `aps_var` and `vista_var`.

Obtain `room_id` from `VEBody.get_index()` and `aps_handle` from `VESurface.get_properties()`.

See `get_results()` for `start_day` and `end_day` details.

`get_energy_uses`(used_only = True) -> dictionary

Returns a dictionary of all (or only the used) energy uses in the file by id.

Each item is a dictionary containing information about a specific energy use.

Dictionary fields include:

<code>id</code>	the use id (iesve.EnergyUse)
<code>name</code>	the use name (string)
<code>tied_source_id</code>	the id of the source the use is fixed to, if any (iesve.EnergySource)
<code>used</code>	whether or not the use has any aps results (bool)

`get_energy_sources`(used_only = True) -> dictionary

Returns a dictionary of all (or only the used) energy sources in the file by id.

Each item is a dictionary containing information about a specific energy source.

Dictionary fields include:

<code>cef</code>	the carbon emission factor of the source, used to convert kgC to kgCO ₂ (float)
<code>id</code>	the source id (iesve.EnergySource)
<code>name</code>	the source name (string)
<code>used</code>	whether or not the source has any aps results (bool)

`get_energy_meters`(used_only = True) -> dictionary

Returns a dictionary of all (or only the used) energy meters in the file by id.

Each item is a dictionary containing information about a specific energy meter.

Dictionary fields include:

<code>has_subs</code>	whether or not the meter has any child (sub) meters (bool)
<code>id</code>	the meter id (iesve.EnergyMeter)
<code>name</code>	the meter name (string)
<code>parent_id</code>	the id of the parent meter this meter belongs to, if any (iesve.EnergyMeter)
<code>source_id</code>	the id of the source the meter is assigned to (iesve.EnergySource)
<code>used</code>	whether or not the meter has any aps results (bool)

`get_energy_results`(use_id = iesve.EnergyUse.unspecified, source_id = iesve.EnergySource.unspecified, meter_id = iesve.EnergyMeter.unspecified, type = 'e', add_subs = -1, start_day = -1, end_day = -1)

-> Numpy array of floats or None

Returns the specified energy results given the use id, source id, meter id and type.

Setting type to 'e' or 'c' determines whether or not energy (W) or carbon (kgC) results are returned.

Setting `add_subs` to 1 or 0 determines whether or not parent meter results include or do not include child (sub) meter results.

When `add_subs` is -1 the option is ignored and the default behaviour will be applied.

See `get_results` for `start_day` and `end_day` details.

Example usage:

```
get_energy_results() -> 'Total Energy'
```

```
get_energy_results( source_id=elec ) -> 'Electricity'
get_energy_results( source_id=elec, meter_id=1 ) -> 'Electricity: Meter 1'
get_energy_results( use_id=prm_pumps ) -> 'Pumps'
get_energy_results( use_id=prm_pumps, source_id=elec ) -> 'Pumps - Electricity'
get_energy_results( use_id=prm_pumps, source_id=elec, meter_id=1 ) -> 'Pumps - Electricity: Meter 1'
```

get_energy_results_ex(use_ids = None, source_ids = None, type = 'e', start_day = -1, end_day = -1)

-> Numpy array of floats or None

Returns the aggregated energy results for the specified use(s) and/or source(s).

The use_ids argument can be None, a single use id, or a list of use ids.

The source_ids argument can be None, a single source id, or a list of source ids.

At least one use or one source must be specified.

Setting type to 'e' or 'c' determines whether or not energy (W) or carbon (kgC) results are returned.

See get_results for start_day and end_day details.

get_units() -> dictionary

Get the list of results file units that are applicable to the loaded file. The return value is a dictionary, keyed by unit type (string). This unit type corresponds to the units_type field for APS variables, as well as the units for process variables.

Each units data field consists of another dictionary with 2 fields: 'units_IP' and 'units_metric'. These provide the conversion from APS raw data to both IP and metric display values.

The available unit data fields are:

divisor

offset

display_name

To achieve conversion of the raw APS results to display_name, divide the variable by divisor, and for temperatures also add offset (to convert degrees C to Fahrenheit).

For example, to display a 'Power' units variable in IP units:

```
aps_data = resultsfile.get_results(var_aps_name, var_model_level)
peak = np.max(aps_data)
aps_units = resultsfile.get_units()
power_units = aps_units['Power']
pu_ip = power_units['units_IP']
peak_IP = (peak / pu_ip['divisor']) + pu_ip['offset']
print('{} {}'.format(peak_IP, pu_ip['display_name']))
```

get_unmet_hours() -> dictionary

Get unmet load hours data.

cooling

heating

get_variables() -> list

Get the list of results file variables that are applicable to the loaded file. The return value is a list of dictionaries, with each dictionary entry containing the various information fields for a variable.

Available data fields are:

aps_varname

APS result name. Use this string for all ResultReader functions

<i>display_name</i>	the Vista / VistaPro name for this variable
<i>model_level</i>	variable level for ResultsReader function parameters
<i>units_type</i>	results unit for this variable. Use this as index to unit dictionary

get_weather_results (aps_var, vista_var, start_day = -1, end_day = -1)
 -> Numpy array of floats

Get the results for specified weather variable. See variable list for available variables, matching level, and units. See get_results for start_day and end_day details.

6.1.14.3 Attributes Defined Here

first_day -> (int) The first simulation day that has results
last_day -> (int) The last simulation day that has results
plot_data_offset_secs -> (int) The plot data offset in seconds
results_per_day -> (int) The number of results per 24 hour period
weather_file -> (string) The weatherfile used for simulation
hvac_file -> (string) The HVAC network used for simulation
plot_year -> (int) The plot year
simulation_year -> (int) The simulation year
first_weekday_plot_year -> (int) The day-of-week (Sun = 1..) of Jan 1 in plot year

6.1.14.4 Enums Defined Here

EnergyUse	unspecified, prm_interior_lighting, prm_exterior_lighting, prm_space_heating, prm_space_cooling, prm_pumps, prm_heat_rejection, prm_fans_interior_central, prm_fans_interior_local, prm_fans_garage, prm_fans_exhaust, prm_fans_process, prm_services_water_heating, prm_receptacle_equipment, prm_interior_lighting_process, prm_refrigeration, prm_data_center_equipment, prm_cooking, prm_elevators_escalators, prm_other_process, prm_humidification, prm_chp, prm_elec_gen_chp, prm_elec_gen_wind, prm_elec_gen_pv, prm_transformer, prm_motor, prm_interior_lighting_unregulated Note: The enums <i>prm_elec_gen_wind</i> and <i>prm_elec_gen_pv</i> must be used only with the energysource enum <i>unspecified</i>
EnergySource	unspecified, elec, nat_gas, oil, coal, misc_a, misc_b, none, lpg, biogas, biomass, waste, misc_c, misc_d, misc_e, misc_f, anthracite, smokeless, dual, grid_disp_elec, misc_g, misc_h, misc_i, misc_j, misc_k, misc_l, misc_m, misc_n, misc_o, misc_p, misc_q, misc_r, misc_s, misc_t, misc_u, misc_v, misc_w, misc_x, misc_y, misc_z, district_heating, grid_disp_elec_pv
EnergyMeter	unspecified
VariableSource	system, project
VariableType	standard, derived, process, energy_meter

6.1.15 RoomAirExchange

Interface for Room Data: Air Exchanges. Used by [VERoomData](#) class to return air exchanges data for a room. The following Air Exchange types are supported:

- Infiltration
- Natural Ventilation
- Auxiliary Ventilation

6.1.15.1 Methods Defined Here

get() -> dictionary

Returns a dictionary containing the data for an air exchange. The following fields are returned:

<i>adjacent_condition_string</i>	(string)
<i>adjacent_condition_val</i>	(int) see below for values
<i>adjacent_condition_val_from_template</i>	(Bool)
<i>max_flow_from_template</i>	(Bool)
<i>max_flows</i>	(Dictionary of 4 floats, indexed by integer) Represent the various flow conversions
<i>name</i>	(string)
<i>offset_temperature</i>	(float) Only present when adjacent_condition == 2
<i>temperature_profile</i>	(string) Profile ID, only present when adjacent_condition references a profile)
<i>temperature_profile_from_template</i>	(Bool)
<i>type_val</i>	(int) See below for values
<i>units_strs</i>	(Dictionary of strings, indexed by integer) String representations of units for max_flows
<i>units_val</i>	(int) See below for values
<i>variation_profile</i>	(string) Profile ID
<i>variation_profile_from_template</i>	(Bool)

Valid values for adjacent_condition_val:

- 1 = External air
- 2 = External air + Offset Temperature
- 3 = From adjacent room
- 5 = Temperature from profile

Valid values for type_val:

- 0 = Infiltration
- 1 = Natural Ventilation
- 2 = Auxiliary Ventilation

Valid values for units_val:

- 0 = ach
- 1 = l/s

2 = l/s/m²

3 = l/s/person

4 = l/(s.m² façade)

6.1.16 RoomInternalGain

Interface for Room Data: Internal Gains. Used by [VERoomData](#) class to return internal gains data for a room. Three categories of internal gains are defined, represented by individual classes:

- RoomPowerGain
- RoomLightingGain
- RoomPeopleGain

6.1.16.1 Methods Defined Here

get () -> dictionary

Returns a dictionary containing the data for the internal gain.

RoomPowerGain

<i>diversity_factor</i>	(float)
<i>diversity_factor_from_template</i>	(Bool) True if value is from template, else False
<i>energy_source</i>	(string) Fuel string
<i>energy_source_from_template</i>	(Bool)
<i>max_latent_gains</i>	(Dictionary of float, indexed by integer)
<i>max_power_consumptions</i>	(Dictionary of float, indexed by integer)
<i>max_sensible_gains</i>	(Dictionary of float, indexed by integer)
<i>name</i>	(string)
<i>radiant_fraction</i>	(float)
<i>radiant_fraction_from_template</i>	(Bool)
<i>type_str</i>	(string) 'Machinery', 'Miscellaneous', 'Cooking', 'Computers'
<i>type_val</i>	(int) 2 = Machinery, 3 = Miscellaneous, 4 = Cooking, 5 = Computers
<i>units_strs</i>	(Dictionary of string, indexed by integer) Units for latent gains, sensible gains and power consumption dictionaries
<i>units_val</i>	(int) Input mode for this gain. 0 = W/m ² , 1 = W
<i>units_val_from_template</i>	(Bool)
<i>variation_profile</i>	(string) profile ID
<i>variation_profile_from_template</i>	(Bool)

RoomLightingGain

<i>ballast</i>	(float)
<i>ballast_from_template</i>	(Bool)
<i>design_illuminance</i>	(float) lux. Only present if lighting is defined as lux (see <i>units_val</i> , <i>units_str</i>)
<i>dimming_profile</i>	(string) profile ID
<i>dimming_profile_from_template</i>	(Bool)
<i>diversity_factor</i>	(float)
<i>diversity_factor_from_template</i>	(Bool)
<i>energy_source</i>	(string) Fuel string
<i>energy_source_from_template</i>	(Bool)
<i>installed_power_density</i>	(float) W/m ² /(100 lux). Only present if lighting is defined as lux (see <i>units_val</i> , <i>units_str</i> = 'lux')
<i>maximum_illuminance</i>	(float) lux. Only present if lighting is defined as lux (see <i>units_val</i> , <i>units_str</i>)

<code>max_power_consumptions</code>	(Dictionary of floats, indexed by int) Provides max power at various units
<code>max_sensible_gains</code>	(Dictionary of floats, indexed by int) Provides sensible gains at various units
<code>minimum_illuminance</code>	(float) lux. Only present if lighting is defined as lux (see <code>units_val</code> , <code>units_str</code>)
<code>name</code>	(string)
<code>power_units</code>	(Dictionary of strings) units for power consumption, sensible gains Dictionaries
<code>radiant_fraction</code>	(float)
<code>radiant_fraction_from_template</code>	(Bool)
<code>type_str</code>	(string) 'Fluorescent Lighting' or 'Tungsten Lighting'
<code>type_val</code>	(int) 0 = Fluorescent Lighting, 1 = Tungsten Lighting
<code>units_str</code>	(string) 'W/m²', 'W', 'lux'
<code>units_val</code>	(int) 0 = W/m², 1 = W, 2 = lux
<code>units_val_from_template</code>	(Bool)
<code>variation_profile</code>	(string) profile ID
<code>variation_profile_from_template</code>	(Bool)

RoomPeopleGain

<code>diversity_factor</code>	(float)
<code>diversity_factor_from_template</code>	(Bool)
<code>max_latent_gain_from_template</code>	(Bool)
<code>max_latent_gains</code>	(Dictionary of floats, indexed by int) Provides latent gains at various units
<code>max_sensible_gain_from_template</code>	(Bool)
<code>max_sensible_gains</code>	(Dictionary of floats, indexed by int) Provides sensible gains at various units
<code>name</code>	(string)
<code>occupancies</code>	(Dictionary of floats, indexed by int) Occupancy values (0: m²/person, 1: people). Matching units are in <code>units_strs</code>
<code>power_units</code>	(Dictionary of strings) units for latent gains, sensible gains Dictionaries
<code>type_str</code>	(string) 'People'
<code>type_val</code>	(int) 6 = People
<code>units_strs</code>	(Dictionary of string, indexed by integer) Units for occupancies dictionary. 0: m²/person, 1: people
<code>units_val</code>	(int) Input mode. 0 = m²/person, 1 = people
<code>units_val_from_template</code>	(Bool)
<code>variation_profile</code>	(string) profile ID
<code>variation_profile_from_template</code>	(Bool)

6.1.17 RoomGroups

Interface for Room Groups and HVAC Zones manipulation.

VE Room Groups consist of nested containers. At the top level, we have a number of Grouping **Schemes**. Each Scheme can hold a number of Groups. Rooms are assigned to groups. A scheme is created with a default group to hold all the unassigned rooms. Assigning a room takes it out of its current group and moves it to the new group.

VE HVAC Zones follow a similar structure. An HVAC zone is a group of rooms that functions as a unit for the purpose of HVAC system control in the context of the ApacheHVAC (ApHVAC) system methodology. At the top level, we have a number of Zone Groups. Each Zone Group can hold a number of Zones. Rooms are assigned to Zones. One of the rooms assigned to each HVAC zone is designated the zone's master room. Conditions in this room (as sensed by ApacheHVAC control sensors placed in the zone's air outflow) may be used to control conditions in all rooms in the zone.

6.1.17.1 Methods Defined Here

assign_rooms_to_group(scheme handle, group handle, [room IDs])

Assigns the rooms specified by room IDs list to the specified group. The room IDs list can be either a list of strings or a list of VEBody objects.

create_grouping_scheme(name) -> int

Creates a room grouping scheme with the given name. Returns the handle of the new grouping scheme

create_room_group(scheme_handle, group_name, colour = (0, 0, 0)) -> int

Creates a room group with the given name and colour in the provided grouping scheme. Colour is expressed as a tuple of (R,G,B) and is optional, defaulting to black. Returns the handle of the new room group

get_grouping_schemes() -> List of dictionaries

Get a list of room grouping schemes stored as dictionaries. Dictionary entries are:

<i>handle</i>	(string)
<i>name</i>	(string)

get_room_groups(grouping scheme handle) -> List of dictionaries

Get a list of room groups (stored as dictionaries) for a grouping scheme. Dictionary entries are:

<i>colour</i>	(tuple of R,G,B ints) each colour entry in range 0-255
<i>handle</i>	(int)
<i>name</i>	(string)
<i>rooms</i>	(list of strings) list of room IDs assigned to group

get_zone_groups() -> List of dictionaries

Get a list of zone groups (stored as dictionaries). Dictionary entries are:

<i>name</i>	(string)
<i>id</i>	(string)

get_zones(zone group ID) -> List of dictionaries

Get a list of zones (stored as dictionaries) for a zone group. Dictionary entries are:

<i>name</i>	(string)
<i>id</i>	(string)
<i>rooms</i>	(list of strings) list of room IDs assigned to zone
<i>master_room</i>	(string) ID of the master room

assign_rooms_to_zone(zone ID)

Assigns all rooms to the specified zone.

create_zone(zone group ID, zone name) -> String

Creates a zone with the given name in the provided zone group. Returns the ID of the newly added zone.

create_zone_group(name) -> String

Creates a zone group with the given name. Returns the ID of the newly added zone group.

6.1.18 TariffsEngine

Support for tariffs engine.

Basic usage:

```
import iesve

strProjectFolder = 'C:\\MyProject'
strAPSFile = 'C:\\MyProject\\vista\\a_tariff analysis.aps'
strAPSComparisonFiles = 'C:\\MyProject\\vista\\prb.aps'
strInfoMessage = iesve.TariffsEngine.String()
strError = iesve.TariffsEngine.String()
eUnitsSystem = iesve.TariffsEngine.EUnitsSystem.UNITS_SYSTEM_METRIC (default)
                iesve.TariffsEngine.EUnitsSystem.UNITS_SYSTEM_IP
eMode = iesve.TariffsEngine.EModes.MODE_NORMAL (default)
        iesve.TariffsEngine.EModes.MODE_UPDATE_XML_FILE_TARIFFS_NODE
        iesve.TariffsEngine.EModes.MODE_CHECK_UTILITIES_VARIABLES
eComputeCosts = iesve.TariffsEngine.EComputeCosts.COMPUTE_COSTS_NO
                iesve.TariffsEngine.EComputeCosts.COMPUTE_COSTS_YES

te = iesve.TariffsEngine(strProjectFolder, strAPSFile, strAPSComparisonFiles, strInfoMessage, strError,
eUnitsSystem, eMode, eComputeCosts)

if (strError.Empty() == False):
    print(strError.GetString())
else:
    print("Init ok")

if (strInfoMessage.Empty() == False):
    print(strInfoMessage.GetString())
```

6.1.18.1 Methods Defined Here

Init(pcProjectFolder, pcAPSFile, pcAPSComparisonFiles, oStrInfoMessage, oStrError, eMode, eEnergyDataset, oComputeCosts) -> Bool

Init the tariffs engine.

Parameters:

<i>eEnergyDataset</i>	(int) Energy option (EEnergyDataset). Default is ENERGY_DATASET_GENERIC.
<i>eMode</i>	(int) TariffsEngine engine mode (EModes). Default is MODE_NORMAL.
<i>oComputeCosts</i>	(int) Compute utilities or not (EComputeCosts). Default is COMPUTE_COSTS_YES.
<i>oStrError</i> [OUT]	(object) Error message if any
<i>oStrInfoMessage</i> [OUT]	(object) Non error message to show after initialization. If not nullptr, it has a message and has to be deleted.
<i>pcAPSComparisonFiles</i>	(string) 3 options:

- Files separated by ? that will be stored for comparing. I.e : File1 ? File2 ? File3

- Empty string. Comparison files will be cleared so no files will be used to compare to.
- nullptr. Comparison files won't be modified and pcAPSFile will be compared to them.

<i>pcAPSFile</i>	(string) APS file with path. I.e: C:\\My project\\vista\\test.aps
<i>pcProjectFolder</i>	(string) MIT project folder ending by '\\' or not. I.e: C:\\My project\\

Returns:

Bool - True if success.
False if fail. See oStrError.

Basic usage:

import iesve

```
strProjectFolder = 'C:\\MyProject'
strAPSFile = 'C:\\MyProject\\vista\\a_tariff analysis.aps'
strAPSComparisonFiles = 'C:\\MyProject\\vista\\prb.aps'
strInfoMessage = iesve.TariffsEngine.String()
strError = iesve.TariffsEngine.String()
```

```
te = iesve.TariffsEngine()
```

```
if (not te.Init(strProjectFolder, strAPSFile, strAPSComparisonFiles, strInfoMessage, strError)):
    print(strError.GetString())
else:
    print("Init ok")
```

```
if (strInfoMessage.Empty() == False):
    print(strInfoMessage.GetString())
```

SetBenchmarkProfile(pcFiles)

Sets the file(s) to compare with.

Parameters:

<i>pcFiles</i>	(string) string with the files separated by ? or empty. I.e: "File1?File2?File3"
----------------	--

Returns:

-

Basic usage:

```
te = iesve.TariffsEngine(...)
te.SetBenchmarkProfile("prb.aps?prb2.aps")
```

GetUtilitiesNamesAndIds() -> List

Gets a list of tuples with the utilities names and its ids.

Parameters:

-

Returns:

List - list with the utilities.

I.e: `[['Electricity', 0], ['Gas', 1], ['Oil', 2], ['Coal', 3],
['Miscellaneous', 4], ['Renewables', 5]]`

Basic usage:

```
te = iesve.TariffsEngine(...)
utilities = te.GetUtilitiesNamesAndIds()
print(utilities)
```

GetSelectedCurrency() -> Tuple

Gets a tuple with the selected currency data.

Parameters:

-

Returns:

Tuple - tuple with the selected currency.

I.e: `['Polish zloty', 'PLN', 'Poland', 'PLN']`

Basic usage:

```
te = iesve.TariffsEngine(...)
currency = te.GetSelectedCurrency()
print(currency)
```

GetSuppliersForUtility(iUtilityId) -> List

Gets a tuples list with the suppliers belonging to a utility.

Parameters:

<i>iUtilityId</i>	(int) EUtilities value
-------------------	--

Returns:

List - tuples list with the suppliers belonging to the utility *iUtilityId*. Each tuple has utility name and id, that can be used for other functions.

I.e: `[['Electricity Commercial Supplier', 1],
['Electricity Domestic Supplier', 4]
]`

Basic usage:

```
te = iesve.TariffsEngine(...)
suppliers = te.SuppliersForUtility(te.EUtilities.GAS)
print(suppliers)
```

GetTariffsForSupplier(iSupplierId, strCurrencyId) -> List

Gets a tuple with the tariffs belonging to a supplier and, optionally, a currency.

Parameters:

<i>iSupplierId</i>	(int) Supplier id whose tariffs will be retrieved.
--------------------	--

strCurrencyId (string) Currency id the tariffs will have to have in order to be retrieved. If "", no currency checking will be carried out. Default "".

Returns:

List - tuples list with the tariffs belonging to the supplier whose id is strSupplierId and using the currency if specified.

```
I.e: [['Unrestricted Tariff (£ GBP)', 12],
      ['Two Rate Tariff (£ GBP)', 13],
      ['Economy-7 Tariff (£ GBP)', 21],
      ['Unrestricted Tariff ($ USD)', 23],
      ['Two Rate Tariff ($ USD)', 24],
      ['Economy-7 Tariff ($ USD)', 28]
      ]
```

Basic usage:

```
te = iesve.TariffsEngine(...)
tariffs = te.GetTariffsForSupplier(2)
print(tariffs)
```

GetTariffName(iUtilityId, iTariffId) -> String

Retrieves the name of the tariff whose id is iTariffId and belongs to the utility iUtilityId.

Parameters:

iTariffId	(int) Tariff's id to check
iUtilityId	(int) EUtilities value. Neither miscellaneous nor renewables are possible.

Returns:

String - tariff name or "" if not found.

Basic Usage:

```
te = iesve.TariffsEngine(...)
name = te.GetTariffName(iesve.TariffsEngine.EUtilities.ELECTRICITY, 4)
print(name)
```

GetTariffName(iUtilityId) -> String

Retrieves the name of the tariff whose id is iTariffId and belongs to the utility iUtilityId.

Parameters:

iUtilityId	(int) EUtilities value. Neither miscellaneous nor renewables are possible.
iTariffId	(int) Tariff's id to check

Returns:

String - tariff name or "" if not found.

Basic Usage:

```
te = iesve.TariffsEngine(...)
name = te.GeSelectedtTariffName(iesve.TariffsEngine.EUtilities.ELECTRICITY)
print(name)
```

GetTariffCurrencyUnits(iUtilityId, iTariffId) -> String

Returns a text with the symbol of the selected currency divided by the unit of the tariff whose id is iTariffId. I.e: \$/T

Parameters:

<i>iTariffId</i>	(int) Tariff's id
<i>iUtilityId</i>	(int) EUtilities value. Neither miscellaneous nor renewables are possible.

Returns:

String - A text with the symbol of the selected currency divided by the unit of the tariff whose id is iTariffId. I.e: \$/T

Basic usage:

```
te = iesve.TariffsEngine(...)
currencyUnits = te.GetTariffCurrencyUnits(iesve.TariffsEngine.EUtilities.ELECTRICITY, 4)
print(currencyUnits)
```

GetRatesTimeSeries(iUtility, iFirstDay, iLastDay) -> Numpy Object

Returns a numpy object with the rates applied per time gap from iFirstDay to iLastDay for utility iUtility, that has previously been computed.

Parameters:

<i>iFirstDay</i>	(int) First day whose rates per time gap will be stored: 1,...365, 366... If -1, it will be taken from the APS. Default value = -1 <i>iFirstDay</i> <= <i>iLastDay</i>
<i>iLastDay</i>	(int) Last day whose rates per time gap will be stored: 1,...365, 366... If -1, it will be taken from the APS. Default value = -1 <i>iFirstDay</i> <= <i>iLastDay</i>
<i>iUtility</i>	(int) EUtilities value.

Returns:

Numpy Object - numpy object with the proposed demand cost per time gap.

Basic usage:

```
te = iesve.TariffsEngine(...)
costsPerGap = te.GetRatesTimeSeries(iesve.TariffsEngine.EUtilities.OIL, 1, 365)
print(str(costsPerGap.size) + " values retrieved")

if costsPerGap.size>0:
    print(costsPerGap[0])
```

ComputeWithTariff(iUtilityId, iSupplierId, iTariffId)

Computes the utility iUtilityId using supplier iSupplierId and tariff iTariffId.

Parameters:

<i>iSupplierId</i>	(int) Supplier id to be set. If not found, an exception is thrown.
<i>iTariffId</i>	(int) Tariff id to be set. If not found or doesn't belong to iSupplierId, an exception is thrown.
<i>iUtilityId</i>	(int) EUtilities value.

Returns:

-

Basic usage:

```
te = iesve.TariffsEngine(...)
te.ComputeWithTariff(iesve.TariffsEngine.EUtilities.OIL, 1, 3)
```

ComputeWithFlatRate(iUtilityId, dbFlatRate)

Computes the utility's costs using the flat rate.

Parameters:

<i>dbFlatRate</i>	(float) flat rate to use. If <0, the current utility's flat rate will be used.
<i>iUtilityId</i>	(int) EUtilities value.

Returns:

-

Basic usage:

```
te = iesve.TariffsEngine(...)
te.ComputeWithFlatRate(iesve.TariffsEngine.EUtilities.COAL, .075)
```

GetDesignNetCost(iUtilityId) -> float

Returns the design cost for a utility or total.

Parameters:

<i>iUtilityId</i>	(int) EUtilities value. If -1, the total cost of all of the utilities summed up will be returned.
-------------------	---

Returns:

float - design total cost.

Basic usage:

```
te = iesve.TariffsEngine(...)
print(te.GetDesignNetCost(iesve.TariffsEngine.EUtilities.GAS))
```

GetBenchmarkNetCost(iUtilityId) -> float

Returns the benchmark cost for a utility or total.

Parameters:

<i>iUtilityId</i>	(int) EUtilities value. If -1, the total cost of all of the utilities summed up will be returned.
-------------------	---

Returns:

float - benchmark total cost.

Basic usage:

```
te = iesve.TariffsEngine(...)
print(te.GetBenchmarkNetCost(iesve.TariffsEngine.EUtilities.GAS))
```

Improvement() -> float

Read only property containing the % of improvement.

Parameters:

-

Return Value:

float - % improvement.

Basic usage:

```
te = iesve.TariffsEngine(...)
print(te.Improvement)
```

AcceptChanges() -> String

Accepts changes done. It will save the results, selected tariffs etc.

Parameters:

-

Returns:

String - Error message if any.

Basic usage:

```
te = iesve.TariffsEngine(...)
error = te.AcceptChanges()
```

if error:

```
print(error)
```

GetEndUseCost(filetype, iUtilityId, use) -> String

Returns the cost of an individual end use.

Parameters:

int filetype - 0 if design, 1 if benchmark

int iUtilityId -

iesve.TariffsEngine.EUtilities.ELECTRICITY for electricity.

iesve.TariffsEngine.EUtilities.GAS for gas.

iesve.TariffsEngine.EUtilities.OIL for oil.

iesve.TariffsEngine.EUtilities.COAL for coal.

iesve.TariffsEngine.EUtilities.MISCELLANEOUS for miscellaneous.

iesve.TariffsEngine.EUtilities.RENEWABLES for renewables.

iesve.EnergyUse use - The end use to obtain data for.

Code sample:

```
cost = te.GetEndUseCost(0, iesve.TariffsEngine.EUtilities.ELECTRICITY,
iesve.EnergyUse.prm_interior_lighting)
```

GetEndUseCosts(filetype, iUtilityId, use) -> String

Returns the cost of multiple end uses.

Parameters:

int filetype - 0 if design, 1 if benchmark

int iUtilityId -

iesve.TariffsEngine.EUtilities.ELECTRICITY for electricity.

iesve.TariffsEngine.EUtilities.GAS for gas.

iesve.TariffsEngine.EUtilities.OIL for oil.

iesve.TariffsEngine.EUtilities.COAL for coal.

iesve.TariffsEngine.EUtilities.MISCELLANEOUS for miscellaneous.
 iesve.TariffsEngine.EUtilities.RENEWABLES for renewables.
 list[iesve.EnergyUse] use - The end uses to obtain data for.

Code sample:

```
cost = te.GetEndUseCosts(0, iesve.TariffsEngine.EUtilities.ELECTRICITY,
[iesve.EnergyUse.prm_interior_lighting, iesve.EnergyUse.prm_space_heating])
```

6.1.18.2 Enums Defined Here

<i>EComputeCosts</i>	COMPUTE_COSTS_NO, COMPUTE_COSTS_YES
<i>EEnergyDataset</i>	ENERGY_DATASET_LOAD_STORED, ENERGY_DATASET_ASHRAE, ENERGY_DATASET_GENERIC, ENERGY_DATASET_UK_PART_L2, ENERGY_DATASET_REFERENCE
<i>EModes</i>	MODE_NORMAL, MODE_UPDATE_XML_FILE_TARIFFS_NODE, MODE_CHECK_UTILITIES_VARIABLES (see 'Mode types' below for details)
<i>EUnitsSystem</i>	METRIC, IP
<i>EUtilities</i>	ELECTRICITY, GAS, OIL, COAL, MISCELLANEOUS, RENEWABLES

Mode types:

<i>Check Utilities Variables</i>	Checks whether the utilities exist or not. Call only Init() and DemandVariablesExists()
<i>Normal</i>	Normal usage (computing, getting/setting data etc)
<i>Update XML File Tariffs Node</i>	Mode for updating <Tariffs> node in an XML file

6.1.19 TDVCalculator

Interface for TDV Calculator

6.1.19.1 Methods Defined Here

```
__init__(
    title_24_standard: eASHRAEStandard,
    climate_zone: Title24ClimateZones,
    is_residential: bool,
    application_path: str
)
```

Constructor.

```
__init__(
    apu_file_path: str,
    application_path: str
)
```

Constructor.

get_elec_tdv() -> Numpy array of floats
Gets TDV factors for Electricity in kBtu/kWh

get_elec_tdv() -> Numpy array of floats
Gets TDV factors for Natural Gas in kBtu/kWh

get_elec_tdv() -> Numpy array of floats
Gets TDV factors for Propane in kBtu/kWh

6.1.19.2 Enums Defined Here

<i>eASHRAEStandard</i>	ashrae_standard_not_set, ashrae_2007, ashrae_2004_with_addenda, ashrae_2010, ashrae_2010_ecb, ashrae_2013, ashrae_2013_ecb, iecc_2012, iecc_florida_2014, necb_2011, ashrae_2016, ashrae_2016_appendix_g, title24_2019, necb_2017, ashrae_2019, ashrae_2019_appendix_g, title24_2022
<i>Title24ClimateZones</i>	zone_1, zone_2, zone_3, zone_4, zone_5, zone_6, zone_7, zone_8, zone_9, zone_10, zone_11, zone_12, zone_13, zone_14, zone_15, zone_16

6.1.20 T24

Interface for Title 24

6.1.20.1 Methods Defined Here

get() -> dictionary
Get Title 24 data. Returns a dictionary with the following keys:

```
building_address
building_city
doc_author_company
doc_author_name
doc_author_phone
```

project_name

get_room_ids_excluded_from_verification() -> list

Gets a list of room IDs excluded from verification.

get_solar_assessment_pv_data() -> list[dict]

Return a list of PV data (Meter grouping) from solar assessment. Keys are:

array

panel_area

azimuth

pitch

annual_solar_access

get_unmet_hours(aps_path, network_path) -> dictionary

Gets a dictionary of unmet load hours data for Title 24. Keys are:

cooling

heating

get_pv_designer() -> T24ResponsiblePerson

Gets data on PV designer if present in project data dialog. Checks for scope of PV then PV and Battery.

is_residential_model() -> bool

True if the model is residential, False otherwise

6.1.21 T24EndUseData

Interface for Title 24 End Use Data Aggregator object

6.1.21.1 Methods Defined Here

__init__(

title_24_standard: eASHRAEStandard,

is_residential: bool,

application_path: str

)

Constructor.

get_end_use_data(use: T24ReportEndUse) -> dict

Gets End Use data for proposed and standard for given end use enum

get_efficiency_use_totals() -> dict

Gets efficiency end use data total for proposed and standard

get_non_regulated_use_totals() -> dict

Gets non-regulated end use data total for proposed and standard

get_flexibility_use_totals() -> dict

Gets flexibility end use data total for proposed and standard

get_all_compliance_use_totals() -> dict

Gets all compliance end use data total for proposed and standard

get_all_use_totals() -> dict

Gets all end use data total for proposed and standard

6.1.21.2 Enums Defined Here

<i>T24ReportEndUse</i>	unspecified, space_heating, space_cooling, indoor_fans, heat_rejection, pumps_misc, domestic_hot_water, indoor_lighting, flexibility, receptacle, process, other_lighting, process_motor, pv, battery
------------------------	---

6.1.22 T24ResponsiblePerson

Object containing information about the responsible person.

6.1.22.1 Attributes Defined Here

<i>address</i>	(str) Address of responsible person.
<i>city</i>	(str) City of responsible person
<i>license</i>	(str) License of responsible person
<i>name</i>	(str) Name of responsible person
<i>organization</i>	(str) Organization of responsible person
<i>phone</i>	(str) Phone number of responsible person
<i>state</i>	(str) State of responsible person
<i>zip</i>	(int) Zip of responsible person

6.1.23 TransformerLosses

Interface to access transformer losses data set in PRM 2016 and 2019 navigators.

6.1.23.1 Methods Defined Here

get_efficiency() -> float

Gets user defined transformer efficiency.

get_adjustment_factor() -> float

Gets adjustment factor for electrical sources, based on efficiency

is_set() -> bool

Has transformer losses been set.

6.1.24 UMLH

Interface for unmet load hours.

6.1.24.1 Methods Defined Here

get_unmet_hours(aps_path, network_path, is_title_24) -> dict

Get a dictionary of unmet load hours data. Keys are:

cooling

heating

refresh_cache(aps_path)

Refresh the UMLH cache using data from the APS file.

6.1.25 VAdjacency

Adjacency data. Returned by [VESurface](#) method `get_adjacencies()`.

6.1.25.1 Methods Defined Here

get_properties() -> dictionary (property)

Returns assorted properties of the adjacency

<i>aps_handle</i>	(int, id) the aps handle
<i>body_id</i>	(string) the room ID of the room on the other side of adjacency
<i>distance</i>	(float, meters) the distance between the two rooms.
<i>surface_index</i>	(int) the index of the surface in the adjacent room

The following area related properties are also returned:

<i>door</i>	(float, m ²) total door area in adjacency.
<i>gross</i>	(float, m ²) gross area including all openings
<i>hole</i>	(float, m ²) total hole opening area
<i>window</i>	(float, m ²) total glazed opening area in adjacency

get_construction() -> construction Id

Returns the construction ID (string) used in the adjacency

6.1.26 VESystem

Apache System object.

6.1.26.1 Methods Defined Here

air_supply() -> dictionary

Air supply data for the Apache system. Returns dictionary with the following keys:

<i>condition</i>	(int) supply condition. See air_supply_cond_names()
<i>cooling_max_flow</i>	(float, l/s) maximum cooling air supply flow rate
<i>OA_max_flow</i>	(float, l/s) maximum outside air supply flow rate
<i>profile (optional)</i>	(string) adjacent condition profile ID, only present if condition = 1
<i>temperature_difference</i>	(float, K) air supply temperature difference (0 for no sizing)

apply_ncm()

Applies NCM wizard data to the system. Call this after setting NCM wizard data to update the system itself. If you don't call this, changes will be made only in the wizard and not the system.

auxiliary_energy(show_ncm) -> dictionary

Auxiliary energy data for the Apache system.

Parameters:

<i>show_ncm (optional)</i>	Whether to show NCM specific data in the output
----------------------------	---

Returns:

Dictionary with the following keys:

<i>AEV</i>	(float, W/m ²) energy value. Only if method != 0
<i>air_supply_mechanism (optional)</i>	Air supply mechanism
<i>equivalent_energy</i>	(float, kWh/m ² /y) equivalent energy in, for 3255hrs operation. Only if method != 0
<i>method</i>	(int) auxiliary energy method. See aux_energy_method_names()
<i>off_schedule_AEV</i>	(float, W/m ²) off-schedule heating/cooling AEV
<i>SFP (optional)</i>	(float, W/l/s) specific fan power (SFP). Only if method != 3

bivalent_systems_ncm() -> dictionary

Bivalent systems NCM data for the Apache system. Returns dictionary with the following keys:

<i>load_for_primary_system</i>	% Load for the primary system
<i>overall_seasonal_efficiency</i>	Overall seasonal efficiency
<i>systems</i>	List of dictionaries of system data (one for each system)

System data dictionary contains the following keys:

<i>gen_seff</i>	GEN SEFF (fraction)
<i>heat_source</i>	Heat source
<i>load</i>	% Load

meter

Energy meter

control() -> dictionary

Control data for the Apache system. Returns dictionary with the following keys:

<code>master_zone</code>	(string) master zone room ID
--------------------------	------------------------------

cooling(show_ncm) -> dictionary

Cooling data for the Apache system.

Parameters:

<code>show_ncm (optional)</code>	Whether to show NCM specific data in the output
----------------------------------	---

Returns:

Dictionary with the following keys:

<code>cool_vent_mechanism</code>	(int) Cooling/ventilation mechanism. See cooling_mech_names()
<code>del_eff</code>	(float) Delivery efficiency
<code>free_cooling</code>	(cmm_free_cooling) Changeover mixed mode free cooling
<code>fuel (optional)</code>	(int) Fuel. Only if <code>has_absorption_chiller == False</code> . See fuel_names()
<code>gen_size</code>	(float, kW) Generator size
<code>has_absorption_chiller</code>	(Bool) Whether the system has an absorption chiller
<code>nominal_eer</code>	(float, kW/kW) Nominal EER
<code>pump_and_fan_power_perc</code>	(float) Pump & fan power (% of rejected heat)
<code>SEER</code>	(float, kW/kW) Seasonal energy efficiency ratio
<code>SSEER</code>	(float, kW/kW) Seasonal system energy efficiency ratio

cooling_ncm() -> dictionary

Cooling NCM data for the Apache system. Returns dictionary with the following keys:

<code>chiller_meter</code>	(VEEnergyMeter) Chiller meter
<code>default_generator_nominal_eer</code>	(float) Default nominal EER value
<code>default_generator_seasonal_eer</code>	(float) Default seasonal EER value
<code>generator_nominal_eer</code>	(float) Nominal EER
<code>generator_nominal_eer_known</code>	(bool) Do you know the generator nominal EER?
<code>generator_seasonal_eer</code>	(float) Seasonal EER
<code>generator_seasonal_eer_known</code>	(bool) Do you know the generator seasonal EER?
<code>mixed_mode</code>	(bool) Mixed mode operation strategy (SBEM only)
<code>power</code>	(ncm_chiller_power) Cooling power
<code>qualify_for_eca</code>	(ncm_eca_list) Does it qualify for ECAs?
<code>type</code>	(ncm_chiller_type) Cooling type

general_ncm() -> dictionary

General NCM data for the Apache system. Returns dictionary with the following keys:

<code>is_proxy_for_hvac</code>	(bool) Whether it is a proxy for an ApacheHVAC system
<code>ncm_system_type</code>	(ncm_system_type) NCM system type

heating(show_ncm) -> dictionary
Heating data for the Apache system.

Parameters:

<i>show_ncm</i> (optional)	Whether to show NCM specific data in the output
----------------------------	---

Returns:

Dictionary with the following keys:

<i>CHP_heat_output</i> (optional)	(float, kW) heat output of CHP. Only if CHP is present
<i>CHP_ranking</i> (optional)	(int) ranking of heat source after the CHP plant. Only if CHP is present
<i>del_eff</i>	(float) delivery efficiency of the generator
<i>fuel</i>	(int) generator fuel. See fuel_names()
<i>gen_seasonal_eff</i>	(float) seasonal efficiency of the generator
<i>gen_size</i>	(float, kW) generator size
<i>HR_effectiveness</i>	(float) ventilation heat recovery effectiveness
<i>HR_return_temp</i>	(float, °C) ventilation heat recovery return air temperature
<i>Is_heat_pump</i>	(bool) Is it a heat pump?
<i>meter_cef</i>	(float) Meter CEF kgCO ₂ /kWh
<i>meter_pef</i>	(float) Meter PEF kgCO ₂ /kWh
<i>SCoP</i>	(float, kW/kW) seasonal coefficient of performance
<i>used_with_CHP</i>	(bool) whether the heat source is used in conjunction with CHP

heating_ncm() -> dictionary
Heating NCM data for the Apache system.

Returns:

Dictionary with the following keys:

<i>boiler_over_15_years_old</i>	(bool) Whether the boiler is over 15 years old
<i>convectors_have_fans</i>	(bool) Whether convectors have fans
<i>default_fan_power_ratio</i>	(float) Default fan power ratio
<i>default_generator_radiant_efficiency</i>	(float) Default generator radiant efficiency
<i>default_generator_seasonal_efficiency</i>	(float) Default generator seasonal efficiency
<i>existing_district_heating</i>	(bool) Whether this system is an existing district heating system
<i>fan_power_ratio</i>	(float) Fan power ratio
<i>fan_power_ratio_known</i>	(bool) Whether the fan power ratio is known
<i>generator_radiant_efficiency</i>	(float) Generator radiant efficiency
<i>generator_radiant_efficiency_known</i>	(bool) Whether the generator radiant efficiency is known
<i>generator_seasonal_efficiency</i>	(float) Generator seasonal efficiency
<i>generator_seasonal_efficiency_known</i>	(bool) Do you know the generator seasonal efficiency?
<i>heat_source</i>	(ncm_heat_source) Heat source
<i>installed_after_98</i>	(bool) Whether this system was installed on or after 1998
<i>meter</i>	(VEEnergyMeter) Energy meter
<i>qualify_for_ecas</i>	(ncm_eca_list) Does it qualify for ECAs?
<i>uses_chp</i>	(bool) Whether this system also uses CHP

hot_water(show_ncm) -> dictionary

Hot water data for the Apache system.

Parameters:

<code>show_ncm</code> (optional)	Whether to show NCM specific data in the output
----------------------------------	---

Returns:

Dictionary with the following keys:

<code>circulation_losses</code> (optional)	(float, W/m) circulation losses. Only if secondary circulation is present
<code>cold_water_inlet_temp</code>	(float, °C) mean cold water inlet temperature
<code>default_generator_seasonal_efficiency</code> (optional)	(float) Default generator seasonal efficiency
<code>del_eff</code>	(float) DHW delivery efficiency
<code>existing_district_heating</code> (optional)	(bool) Existing district heating system?
<code>generator_meter</code> (optional)	(VEEnergyMeter) Generator meter
<code>generator_seasonal_efficiency</code> (optional)	(float) Seasonal generator efficiency
<code>generator_type</code> (optional)	(dhw_parent_type) Generator type
<code>has_ApHVAC_boiler</code>	(Bool) whether the system is served by an ApacheHVAC boiler
<code>has_secondary_circulation</code>	(Bool) whether the system has secondary circulation
<code>has_time_switch</code> (optional)	(Bool) whether or not there is a time switch. Only if secondary circulation is present
<code>insulation_thickness</code> (optional)	(float, mm) insulation thickness. Only if storage is used
<code>insulation_type</code> (optional)	(int) type of insulation. See insulation_names() . Only if storage is used
<code>is_storage_system</code>	(Bool) whether the system is a storage system
<code>later_than_1998</code> (optional)	(Bool) Later than 1998?
<code>loop_length</code> (optional)	(float, meters) loop length. Only if secondary circulation is present
<code>pump_power</code> (optional)	(float, kW) pump power. Only if secondary circulation is present
<code>storage_losses</code> (optional)	(float, kWh/l/day) storage losses. Only if storage is used
<code>storage_volume</code> (optional)	(float, liters) storage volume. Only if storage is used
<code>supply_temp</code>	(float, °C) hot water supply temperature
<code>use_default_generator_seasonal_efficiency</code> (optional)	(bool) Do you know the generator seasonal efficiency?
<code>pump_meter</code>	(VEEnergyMeter) Pump meter

metering_provision_ncm() -> dictionary

Metering provision NCM data for the Apache system.

Returns:

Dictionary with the following keys:

<code>control_correction</code>	(float) Control correction
<code>provision_for_metering</code>	(bool) Does the system have provisioning for metering?
<code>warns</code>	(bool) Does the metering warn 'out of range' values?

set_air_supply(air_supply_data)

Set air supply data from the supplied dictionary.

Dictionary keys are:

condition
cooling_max_flow
OA_max_flow
profile
temperature_difference

set_auxiliary_energy(aux_data)

Set auxiliary energy data from the supplied dictionary.

Dictionary keys are:

AEV
air_supply_mechanism
fan_fraction
method
off_schedule_AEV
SFP

set_bivalent_systems_ncm(data)

Set bivalent systems from the supplied list of dictionaries.

Dictionary keys are:

heat_source
meter
gen_seff
load

set_control(control_id)

Set the master zone control.

set_cooling(cooling_data)

Set cooling data from the supplied dictionary.

Dictionary keys are:

cool_vent_mechanism
del_eff
free_cooling
fuel
gen_size
has_absorption_chiller
nominal_eer
pump_and_fan_power_perc
SEER
SSEER

set_cooling_ncm(data)

Set cooling NCM data from the supplied dictionary.
Dictionary keys are:

chiller_meter
generator_nominal_eer
generator_nominal_eer_known
generator_seasonal_eer
generator_seasonal_eer_known
mixed_mode
power
qualify_for_eca
type

set_default(system_id)
Set the default system ID.

set_general_ncm(data)
Set general NCM data from the supplied dictionary.
Dictionary keys are:

is_proxy_for_hvac
ncm_system_type

set_heating(heating_data)
Set heating data from the supplied dictionary.
Dictionary keys are:

CHP_heat_output
CHP_ranking
fuel
gen_seasonal_eff
gen_size
HR_effectiveness
HR_return_temp
is_heat_pump
meter_cef
meter_pef
SCoP
used_with_CHP

set_heating_ncm(data)
Set heating NCM data from the supplied dictionary.
Dictionary keys are:

boiler_over_15_years_old
convectors_have_fans
default_fan_power_ratio
default_generator_radiant_efficiency
default_generator_seasonal_efficiency

fan_power_ratio
 fan_power_ratio_known
 generator_radiant_efficiency
 generator_radiant_efficiency_known
 generator_seasonal_efficiency
 generator_seasonal_efficiency_known
 heat_source
 installed_after_98
 is_heat_pump
 meter
 meter_cef
 meter_pef
 qualify_for_ecas
 uses_chp

set_hot_water(hot_water_data)

Set hot water data from the supplied dictionary.

Dictionary keys are:

circulation_losses
 cold_water_inlet_temp
 del_eff
 existing_district_heating
 generator_meter
 generator_seasonal_efficiency
 generator_type
 has_ApHVAC_boiler
 has_secondary_circulation
 has_time_switch
 insulation_thickness
 insulation_type
 is_storage_system
 later_than_1998
 loop_length
 pump_power
 storage_losses
 storage_volume
 supply_temp
 use_default_generator_seasonal_efficiency

set_metering_provision_ncm(data)

Set metering provision NCM data from the supplied dictionary.

Dictionary keys are:

provision_for_metering
 warns

set_name(name)

Set the system name.

set_solar_water_heating(sol_water_data)

Set solar hot water heating data from the supplied dictionary.

Dictionary keys are:

abso_absorptance
abso_radius
area
azimuth
coeff_a1
coeff_a2
coeff_c1
coeff_c2
conversion_eff
cover_trans
degradation_factor
flow_rate
fluid_flow
has_space_or_process_heating
heat_capacity
HX_effectiveness
intercept_factor
mirror_reflectance
num_rows
pump_fuel
pump_power
refl_focal_length
refl_length
refl_width
return_temp
shading_factor
storage_loss
supply_temp
tank_heat_loss
tank_volume
tilt
tube_extension
type
units_per_row
volume

set_solar_water_heating_ncm(data)

Set solar hot water heating NCM data from the supplied dictionary.

Dictionary keys are:

a1
a2
circulation_system
heat_transfer_rate

iam
 insulation_thickness
 insulation_type
 is_heat_exchanger
 is_solar_heating
 nominal_pump_power
 overall_heat_loss_coeff
 panel_area
 panel_azimuth
 panel_tilt
 performance_parameters
 performance_parameters_known
 pipes_insulated
 preheating_type
 sigma_o
 volume

set_system_adjustment_ncm(data)

Set system adjustment NCM data from the supplied dictionary.

Dictionary keys are:

ahu_meets_standards
 cen_class
 ductwork_leakage_test
 ductwork_leakage_test_done
 pump_type
 specific_fan_power
 specific_fan_power_known

set_system_controls_ncm(data)

Set system control NCM data from the supplied dictionary.

Dictionary keys are:

central_time_control
 local_temperature_control
 local_time_control
 optimum_start_stop_control
 weather_compensation_control

set_ventilation_ncm(data)

Set ventilation NCM data from the supplied dictionary.

Dictionary keys are:

air_supply_mechanism
 heat_recovery_type
 heat_recovery_efficiency_known
 heat_recovery_efficiency
 variable_heat_recovery

solar_water_heating() -> dictionary

Solar water heating data for the Apache system. Returns dictionary with the following keys:

type	Collector type
------	----------------

Flat collectors also have the following keys:

area	(float, m ²) Area
azimuth	(float, Degrees clockwise from North) azimuth
coeff_a1	(float, W/m ² /K) First order heat loss coefficient (a1)
coeff_a2	(float, W/m ² /K) Second order heat loss coefficient (a2)
conversion_eff	(float) Conversion efficiency at ambient temperature
degradation_factor	(float) Degradation factor
flow_rate	(float, l/h/m ²) Flow rate
HX_effectiveness	(float) Heat exchanger effectiveness
pump_fuel	(fuel_type) Pump fuel
pump_meter_branch	(tuple) Pump meter branch
pump_power	(float, kW) Pump power
shading_factor	(float) Shading factor
storage_loss	(float, kWh/l/day) Storage loss at max. temperature
tilt	(float, degrees from horizontal) Tilt
volume	(float, liters) Volume

Parabolic collectors also have the following keys:

abso_absorptance	(float) Absorptance of absorber tube
abso_radius	(float, m) Radius of absorber tube
azimuth	(float, degrees from North) Angle of collector tube
coeff_c1	(float) First order loss coefficient per unit aperture area
coeff_c2	(float) Second order loss coefficient per unit aperture area
cover_trans	(float) Transmittance of cover tube
fluid_flow	(float, l/h/m ²) Total fluid flow
heat_capacity	(float, J/kg/K) Fluid specific heat capacity
HX_effectiveness	(float) Heat exchanger effectiveness
intercept_factor	(float) Fraction of beam radiation intercepted by receiver
mirror_reflectance	(float) Reflectance of the mirror
num_rows	(int) Number of rows (in parallel)
pump_fuel	(fuel_type) Pump fuel
pump_meter_branch	(tuple) Pump meter branch
pump_power	(float, kW) Pump power
refl_focal_length	(float, m) Focal length of one reflector collector unit
refl_length	(float, m) Length of one reflector collector unit
refl_width	(float, m) Width of one reflector collector unit
tank_heat_loss	(float, kWh/l/day) Design tank heat loss
tank_volume	(float, l) Tank volume
tube_extension	(float, m) Extension of tube beyond a single unit

units_per_row

(int) Collector units per row (in series)

Flat and parabolic collectors also have the following keys:

has_space_or_process_heating	Whether there is space or process heating
return_temp (optional)	Water loop design return temperature (°C)
supply_temp (optional)	Water loop design supply temperature (°C)

solar_water_heating_derived() -> dictionary

Derived solar water heating data for the Apache system. Returns:

Empty dictionary for flat/none, or dictionary with the following keys for parabolic:

aperture_area	Total aperture area (m ²)
refl_rim_angle	Reflector rim angle (degrees)
conc_ratio	Ratio of aperture area to projected absorber tube area
ref_collector_eff	Reference optical efficiency of collector
fluid_flow_rate	Total fluid flow rate (l/h)
coeff_k1	First order loss coefficient per unit tube length
coeff_k2	Second order loss coefficient per unit tube length

solar_water_heating_ncm() -> dictionary

Solar water heating data for NCM mode. Returns dictionary with the following keys:

a1	a1 W/m ² K
a2	a2 W/m ²
circulation_system	Circulation system
heat_transfer_rate	Heat transfer rate (W/K)
iam	IAM
insulation_thickness	Insulation thickness (mm)
insulation_type	Insulation type
is_heat_exchanger	Is there a heat exchanger?
is_solar_heating	Is there a solar heating system?
nominal_pump_power	Nominal pump power, Paux (W)
overall_heat_loss_coeff	Overall heat loss coeff (W/K)
panel_area	Solar panel area (m ²)
panel_azimuth	Solar panel azimuth (degrees clockwise from north)
panel_tilt	Solar panel tilt (degrees from horizontal)
performance_parameter	Performance parameter
performance_parameters_known	Do you know the performance parameters?
pipes_insulated	Are pipes to back-up system insulated?
preheating_type	Pre-heating type
sigma_o	Sigma-o
volume	Solar storage volume (l)

system_adjustment_ncm() -> dictionary

System adjustment NCM data for the Apache system. Returns dictionary with the following keys:

ahu_meets_standards	(bool) Does the AHU meet CEN leakage standards?
air_leakage	(float) Air leakage

<code>cen_class</code>	(ncm_leakage_standard) CEN classification met by the AHU
<code>default_specific_fan_power</code>	(float) Default specific fan power
<code>ductwork_leakage_test</code>	(ncm_leakage_test) CEN classification met by the ductwork
<code>ductwork_leakage_test_done</code>	(bool) Has the ductwork been leakage tested?
<code>pump_type</code>	(ncm_pump_type) Pump type
<code>specific_fan_power</code>	(float) Specific Fan Power of the system
<code>specific_fan_power_known</code>	(bool) Do you know the Specific Fan Power?

system_controls_ncm() -> dictionary

System controls NCM data for the Apache system. Returns dictionary with the following keys:

<code>central_time_control</code>	(bool) Central time control
<code>local_temperature_control</code>	(bool) Local temperature control (i.e., room by room)
<code>local_time_control</code>	(bool) Local time control (i.e., room by room)
<code>optimum_start_stop_control</code>	(bool) Optimum start/stop control
<code>weather_compensation_control</code>	(bool) Weather compensation control

ventilation_ncm() -> dictionary

Ventilation NCM data for the Apache system. Returns dictionary with the following keys:

<code>air_supply_mechanism</code>	(ncm_air_supply_mechanism) Air supply mechanism
<code>cool_vent_mechanism</code>	(CoolingMechanism_type) Cooling/ventilation mechanism
<code>default_heat_recovery_efficiency</code>	(float) Default heat recovery seasonal efficiency
<code>heat_recovery_efficiency</code>	(float) Heat recovery seasonal efficiency
<code>heat_recovery_efficiency_known</code>	(bool) Do you know the heat recovery seasonal efficiency?
<code>heat_recovery_type</code>	(ncm_heat_recovery_type) Heat recovery type
<code>variable_heat_recovery</code>	(bool) Variable heat recovery efficiency?

6.1.26.2 Static Methods Defined Here

default() -> str

ID of the default Apache system.

fuel_name(fuel_id) -> str

Gets the fuel name for the input fuel ID.

Args:

<code>fuel_id</code>	(int) The ID of the fuel
----------------------	--------------------------

Returns:

Name of fuel

Raises:

<code>ValueError</code>	If no name is found for the input fuel ID
<code>RuntimeError</code>	In the case of an internal data handling error

meter_name(fuel_id, meter_branch) -> str

Gets the meter name for the input meter ID.

Args:

<code>fuel_id</code>	(int) The ID of the fuel that the meter belongs to
<code>meter_branch</code>	(tuple[int]) The branch of the meter

Returns:

Name of meter

Raises:

<code>ValueError</code>	If no name is found for the meter ID
<code>RuntimeError</code>	In the case of an internal data handling error

6.1.26.3 Attributes Defined Here

<code>id</code>	(string) Apache system ID
<code>name</code>	(string) Apache system's name

6.1.26.4 Enums Defined Here

<code>AuxEnergyMethod_type</code>	SFPs, SFPs_with_min_AEV, SFPs_and_AEV, AEV
<code>cmm_free_cooling</code>	not_a_cmm_system, natural_ventilation, mechanical_ventilation
<code>Conditioning_type</code>	external_air, temperature_from_schedule
<code>CoolingMechanism_type</code>	air_conditioning, mechanical_ventilation, natural_ventilation
<code>dhw_parent_type</code>	unset, dedicated_dhw_boiler, standalone_water_heater, instantaneous_dhw_only, instantaneous_combi, heat_pump
<code>fuel_type</code>	anthracite, biogas, biomass, coal, district_heating, dual, elec, grid_displaced_elec, lpg, misc_a, misc_b, misc_c, misc_d, misc_e, misc_f, misc_g, misc_h, misc_i, misc_j, misc_k, misc_l, misc_m, misc_n, misc_o, misc_p, misc_q, misc_r, misc_s, misc_t, misc_u, misc_v, misc_w, misc_x, misc_y, misc_z, nat_gas, none, oil, smokeless, waste, grid_displaced_pv
<code>Insulation_type</code>	Uninsulated, loose_jacket, factory_insulated
<code>ncm_air_supply_mechanism</code>	centralised_ac, zonal_supply_system_with_remote_fan, zonal_extract_system_with_remote_fan, local_ventilation_serving_single_area, other_local_ventilation_units
<code>ncm_chiller_power</code>	less_than_100_kw, less_than_500_kw, less_than_750_kw, greater_than_750_kw
<code>ncm_chiller_type</code>	air_cooled, water_cooled, remote_condenser, heat_pump_gas_oil, heat_pump_electric
<code>ncm_eca_list</code>	not_on_list, after_2001, before_2005, from_2005, uplift_in_2006
<code>ncm_heat_recovery</code>	no_heat_recovery, plate_heat_exchanger, heat_pipes, thermal_wheel, run_around_coil
<code>ncm_heat_source</code>	lthw_boiler, mthw_boiler, hthw_boiler, air_heater, unflued_gas_warm_air_heater, unitary_radiant_heater, radiant_heater, unflued_radiant_heater, direct_or_storage_electric_heater, heat_pump_air_source, heat_pump_ground_or_water_source, room_heater, district_heating, direct_gas_firing, chp, heat_pump_gas_oil_air_source, heat_pump_gas_oil_ground_or_water_source, heat_pump_electric_air_source, heat_pump_electric_ground_or_water_source
<code>ncm_leakage_standard</code>	class_l1, class_l2, class_l3, not_compliant
<code>ncm_leakage_test</code>	not_tested, class_a, class_b, class_c, class_d
<code>ncm_pump_type</code>	constant_speed, variable_speed_differential_sensor_pump, variable_speed_differential_sensor_system, variable_speed_multiple_pressure_sensors, none
<code>ncm_system_type</code>	not_set, generic_heating_only_electrical_resistance, generic_heating_only_other_systems, generic_heating_and_mechanical_cooling, central_heating_using_water_radiators, central_heating_using_water_convectors, central_heating_using_water_floor_heating, central_heating_using_air_distribution, other_local_room_heater_fanned, other_local_room_heater_unfanned, unflued_radiant_heater,

	flued_radiant_heater, multi_burner_radiant_heaters, flued_forced_convection_air_heaters, unflued_forced_convection_air_heaters, single_duct_vav, dual_duct_vav, indoor_packaged_cabinet, fan_coil_systems, induction_system, constant_volume_system_fixed, constant_volume_system_variable, multizone, terminal_reheat, dual_duct, chilled_ceilings_or_passive_chilled_beams_displacement_ventilation, active_chilled_beams, water_loop_heat_pump, split_or_multisplit_systems, single_room_cooling_system, chilled_ceilings_or_passive_chilled_beams_mixed_ventilation, variant_refrigerant_flow, no_heating_or_cooling
<i>shw_circulation_sys</i>	thermosiphon_system, forced_circulation_system_with_pv, forced_circulation_system_with_no_pv
<i>shw_collector_type</i>	unglazed, flat_panel, evacuated
<i>shw_insulation_type</i>	uninsulated, loose_jacket, factory_insulated
<i>shw_preheating_type</i>	separate_solar_cylinder, combined_cylinder
<i>SolarHeating_type</i>	none, flat, parabolic

6.1.27 VEBody

Interface for a VE Body object.

Basic usage:

```
project = iesve.VEProject.get_current_project()
realmodel = project.models[0]
bodies = realmodel.get_bodies(False)
for body in bodies:
    area = body.get_areas()
    windowArea = area['ext_wall_glazed'] + area['int_wall_glazed']
    print('Body: {}. Glazing area: {}'.format(body.id, windowArea))
```

6.1.27.1 Notes on VE Body Objects

A model can consist of bodies of various types. The VEBody object has a type attribute to discover the object type of that particular space. Object types can be categorised as follows:

- 3D bodies: rooms, local or topographical shades, adjacent buildings
- 2D bodies: masterplanning surfaces, boundaries
- 1D bodies: annotations

Each body has a unique ID. The ID is unique only within the model that contains the room. In a variant model, the same room will have the same ID - this allows you to find the same room across models and compare them.

6.1.27.2 Methods Defined Here

assign_construction(construction, surface)

Assign construction to surface. If the model has inner volumes and newly-assigned constructions have differing thicknesses to the previous ones, `rebuild_adjacencies` must be called on the VEModel.

assign_construction_to_opening(construction, surface, opening_id)

Assign construction to an opening with the specified id.

assign_opening_type_by_id(surface_index, macroflo_id, opening_id)

Assign macroflo opening type to opening.

get_areas() -> {areas}

Returns a dict of areas (and volume) for this space.

The following areas are returned (All areas are returned in m² units. Room volume is returned in m³):

int_floor_area	ext_floor_area	int_floor_opening	ext_floor_opening
int_floor_glazed	ext_floor_glazed		
int_ceiling_area	ext_ceiling_area	int_ceiling_opening	ext_ceiling_opening
int_ceiling_glazed	ext_ceiling_glazed	int_ceiling_door	ext_ceiling_door
int_wall_area	ext_wall_area	int_wall_opening	ext_wall_opening
int_wall_glazed	ext_wall_glazed	int_wall_door	ext_wall_door
volume			

Note: space total floor / ceiling etc. area comprises both internal and external areas.

get_assigned_constructions() -> [(construction ID, "")]

Returns a list of tuples containing construction IDs assigned to this space. The second element in the tuple is currently unused.

get_assigned_profiles() -> [(profile ID, "")]

Returns a list of tuples containing profile IDs in use by this space. The second element in the tuple is currently unused.

get_index()

Get the index of the body

get_processes() -> list[VEComponentProcess]

Returns a list of VEComponentProcess objects representing the component processes in the body.

get_properties() -> dict

Returns a dictionary with the following properties:

min_height

max_height

floor_height_above_ground

get_surfaces() -> [surfaces]

Returns a list of [VESurface](#) objects that bound this space.

get_room_data(type = *attribute_type.active_attributes*) -> [VERoomData](#) object

Args:

type (*attribute_type*)

Returns a VERoomData object for closer inspection of data assigned to this room. The returned data depends on the input type: the default type of 'real_attributes' gives the default data for the room, but NCM, PRM and Title 24 specific data can be requested (but only for their respective building variants).

is_3d_shade() -> bool

Checks whether the body is a 3d shade. Returns True for a topographical or local shade but False for a vegetated shade.

select()

Selects this body. Perform operations on the selected body using the [VEGeometry](#) API.

6.1.27.3 Attributes Defined Here

<i>bim_id</i>	(string) Body BIM ID
<i>cad_object_id</i>	(string) Body CAD Object ID
<i>id</i>	(string) Body ID
<i>hvac_methodology</i>	(VEBody_hvac_methodology enum) Body HVAC methodology
<i>name</i>	(string, get & set) Body name
<i>selected</i>	(Bool) True if body is selected, else False
<i>subtype</i>	(VEBody_subtype enum) Body subtype
<i>type</i>	(VEBody_type enum) Body type

6.1.27.4 Enums Defined Here

<i>hvac_methodology</i>	apache_system, apache_hvac
<i>VEBody_subtype</i>	Boundaries: boundary_site, boundary_leed, boundary_stormwater, boundary_stormwater_collect_drainage, boundary_building_footprint,

Roads: road_motorway, road_primary, road_secondary, road_tertiary, road_unclassified

Carparks: parking_bay_indoor, parking_bay_outdoor

Spaces: room, void, ra_plenum, sa_plenum

VEBody_type

3D body types: room, adjacent_building, topographical_shade, local_shade, tree

2D body types: road, pavement, parking_bay, hard_landscape, pervious_landscape, soft_landscape_turf, soft_landscape_shrubs, soft_landscape_groundcover, soft_landscape_mixedveg, soft_landscape_wetlands, vegetated_shade, water, boundary

1D body type: annotation

attribute_type

real_attributes, ncm_attributes, bprm_attributes, t24_attributes, green_star_attributes, new_zealand_attributes, green_mark_attributes, necb_attributes, active_attributes

6.1.28 VECdbConstruction

Basic usage:

See cdb.py

6.1.28.1 Methods Defined Here

add_layer(material_id, is_cavity)

Add a layer to this construction

delete_layer(layer_id)

Delete a layer from this construction

get_default_resistances()

Get default surface resistances for different methodologies

get_g_values()

Get the different g-values as a dictionary.

Dictionary entries are:

<i>bs_en_410</i>	(float)
<i>building_regulations</i>	(float)
<i>bfrc</i>	(float)

get_layers()

Get a list of layers in this CDB construction

get_properties(u_value_types (optional))

Get a dict of the properties of this CDB construction. The result depends on the optional U value type parameter of type [uvalue_types](#). If no parameter is specified the U value type set in the Project Constructions dialog is used.

get_review_summary_string(...)

Get a summary of this CDB construction as a formatted string. Units are converted to local display preferences.

get_u_factor(uvalue_types)

Get the U Factor of this CDB construction.

The result depends on the U value type parameter of type [uvalue_types](#).

insert_layer(layer_id, insert_before)

Insert a layer into this construction

set_const_class(construction_class)

Set construction class of this CDB construction. See 'construction_class' enum for possible values.

set_f_factor(ffactor)

Set F Factor of this CDB construction

set_properties(properties)

Set properties of this CDB construction via a dictionary.

Possible dictionary entries are:

bfrc_active

display_window
 external_shade_active
 external_shade_code
 external_shade_day_resistance
 external_shade_ground_transmittance
 external_shade_ground_transmittance_default
 external_shade_night_resistance
 external_shade_profile
 external_shade_radiation_to_lower
 external_shade_radiation_to_raise
 external_shade_sky_transmittance
 external_shade_sky_transmittance_default
 external_shade_transmittance_15
 external_shade_transmittance_75
 external_shade_transmittance_0
 external_shade_transmittance_30
 external_shade_transmittance_45
 external_shade_transmittance_60
 external_shade_transmittance_90
 frame_absorptance
 frame_inside_surface_area_ratio
 frame_material
 frame_outside_surface_area_ratio
 frame_percent
 frame_resistance
 frame_type
 g_value
 glazing_type
 ground_contact
 inside_surface_emissivity
 inside_surface_resistance
 inside_surface_solar_absorptivity
 internal_shade_active
 internal_shade_blind_or_curtain
 internal_shade_code
 internal_shade_day_resistance
 internal_shade_frac_daylight_closed
 internal_shade_night_resistance
 internal_shade_profile
 internal_shade_radiation_to_lower
 internal_shade_radiation_to_raise
 internal_shade_shading_coefficient
 internal_shade_short_wave_radiant_fraction
 light_transmittance
 local_shade_active
 local_shade_balcony_depth_h
 local_shade_balcony_height
 local_shade_balcony_projection
 local_shade_code
 local_shade_left_fin_offset
 local_shade_left_fin_projection
 local_shade_overhang_type
 local_shade_projection_offset
 local_shade_projection_overhang
 local_shade_right_fin_offset
 local_shade_right_fin_projection
 local_shade_window_height

local_shade_window_width
outside_surface_emissivity
outside_surface_resistance
outside_surface_solar_absorptivity
percent_sky_blocked
spacer_type
thermal_bridging_coefficient
visible_light_transmittance

6.1.28.2 Attributes Defined Here

category

CDB construction category

c_factor

The C Factor of this CDB construction
 Only relevant if the regulation is 3 (ASHRAE901)

is_editable

Is the CDB construction editable?

f_factor

The F Factor of this CDB construction

id

The id of the CDB construction

opaque

Is the CDB construction opaque?

reference

The reference (description) of the CDB construction

regulation

The regulation applied to the CDB construction:

0	CIBSE
1	EN_ISO
2	ASHRAE 901
3	ASHRAE901,
4	Title 24

6.1.29 VECdbDatabase

Basic usage:

See cdb.py

6.1.29.1 Methods Defined Here

get_projects(...)

Get a dict of all the projects in the database

For each project type (0=Project, 1=System, 2=Manufacturer) the dict holds a list of the projects of that type.

6.1.29.2 Static Methods Defined Here

get_current_database(...)

Get the current CDB database

NB this method is static

get_project_type_string(...)

Get a string describing the project type

6.1.30 VECdbLayer

Basic usage:

see cdb.py

6.1.30.1 Methods Defined Here

get_id(...)

Get the layer ID

get_material(is_opaque)

Get the material of this CDB layer as a formatted string. Units are converted to local display preferences.
Pass in true for an opaque layer, false for a glass layer.

get_review_summary_string(is_opaque, u_value_type (optional))

Get a summary of this CDB layer as a formatted string. Units are converted to local display preferences.
The result depends on the optional U value type parameter of type [uvalue_types](#).
If no parameter is specified the U value type set in the Project Constructions dialog is used.

get_properties(u_value_type (optional))

Get a dictionary of CDB layer data.
The result depends on the optional U value type parameter of type [uvalue_types](#).
If no parameter is specified the U value type set in the Project Constructions dialog is used.

Dictionary entries are:

convection_coefficient (W/m²K)
resistance (m²K/W)
thickness (m)

set_properties()

Set properties of this CDB layer via a dictionary.
Possible dictionary entries are:

convection_coefficient (W/m²K)
resistance (m²K/W)
thickness (m)

6.1.31 VECdbMaterial

Basic usage:

See cdb.py

6.1.31.1 Methods Defined Here

get_review_summary_string(...)

Get a summary of this CDB material as a formatted string. Units are converted to local display preferences.

get_properties(...)

Get a dictionary of CDB material data

set_properties()

Set properties of this CDB material via a dictionary.

Possible dictionary entries are:

angular_dependence

conductivity

density

description

inside_emissivity

inside_reflectance

inside_visible_reflectance

outside_emissivity

outside_reflectance

outside_visible_reflectance

refractive_index

specific_heat_capacity

surface_type

thickness

transmittance

vapour_resistivity

visible_transmittance

6.1.32 VECdbProject

Basic usage:

See cdb.py

6.1.32.1 Methods Defined Here

create_construction(construction_element_category)

Create a construction of the specified category. Use the ['element_categories'](#) enum.

get_construction(...)

Get the construction with the given id in the given class in the project

Pass class = 'iesve.construction_class.none' to return the construction with the given id in any class in the project

get_construction_ids(...)

Get a list of the construction ids in the given class in the project

Pass class = 'iesve.construction_class.none' to return a list of the construction ids in all classes in the project

get_number_of_constructions_in_category(...)

Get the number of constructions in the given category in the project

create_material(material_category)

Create a material of the specified category. Returns an instance of the newly created material. Use the ['material_categories'](#) enum.

get_material_ids(material_category)

Get all material IDs matching the specified category

Passing material_categories.all will return all material IDs in the project

get_number_of_materials_in_category(material_category)

Get the number of materials matching the specified category

Passing material_categories.all will return the number of materials in the project

get_material(material_id)

Get an instance of the material matching the specified ID.

delete_material(material_id)

Delete the material matching the specified ID from the project materials.

6.1.32.2 Static Methods Defined Here

get_category_string(...)

Get a string describing the construction category

NB this method is static

get_class_string(...)

Get a string describing the construction class

NB this method is static

6.1.32.3 Attributes Defined Here

has_constructions

Whether the CDB project has constructions

has_materials

Whether the CDB project has materials

name

The DFA file name of the CDB project

number_of_constructions

The number of constructions in the project

number_of_materials

The number of materials in the project

path

The path of the CDB project

title

The title of the CDB project

6.1.32.4 Enums Defined Here

<i>angular_dependence_type</i>	none, fresnel, explicit, constant, lbnl
<i>construction_class</i>	none, opaque, glazed, hard_landscaping, soft_landscaping, shade, misc
<i>element_categories</i>	roof, ceiling, wall, partition, ground_floor, roof_light, ext_glazing, int_glazing, door, int_floor, hard_road, hard_parking, hard_landscape, hard_pavement, hard_pervious, soft_turf, soft_mixed_veg, soft_tree, soft_veg_shade, soft_shrub, soft_ground, soft_wetland, soft_water, local_shade, topographical_shade, foundation, struct_frame, gutter, shape_beam, solid_beam, shape_column, solid_column, shape_lintel, solid_lintel, shape_stair, solid_stair, double_facade
<i>frame_materials</i>	none, softwood, hardwood, steel, aluminium, pvc, metal, metal_with_thermal_break
<i>frame_type</i>	no_frame, a901_wall_mass, a901_wall_metal_building, a901_wall_steel_framed, a901_wall_wood_framed, a901_roof_metal_building, a901_roof_attic_roof_with_joist, a901_roof_attic_roof_with_steel_joist, a901_roof_insulation_above_deck, a901_floor_mass, a901_floor_steel_joist, a901_floor_wood_framed_and_others, t24_wall_metal_frame, t24_wall_metal_building, t24_wall_medium_mass, t24_wall_heavy_mass, t24_wall_wood_framed_and_others, t24_roof_wood_framed_attic, t24_roof_wood_framed_rafter, t24_roof_structurally_insulated_panel, t24_roof_metal_framed_attic, t24_roof_metal_framed_rafter,

	t24_roof_metal_builing, t24_floor_heavy_mass, t24_floor_wood_frame, t24_floor_wood_foam_panel, t24_floor_metal_frame
<i>glazing_types</i>	none, single_glazed, double_glazed_air_or_argon_filled, double_glazed_low_e_hard_coat, double_glazed_low_e_soft_coat, triple_glazed_air_or_argon_filled, triple_glazed_low_e_hard_coat, triple_glazed_low_e_soft_coat
<i>material_categories</i>	all, asphalts, boards, bricks, carpets, concretes, gravels, insulating, metals, plaster, screeds, sands, titles, timber, index_glass, other, floor_finish, susp_ceiling, composite_layer, glass
<i>percent_sky_blocked_options</i>	more_than_80_percent, 60_to_80_percent, 20_to_60_percent, less_than_20_percent
<i>project_types</i>	project, system, manufacturer
<i>standard</i>	generic, cibse, ashrae_general, ashrae_901, t24, ncm, sbem, nz, gm, bre, monodraught
<i>surface_type</i>	metal, plastic, glass, dielectric, translucent
<i>uvalue_types</i>	cibse, iso, ashae, t24

6.1.33 VEComponentProcess

Provides access to the object process data for a component process.

6.1.33.1 Methods Defined Here

get_name() -> string

Gets the process' name.

Returns:

Process' name

get_system_inputs() -> List of dictionaries

Gets the data for the process' system inputs.

Returns:

List of dictionaries, each containing the data for a system input

get_waste_heats() -> List of dictionaries

Gets the data for the process' waste heats.

Returns:

List of dictionaries, each containing the data for a waste heat

get_material_inputs() -> List of dictionaries

Gets the data for the process' material inputs.

Returns:

List of dictionaries, each containing the data for a material input

get_material_outputs() -> List of dictionaries

Gets the data for the process' material outputs.

Returns:

List of dictionaries, each containing the data for a material output

get_energy_inputs() -> List of dictionaries

Gets the data for the process' energy inputs.

Returns:

List of dictionaries, each containing the data for an energy input

get_heat_outputs() -> List of dictionaries

Gets the data for the process' heat outputs.

Returns:

List of dictionaries, each containing the data for a heat output

get_miscellaneous() -> List of dictionaries

Gets the data for the process' miscellaneous information.

Returns:

List of dictionaries, each containing the data for a miscellaneous entry

get_product() -> string

Gets the name of the product that the process produces.

Returns:

Product name as a string

6.1.34 VEEnergyMeter

Interface for energy meters. Can be obtained from [EnergySources](#).

6.1.34.1 Methods Defined Here

get() -> dictionary

Get energy meter data.

Dictionary entries are name, source_data and id.

6.1.35 VEGeometry

Interface for geometry.

6.1.35.1 Methods Defined Here

get_construction() -> string

Gets the construction ID of the opening. Returns an empty string if none are found.

get_id() -> ID (string)

Gets the opening ID

get_macroflo_id() -> ID (string)

Gets the macroflo opening ID

get_properties() -> dictionary

Gets a dictionary of opening properties. Possible keys are:

aps_handle

area

aspect_ratio

height

index

macroflo_type

perimeter

sill_height

type

width

Note that values for width and height are only accurate for rectangular openings and horizontal rotations.

move_opening(xdistance, ydistance)

Move an associated opening

6.1.35.2 Static Methods Defined Here

centre_to_origin()

Centres the model on the grid origin (0,0).

get_building_orientation()

Get building orientation

reduce_ext_windows(max_wwr)

Reduce external windows so that the max window-to-wall ratio is not exceeded

Arguments:

max_wwr

(float) Max window-to-wall ratio

get_wwr() -> Window-to-wall ratio (float)

Get the model's window-to-wall ratio

remove_doors(flag, opening_area)

Remove doors from the model.

Arguments:

flag (int): 0 = all, 1 = internal, 2 = external

opening_area (float): Opening area threshold, all doors with an area below this threshold are removed.

If 0.0, removes all doors

remove_holes(flag, opening_area)

Removes holes from the model.

Arguments:

flag (int) 0 = all, 1 = internal, 2 = external

opening_area (float) Opening area threshold, all holes with an area below this threshold are removed. If 0.0, removes all holes

remove_openings_below_area_threshold(area_threshold)

Remove openings below the provided threshold area for the selected body. Use [VEBody.select\(\)](#) to select a body.

Arguments:

area_threshold (float) Area threshold in square metres

set_body_opening_type(opening_type)

Change opening type for selected body's openings. Use [VEBody.select\(\)](#) to select a body

Arguments:

opening_type (string) Opening type. Valid entries are 'door', 'window' and 'hole'

set_building_orientation(orientation)

Set building orientation

Arguments:

orientation (float) Orientation from north in degrees

set_colour(colour_index)

Set colour of the selected body

Arguments:

colour_index (int) Index of the colour to set.

Possible colour index values:

0	BLUE
1	GREEN
2	RED
3	YELLOW
4	PURPLE
5	ORANGE
6	CYAN
7	LIGHT GREY
48	WHITE
56	BLACK

set_percent_doors(percent_doors)

Set percent doors for the selected body. Use [VEBody.select\(\)](#) to select a body.

Arguments:

percent_doors (int) Percent doors to set

set_percent_glazing(percent_glazing)

Set percent glazing of external surfaces for the selected body. Use [VEBody.select\(\)](#) to select a body.

Arguments:

<code>percent_glazing</code>	(int) Percent glazing to set
------------------------------	------------------------------

set_percent_holes(percent_holes)

Set percent holes for the selected body. Use [VEBody.select\(\)](#) to select a body.

Arguments:

<code>percent_holes</code>	(int) Percent holes to set
----------------------------	----------------------------

set_percent_wall_glazing(percent_glazing)

Set percent glazing of walls for the selected body. Use [VEBody.select\(\)](#) to select a body.

Arguments:

<code>percent_glazing</code>	(int) Percent glazing to set.
------------------------------	-------------------------------

6.1.35.3 Enums Defined Here

<code>opening_internality</code>	any, internal, external
----------------------------------	-------------------------

6.1.36 VELocate

Interface for getting VE location data.

6.1.36.1 Methods Defined Here

close_wea_data() -> None

Close the location database.

get() -> dict

Get location data. Returns a dictionary with the following keywords:

<i>altitude</i>	(float, meters)
<i>city, country</i>	(strings)
<i>dst_correction</i>	(float)
<i>dst_from_month, dst_to_month</i>	(float) range 1-12
<i>external CO2</i>	(float, ppm)
<i>ground_reflectance_summer,</i> <i>ground_reflectance_winter</i>	(float)
<i>ground_reflectance_summer_from_month,</i> <i>ground_reflectance_summer_to_month</i>	(float) range 1-12
<i>heating_loads percentile, cooling_loads percentile</i>	(float, %)
<i>latitude, longitude</i>	(float, degrees)
<i>national_climate_zone</i>	(int or string)
<i>summer_drybulb, summer_wetbulb, winter_drybulb</i>	(float, °C)
<i>time_zone</i>	(float)
<i>weather_source</i>	(string)
<i>weather_file</i>	(string)

open_wea_data() -> int

Open location data. Returns -1 on failure, else success.

save_and_close() -> None

Save and close the location database.

set(dict) -> None

Set location data. Possible dictionary keywords are:

<i>altitude</i>
<i>city</i>
<i>cooling_loads_percentile</i>
<i>country</i>
<i>dst_correction</i>
<i>dst_from_month</i>
<i>dst_to_month</i>
<i>external_CO2</i>
<i>ground_reflectance_summer</i>
<i>ground_reflectance_summer_from_month</i>
<i>ground_reflectance_summer_to_month</i>
<i>ground_reflectance_winter</i>
<i>heating_loads_percentile</i>

latitude
longitude
ref_air_density
summer_drybulb
summer_wetbulb
time_zone
winter_drybulb
weather_file

set_max_dry_bulb(temperature, month)
Set the maximum dry bulb temperature in degrees Celsius for a given month.

set_max_wet_bulb(temperature, month)
Set the maximum wet bulb temperature in degrees Celsius for a given month.

set_min_dry_bulb(temperature, month)
Set the minimum dry bulb temperature in degrees Celsius for a given month

6.1.36.2 Enums Defined Here

design_weather_source	ashrae, custom, apl, user
month	january, february, march, april, may, june, july, august, september, october, november, december

6.1.37 VEMacroFlo

Interface for Macroflo

6.1.37.1 Methods Defined Here

get() -> [opening data]

Returns data for all Macroflo openings.

Dictionary fields are:

<i>crack_flow_coefficient</i>	Crack flow coefficient
<i>crack_length</i>	Crack length as % of opening perimeter
<i>description</i>	Opening description
<i>equivalent_orifice_area</i>	Equivalent orifice area as % of gross
<i>exposure_type</i>	Exposure type name
<i>openable_area</i>	Openable area as %
<i>opening_category</i>	Opening category name
<i>opening_threshold</i>	Opening threshold temperature
<i>profile</i>	Degree of opening (modulating profile name)
<i>reference_id</i>	Reference ID of the opening

set(data)

Set data for a Macroflo opening from a dictionary of data.

Possible dictionary fields are:

<i>description</i>
<i>exposure_type</i>
<i>opening_category</i>
<i>openable_area</i>
<i>equivalent_orifice_area</i>
<i>crack_flow_coefficient</i>
<i>crack_length</i>
<i>opening_threshold</i>
<i>profile</i>

reload_wind_coefficients()

Force the VE to reload the wind coefficients file from the Apache folder.

6.1.37.2 Enums Defined Here

<i>opening_type</i>	custom_sharp_edge_orifice, window_door_side_hung, window_centre_hung, window_top_hung, window_bottom_hung, parallel_hung_windows_flaps, window_sash, sliding_roller_door, louvre, grille, duct, acoustic_duct
---------------------	--

6.1.38 VEModel

Interface for VE models.

Basic usage:

```
project = iesve.VEProject.get_current_project()
realmodel = project.models[o]
selectedbodies = realmodel.get_bodies(True)
```

6.1.38.1 Notes on VE Model objects

A project will consist of at least one building model, this is usually the real (proposed) building.

When working with rating or compliance systems where baseline / reference etc variant models are created. These variants will follow the real model in the models list. Some variant models are only available from within the context of a particular view (as they are generated on demand). An example of such an on-demand variant is UK Compliance (NCM) Notional building. The PRM Baseline building, on the other hand, is always available once it has been generated using the appropriate Navigator step.

6.1.38.2 Methods Defined Here

assign_thermal_template_to_rooms(template, [roomID])

Assign the [VEThermalTemplate](#) template to the rooms, indicated by list of Room IDs (strings or [VEBody](#) objects).

get_assigned_profiles(expandProfiles = False) -> list

Returns a list of strings representing the profile IDs in use by this model variant. Set expandProfiles to True to also return daily profiles.

get_bodies(selectedOnly) -> list

Returns the list of [VEBody](#) objects for this model variant. Set selectedOnly to True to return only selected bodies, False for all bodies.

get_bodies_and_ids(selectedOnly) -> dictionary

Returns the dictionary of [VEBody](#) objects with the room ID as key. Set selectedOnly to True to return only selected bodies, False for all bodies.

get_exterior_lighting_details([LightingStandard](#)) -> dict

Obtains exterior lighting data for the model. Includes a 'results' key which contains an ordered list of tuples for each of the sections.

get_included_bodies_for_umlh() -> [[VEBody](#)]

Returns the list of [VEBody](#) objects for this model variant, except those excluded for UMLH.

get_excluded_bodies_for_umlh() -> [[VEBody](#)]

Returns the list of [VEBody](#) objects for this model variant that are excluded for UMLH.

get_mep_details (attribute_type) -> dict

Obtains MEP power data from the model. The attribute_type parameter is optional and does not need to be specified unless it is an NECB model, where it must be supplied to provide the correct results.

rebuild_adjacencies()

Rebuilds the adjacencies in the model. If the model has inner volumes and newly-assigned constructions have differing thicknesses to the previous ones, this is required to be called.

suncast() -> [VESuncast](#) Object
Get a VESuncast object for this model

6.1.38.3 Attributes Defined Here

<i>id</i>	(string) Model description
<i>model_type</i>	(VEModels) Model type

6.1.38.4 Enums Defined Here

<i>VEModels</i>	VEModels_NA, RealBuilding, ActualBuilding, NotionalBuilding, TypicalBuilding, ReferenceBuilding, ProposedBuilding, BaselineBuilding, NewZealandProposedBuilding, NewZealandReferenceBuilding, GreenMarkProposedBuilding, GreenMarkReferenceBuilding, SunCastSolarVisBuilding, LEEDSunCastSS71Building, Title24ProposedBuilding, Section63PrescriptiveBuilding, Title24StandardDesignBuilding, NECBProposedBuilding, NECBReferenceBuilding
<i>LightingStandard</i>	PRM2007, PRM2010, PRM2013, IECC2012, NECB2011, PRM2016, NECB2017

6.1.39 VEProfile

Profile manipulations. Support for Daily, Weekly, Yearly, Compact and Free-form profiles. The latter are considered group profiles and have additional API calls available.

See the Tools\plot_profiles.py sample for a detailed reference on how to read all profile types data. See [VEProject API](#) for details on how to create new profiles

Basic usage:

```
project = iesve.VEProject.get_current_project()
dayprofiles, groupprofiles = project.profiles()
for id, profile in groupprofiles.items():
    if profile.is_weekly():
        print('Weekly profile {}, {}'.format(id, profile.reference))
    elif profile.is_yearly():
        print('Yearly profile {}, {}'.format(id, profile.reference))
```

6.1.39.1 Methods Defined Here

get_data() -> profile data

Retrieve profile data. The returned data depends on profile type. All the data structures return are compatible with the corresponding set_data() method.

- Daily profile: a list of [x, y, formula] lists that represent the data for the profile.
- Weekly profile: a list of daily profile IDs that make up the weekly profile.
- Yearly profile: a list of lists: [weekly profile ID, fromDay, toDay], where fromDay = year day where the profile becomes active, toDay = last day of profile
- Compact profile: a nested sets of lists, each list representing on/off time periods
- Free form profile : list of lists : [month, day, hour, minute, value]

set_data() -> bool

Set profile data. The data to be passed depends on profile type. You need to call save_profiles() on the originating VEProject instance to permanently save any changes.

Daily Profile:

Data is represented as a list of [x,y,formula] lists
[[x,y,formula], [x,y,formula], ...]

Weekly Profile:

Data is represented as a list of daily profile IDs
[ID1, ID2, ...]

Yearly Profile:

Data is represented as a list of [weekly profile ID(string), fromDay(int), toDay(int)] lists
[[ID1, from1, to1], [ID2, from2, to2], ...]

Compact Profile:

Data is represented as a list of lists that contains compact profile data:

The outer list is for the number of time periods

Each time period is represented by a list of lists:

[[toDay, toMonth], [entry], [entry], [entry], ...]

Where an entry list is:

[description, firstTime, secondTime]

Where firstTime and secondTime are:

[True, startHour, startMinute, endHour, endMinute]

OR

[False, 0, 0] if time is not active

Description: "Label: DaysOfWeek" (eg. "Weekends: Sat, Sun")
 StartHour, startMinute: 0-24, 0-59 for the start time
 EndHour, endMinute: 0-24, 0-59 for the end time

Free-form Profile:

Set the free-form entries for this profile.

Data is a list of lists, where each list entry consists of a list containing the month, day, hour, minute and value of that entry.

Value is a double if no formula is set, and a string otherwise.

First and last entries cannot be changed.

[[month, day, hour, minute, value],...]

month: 1-12

day: 1-31

hour: 0-23

minute: 0-59

is_absolute() -> Bool

True if the profile is absolute, else False.

is_group() -> Bool

True if the profile is a group profile (weekly, yearly, compact or free-form), False if a daily profile.

is_modulating() -> Bool

True if the profile is modulating, else False.

add_category(category)

Adds the given category (profile_category enum) to the profile.

clear_category(category)

Removes the given category (profile_category enum) from the profile.

get_categories() -> list[profile_category]

Gets all the categories assigned to the profile.

set_categories(list[profile_category])

Replaces the categories assigned to the profile with the given list.

is_supported_category(category) -> Bool

Checks whether the category is supported by the profile.

has_category(category) -> Bool

Checks whether the category is assigned to the profile.

For Group Profiles (derived from GroupProfile class) these additional methods are available:

is_compact() -> Bool

True if the profile is a compact profile, else False.

is_freeform() -> Bool

True if the profile is a free form profile, else False.

is_weekly() -> Bool

True if the profile is a weekly profile, else False.

is_yearly() -> Bool

True if the profile is a yearly (annual) profile, else False.

For Free form Profiles (FreeFormProfile class) these additional methods are available:

Note: Free form profiles require an appropriate licence to read/write. If such a licence is not available, any attempt to access free form profile data will throw an exception.

is_graphable() -> Bool

True if the free form data was loaded successfully and contains no error. False otherwise.

load_data(hide_ui)

Load the free form profile data. Note that this may take a long time. The optional Bool parameter `hide_ui` (default: True) can be passed as False to show a wait indicator whilst loading.

save_data(hide_ui)

Save the free form profile data back to disk. Note that this may take a long time. The optional Bool parameter `hide_ui` (default: True) can be passed as False to show a wait indicator whilst loading.

6.1.39.2 Attributes Defined Here

<i>id</i>	(string) Profile ID
<i>reference</i>	(string) Profile reference (name)

For Compact Profiles (CompactProfile class) only these additional attributes are available:

<i>num_periods</i>	(int) Number of periods in profile
--------------------	------------------------------------

6.1.39.3 Enums Defined Here

<i>profile_category</i>	none, occupancy, lighting, hvac, misc, ventilation, daylighting, equipment, solar, heating, cooling, plant, water, hvac_linked, emissions_factors
-------------------------	---

6.1.39.4 Detailed Data Description

Daily profile

a list of [x, y, formula] lists that represent the data for the profile.

x: time of day

y: value for time of day (0 if a formula is used)

formula: empty string if no formula is used, else the formula for that time value

Weekly profile

a list of daily profile IDs (strings) that make up the weekly profile.

Yearly profile

a list of lists: [weekly profile ID, fromDay, toDay]

weekly profile ID: the weekly profile ID for this time period

fromDay: year day where the profile becomes active (ie day 1 to 365)

toDay = last day of profile (ie day 1 to 365)

Compact profile

a list of time periods.

Each time period is defined as:

[[toDay, toMonth], entry, entry]

Where an entry consists of the following list items:

[InUse, startHour, startMinute, endHour, endMinute]

InUse: True / False Bool

StartHour, startMinute: 0-23, 0-59 for the start time (24:00 is permitted and equal to 00:00)

EndHour, endMinute: 0-23, 0-59 for the end time (24:00 is permitted and equal to 00:00)

Free form profile

list of lists: [month, day, hour, minute, value]

Note! You must call `load_data()` before the data is available. Failure to call `load_data()` will return only a dummy start/end node.

6.1.40 VEProject

Basic usage:

```
project = iesve.VEProject.get_current_project()
dayprofiles, groupprofiles = project.profiles()
print('Project folder: {}'.format(project.path))
```

6.1.40.1 Methods Defined Here

air_exchanges() -> list

Returns a list of all defined air exchanges as [AirExchange](#) objects.

apache_systems() -> list

Returns a list of all defined Apache Systems as [VEApacheSystem](#) objects.

archive_project(destination, full) -> bool

Archive the current project. Destination parameter (string) gives path + filename of the resulting Zip file archive. Pass True for full to include result files, False to exclude these large files.

casual_gains() -> list

Returns a list of all defined casual gains as [CasualGain](#) objects.

create_profile(type, reference, modulating, units) -> profile

Creates a new (empty / default) profile of the specified type.

Arguments:

<i>modulating</i>	(bool, optional [default = True]) True for a modulating profile, False otherwise
<i>reference</i>	(string, optional) name for profile
<i>type</i>	(string) one of the following values: 'daily', 'weekly', 'yearly', 'compact', 'freeform'
<i>units</i>	(int, optional [default = -1 (none)]) 0 for metric, 1 for IP or -1 for none

daily_profile(profileID) -> profile

Takes a profile ID string, and returns the corresponding daily profile as [DailyProfile](#) object, or None if ID was invalid.

daily_profiles([profileIDs]) -> dictionary

Takes a list of profile ID strings and returns a dictionary of corresponding [DailyProfile](#) objects, keyed by profile ID.

register_content(path, category, displayString, notification)

Register a file with Content Manager using category and display strings. Pass True for notification to have a notification displayed, False to add silently.

deregister_content(path)

Remove a file from Content Manager. No effect if the file does not exist in the Content Manager database.

get_display_units() -> DisplayUnits

Get the current VE unit display settings.

get_language_code() -> tuple[str, str]

Get the language code of the system the VE is being run on.

get_macro_flo_opening_by_id(...)

get_macro_flo_opening_by_id(openingID) -> opening type

Takes an opening ID and returns the corresponding [Macroflo](#) opening type.

get_macro_flo_opening_types(...)
 get_macro_flo_opening_types() -> [opening types]

Returns a list of all defined [Macroflo](#) opening types.

get_scenario_base_project_path() -> string
 If a scenario is currently active, returns the project path for the parent (non-scenario) model.

get_version() -> string
 Returns the version of the VE in the form of major.minor.revision.yy. The major version will indicate the primary release cycle, the minor version which feature pack level. The revision number indicates hotfix number.

group_profile(profileID) -> profile
 Takes a profile ID string, and returns the corresponding group profile as a [VEProfile](#) object.

group_profiles([profileIDs]) -> dictionary
 Takes a list of profile ID strings and returns a dictionary of corresponding [VEProfile](#) objects, keyed by profile ID.

profiles() -> ({daily},{group})
 Returns a tuple containing dictionaries of daily and group profiles.
 Each tuple entry is a dictionary of key=profile ID, value=[VEProfile](#) object.

save_profiles() -> Bool
 If profiles have been altered using [VEProfile](#) set_data method, the result won't be committed to the project until this method is called. The exception is saving a free form profile, which takes effect immediately and is irrevocable.

thermal_templates(assigned = True, allow_ncm = False) -> dictionary
 Returns a dictionary of [VEThermalTemplate](#) objects, keyed by template handle. Pass assigned = False to return all, or True (default) to only get the templates in use. Pass allow_ncm = True to include NCM profiles, False (default) to allow all profile types.

6.1.40.2 Static Methods Defined Here

get_current_project() -> project
 Returns the project currently loaded in the VE.

6.1.40.3 Attributes Defined Here

<i>content_folder</i>	(string) Project content folder path
<i>models</i>	(list) List of VEModel objects for active model variants. The first entry is always the 'real' model.
<i>name</i>	(string) Project name (title)
<i>path</i>	(string) Project folder path

6.1.40.4 Enums Defined Here

<i>DisplayUnits</i>	metric, imperial
<i>ProfileUnits</i>	metric, imperial, none

6.1.41 VERenewables

Interface for Renewables

6.1.41.1 Methods Defined Here

get_chp_data()->[CHP data dictionary]

Returns all CHP data.

Dictionary fields are:

<i>electrical_generation_meter</i>	Electrical generation meter name
<i>is_enabled</i>	Whether CHP data is enabled
<i>min_fraction_rated_heat_output</i>	Fraction of rated heat
<i>min_power_efficiency</i>	Minimum power efficiency
<i>min_thermal_efficiency</i>	Minimum thermal efficiency
<i>profile</i>	Name of profile for heat matching strategy
<i>quality_index</i>	Quality index
<i>rated_heat_output</i>	Heat output
<i>rated_power_efficiency</i>	Power efficiency at rated output
<i>rated_thermal_efficiency</i>	Thermal efficiency at rated output
<i>source_meter</i>	Source meter name

get_pv_data()->[List of dictionaries]

Returns data for all PVs.

Possible dictionary fields are (varies by PV class):

<i>area</i>	Area of the PV panel
<i>azimuth</i>	Azimuth of the PV
<i>cell_efficiency</i>	Cell efficiency
<i>cell_surface</i>	Cell surface
<i>description</i>	PV panel description
<i>geometric_concentration</i>	Geometric concentration
<i>id</i>	ID of the PV panel
<i>inclination</i>	PV inclination
<i>linear_temperature_factor</i>	Linear temperature factor
<i>meter</i>	Energy meter
<i>num_cells</i>	Number of cells
<i>optical_efficiency</i>	Optical efficiency
<i>power_temperature_coefficient</i>	Power temperature coefficient
<i>shading_factor</i>	PV shading factor
<i>spectral_factor</i>	Spectral factor
<i>tracking_device_power_losses</i>	Tracking device power losses
<i>tracking_error</i>	Tracking error
<i>type_id</i>	PV type ID. Use with <code>get_pv_type_by_id()</code> to get PV type data

get_pv_data_by_id(id)->[Dictionary of PV data]

Returns PV data from the given ID.

Possible dictionary fields are (varies by PV class):

<i>area</i>	Area of the PV panel
<i>azimuth</i>	Azimuth of the PV

<code>cell_efficiency</code>	Cell efficiency
<code>cell_surface</code>	Cell surface
<code>description</code>	PV panel description
<code>geometric_concentration</code>	Geometric concentration
<code>id</code>	ID of the PV panel
<code>inclination</code>	PV inclination
<code>linear_temperature_factor</code>	Linear temperature factor
<code>meter</code>	Energy meter
<code>num_cells</code>	Number of cells
<code>optical_efficiency</code>	Optical efficiency
<code>power_temperature_coefficient</code>	Power temperature coefficient
<code>shading_factor</code>	PV shading factor
<code>spectral_factor</code>	Spectral factor
<code>tracking_device_power_losses</code>	Tracking device power losses
<code>tracking_error</code>	Tracking error
<code>type_id</code>	PV type ID. Use with <code>get_pv_type_by_id()</code> to get PV type data

Args:

`id` (string): ID of the PV to get.

get_pv_type_by_id(...)

`get_pv_type_by_id(id)`->[Dictionary of PV type data]

Returns data for a PV type from the given ID.

Dictionary fields are:

<code>degradation_factor</code>	Degradation factor
<code>description</code>	PV type description
<code>electrical_conversion_efficiency</code>	Electrical conversion efficiency
<code>id</code>	ID of the PV type
<code>meter</code>	Energy meter
<code>module_nominal_efficiency</code>	Module nominal efficiency
<code>noct</code>	Nominal cell temperature (NOCT)
<code>ref_irradiance</code>	Reference irradiance for NOCT
<code>technology</code>	Name of PV type technology
<code>temp_coefficient</code>	Temperature coefficient for module efficiency

Args:

`id` (string): ID of the PV type to get.

get_pv_types(...)

`get_pv_types()`->[List of PV type data dictionaries]

Returns data for all PV types.

Dictionary fields are:

<code>degradation_factor</code>	Degradation factor
<code>description</code>	PV type description
<code>electrical_conversion_efficiency</code>	Electrical conversion efficiency
<code>id</code>	ID of the PV type
<code>meter</code>	Energy meter
<code>module_nominal_efficiency</code>	Module nominal efficiency

<code>noct</code>	Nominal cell temperature (NOCT)
<code>ref_irradiance</code>	Reference irradiance for NOCT
<code>technology</code>	Name of PV type technology
<code>temp_coefficient</code>	Temperature coefficient for module efficiency

get_wind_data()→[Wind data dictionary]

Returns all wind data.

Dictionary fields are:

<code>hub_height</code>	Hub height
<code>is_enabled</code>	Whether wind data is enabled
<code>meter</code>	Energy meter
<code>rated_power</code>	Rated power

set_chp_data(Dictionary of CHP data)

Sets CHP data.

Possible dictionary entries are:

`is_enabled`
`min_fraction_rated_heat_output`
`min_power_efficiency`
`min_thermal_efficiency`
`profile`
`quality_index`
`rated_heat_output`
`rated_power_efficiency`
`rated_thermal_efficiency`

set_pv_data(Dictionary of PV data, id)

Sets data for a PV panel.

Possible dictionary entries are:

`area`
`azimuth`
`cell_efficiency`
`cell_surface`
`description`
`geometric_concentration`
`inclination`
`linear_temperature_factor`
`meter`
`num_cells`
`optical_efficiency`
`power_temperature_coefficient`
`shading_factor`
`spectral_factor`
`tracking_device_power_losses`
`tracking_error`
`type_id`

set_pv_type_data(Dictionary of PV type data, id)

Sets data for a PV type.

Possible dictionary entries are:

degradation_factor

description

electrical_conversion_efficiency

meter

module_nominal_efficiency

noct

ref_irradiance

technology

temp_coefficient

set_wind_data(Dictionary of wind data)

Sets wind data.

Possible dictionary entries are:

is_enabled

hub_height

meter

rated_power

6.1.41.2 Enums Defined Here

pvs_type

monocrystalline_silicon, polycrystalline_silicon,
amorphous_silicon, other_thin_film,
thin_film_cadmium, thin_film_copper

reference_irradiance

irad_800wm2, irad_1000wm2

6.1.42 VERoomData

Room data. Returned by [VEBody](#) method `get_room_data()`

6.1.42.1 Methods Defined Here

get_air_exchanges() -> list

Get the room's air exchanges.

Returns:

List of RoomAirExchange objects.

get_apache_systems() -> dictionary

Gets the system data.

Returns:

Dictionary with the following information, including whether values are from template:

- Systems: HVAC system ID; methodology and whether the space is conditioned; auxiliary ventilation system ID and whether it matches the HVAC system; DHW system ID and whether it matches the HVAC system
- Heating: size (kW), radiant fraction, supply air temperature and whether the heating capacity is unlimited. If the capacity is not unlimited then the capacity and its units are included.
- Cooling: size (kW), radiant fraction, supply air temperature and whether the cooling capacity is unlimited. If the capacity is not unlimited then the capacity and its units are included.
- Outside air supply: flow rate and units, free cooling flow capacity and units, and variation profile ID.
- Ventilation & extract: whether room has local mechanical exhaust. If it does then also included are: the extract flow rate and units, SFP (W/l/s) and whether the extract fan is remote from the room.

Additional data is provided for HVAC and NCM modes.

NCM mode will return entries for:

`demand_controlled_ventilation_type`
`demand_controlled_ventilation_air_flow`
`heat_recovery`
`seasonal_efficiency`
`use_default_seasonal_efficiency`
`default_seasonal_efficiency`
`use_night_cooling`
`night_cooling_sfp`
`night_cooling_max_flow_rate`
`night_cooling_max_hrs_month`

Apache HVAC Methodology will return entries for:

`outside_air_ventilation_increase`
`outside_air_ventilation_increase_from_template`
`ventilation_req`
`max_req`
`exhaust_air_change_req_is_checked`
`exhaust_air_change_req`

get_building_regs() -> dictionary

Get the room's building regulation data. Only use this on NCM models.

Returns:

Dictionary with the following entries:

air_permeability_and_dhw_pipe_length
associated_occupied_room
criterion_3_solar_gain_benchmark
criterion_3_solar_gain_total
criterion_3_status
destratification_fans
external_ventilation_rate
external_ventilation_rate_from_template
high_pressure_drop_air_treatment
high_pressure_drop_air_treatment_from_template
include_room_in_building_regs_analysis
ncm_activity
ncm_activity_from_template
ncm_building_area_type
ncm_building_area_type_from_template
room_part_of_core_area
room_type
room_type_from_template
section_63_loft_access
section_63_loft_access_from_template

get_general() -> dictionary

Gets the general room data.

Returns:

Dictionary with the following information:

- Body name & ID
- General and thermal templates assigned to the body
- Room volume (m³) and floor area (m²)
- Lettable percentage, circulation percentage, whether or not the floor area is included in the building floor area, and whether these values are from template.

Additional data is provided for HVAC mode

get_internal_gains() -> list

Get the room's internal gains.

Returns:

List of [RoomInternalGain](#) objects (lighting / power / people).

get_ncm_lighting() -> dictionary

Get NCM lighting data. Only use this on NCM models.

Returns:

Dictionary with the following entries:

automatic_daylight_zoning

automatic_daylight_zoning_percentage
 constant_illuminance_control
 control_type
 design_illuminance
 different_sensor_control_back
 display_lighting_time_switching
 has_display_lighting
 lamp_efficacy
 lamp_type
 light_output_ratio
 lighting_case
 lumens_circuit_watt
 luminaires_fitted
 occupancy_parasitic_power
 occupancy_parasitic_power_default
 occupancy_parasitic_power_unit
 occupancy_time_switch_control
 occupancy_sensing
 photoelectric_parasitic_power
 photoelectric_parasitic_power_default
 photoelectric_parasitic_power_unit
 photoelectric_time_switch_control
 sensor_type
 use_efficient_lamps
 use_photoelectric
 wattage
 wattage_unit

get_necb_2017_lighting_controls() -> dictionary

Gets the NECB 2017 Lighting Controls information.

Returns:

Dictionary containing the following entries:

is_manual
 is_restricted_to_manual_on
 is_restricted_to_partial_auto_on
 is_bilevel
 is_auto_daylight_responsive_side_lighting
 is_auto_daylight_responsive_top_lighting
 is_auto_partial_off
 is_auto_full_off
 is_scheduled_shut_off

get_room_conditions() -> dictionary

Gets the room conditions.

Returns:

Dictionary with the following information, including whether values are from template:

- Heating: profile ID, whether setpoint is timed/constant/two-value, and either setpoint values (°C) or setpoint profile ID

- Cooling: profile ID, whether setpoint is timed/constant/two-value, and either setpoint values (°C) or setpoint profile ID
- DHW: value and units, whether DHW is linked to occupancy or an independent profile and a usage profile ID if relevant
- Plant (auxiliary energy): profile type and profile ID.
- Solar reflected fraction
- Furniture mass factor
- Humidity control: percentage saturation lower/upper limits and max heating (+ humidification) and max dehumidification (+ cooling), in kW.

Additional data is provided for HVAC mode.

get_transpired_solar_collectors() -> list

Get the room's transpired solar collectors. Only use this on NCM models.

Returns:

List containing a dictionary for each transpired solar collector.

Dictionary contains the following entries:

adjacency_area

collector_area

surface_number

set(general_data, room_conditions_data, systems_data)

Sets all non-NCM room data using the provided dictionaries.

See `set_general()`, `set_room_conditions()` and `set_apache_systems()` for dictionary keys.

set_apache_systems(data)

Sets system data from the provided dictionary.

Possible dictionary keys are:

associated_return_air_plenum

associated_supply_air_plenum

auto_associate_adjacent_plenums

aux_vent_system

aux_vent_system_from_template

aux_vent_system_same

cooling_capacity_unit

cooling_capacity_unlimited

cooling_capacity_unlimited_from_template

cooling_capacity_value

cooling_design_supply_temp

cooling_design_supply_temp_from_template

cooling_plant_radiant_fraction

cooling_plant_radiant_fraction_from_template

cooling_unit_size

demand_controlled_ventilation_air_flow

demand_controlled_ventilation_type

dhw_system

dhw_system_from_template

dhw_system_same

extract_fan_is_remote

extract_flow_rate

extract_flow_rate_unit

extract_SFP
 has_mech_exhaust
 heat_recovery_seasonal_efficiency
 heating_capacity_unit
 heating_capacity_unlimited
 heating_capacity_unlimited_from_template
 heating_capacity_value
 heating_design_supply_temp
 heating_design_supply_temp_from_template
 heating_plant_radiant_fraction
 heating_plant_radiant_fraction_from_template
 heating_unit_size
 HVAC_methodology
 HVAC_methodology_from_template
 HVAC_system
 HVAC_system_from_template
 include_in_room_zone_loads_analysis
 include_in_room_zone_loads_analysis_template
 mech_sfp
 mech_supply_in_room
 night_cooling_max_flow_rate
 night_cooling_max_hrs_month
 night_cooling_sfp
 outside_air_ventilation_increase
 outside_air_ventilation_increase_from_template
 scope_of_extract_system
 supply_condition
 supply_condition_profile
 supply_condition_template
 system_air_free_cooling
 system_air_free_cooling_from_template
 system_air_minimum_flowrate
 system_air_minimum_flowrate_from_template
 system_air_variation_profile
 system_air_variation_profile_from_template
 use_default_seasonal_efficiency
 use_night_cooling

set_building_regs(data)

Sets building regs data from the provided dictionary.
 Possible dictionary keys are:

air_permeability
 associated_occupied_room
 destratification_fans
 dhw_pipe_length
 external_ventilation_rate
 external_ventilation_rate_from_template
 high_pressure_drop_air_treatment
 high_pressure_drop_air_treatment_from_template
 include_room_in_building_regs_analysis
 ncm_activity
 ncm_activity_from_template
 ncm_building_area_type

ncm_building_area_type_from_template
room_part_of_core_area
room_type
room_type_from_template
section_63_loft_access
section_63_loft_access_from_template

set_collector_area(surface_number, collector_area)
Sets the collector area for the transpired solar collector of the indicated surface.

set_general(data)
Sets general room data from the provided dictionary.
Possible dictionary keys are:

circ_perc
circ_perc_from_template
included_in_building_floor_area
included_in_building_floor_area_from_template
lettable_perc
lettable_perc_from_template
name

set_ncm_lighting(data)
Sets NCM lighting data from the provided dictionary.
Possible dictionary keys are:

automatic_daylight_zoning
automatic_daylight_zoning_percentage
constant_illuminance_control
control_type
design_illuminance
different_sensor_control_back
display_lighting_time_switching
lamp_efficacy
lamp_type
light_output_ratio
lighting_case
lumens_circuit_watt
luminaires_fitted
occupancy_parasitic_power
occupancy_parasitic_power_default
occupancy_parasitic_power_unit
occupancy_sensing
occupancy_time_switch_control
photoelectric_parasitic_power
photoelectric_parasitic_power_default
photoelectric_parasitic_power_unit
photoelectric_time_switch_control
sensor_type
use_efficient_lamps
use_photoelectric

wattage

wattage_unit

set_room_conditions(data)

Sets room conditions data from the provided dictionary.

Possible dictionary keys are:

cooling_profile
cooling_profile_from_template
cooling_setpoint
cooling_setpoint_constant
cooling_setpoint_from_template
cooling_setpoint_profile
cooling_setpoint_twovalue_main_setpoint
cooling_setpoint_twovalue_profile
cooling_setpoint_twovalue_setback
cooling_setpoint_type
dhw
dhw_from_template
dhw_linked_to_occupancy
dhw_linked_to_occupancy_from_template
dhw_profile
dhw_profile_from_template
dhw_room_level_setting
dhw_room_level_setting_from_template
furniture_mass_factor
furniture_mass_factor_from_template
heating_profile
heating_profile_from_template
heating_setpoint
heating_setpoint_constant
heating_setpoint_from_template
heating_setpoint_profile
heating_setpoint_twovalue_main_setpoint
heating_setpoint_twovalue_profile
heating_setpoint_twovalue_setback
heating_setpoint_type
ncm_plant_profile
plant_profile
plant_profile_from_template
plant_profile_type
sat_perc_lower
sat_perc_lower_from_template
sat_perc_upper
sat_perc_upper_from_template
solar_reflected_fraction
solar_reflected_fraction_from_template

6.1.42.2 Attributes Defined Here

id (string) ID of the room this data belongs to

6.1.42.3 Enums Defined Here

air_flow_regulation_type	air_flow_damper_control, air_flow_speed_control
condition_type	external_air, temperature_from_profile
conditioned_flag	no_free_floating, not_applicable, no_tempered, yes
demand_controlled_ventilation	none, occupancy_density, gas_sensors, enhanced_ventilation
extract_system_scope	fan_within_zone, fan_remote_from_zone, fan_remote_from_zone_with_grease_filter
heat_recovery	no_heat_recovery, plate_heat_exchanger, heat_pipes, thermal_wheel, run_around_coil
lamp_types	t8_halophosphate_fluorescent_low_frequency, metal_halide, tungsten_or_halogen, fluorescent_no_details, compact_fluorescent
lighting_case	high_pressure_mercury, high_pressure_sodium, t8_triphosphor_fluorescent_high_frequency, t8_halophosphate_fluorescent_high_frequency, led, t12_halophosphate_fluorescent_low_frequency, t5_triphosphor_fluorescent_high_frequency
occupancy_sensing	unknown, full, uncalculated, lamp_type_only
photoelectric_control_type	manual_on_off_ext, auto_on_dimmed, auto_on_off, manual_on_dimmed, manual_on_auto_off, none
room_type	switching, dimming
sensor_type	heated, unheated_roof, unheated_loft, room_in_roof, glazing_cavity, unheated, dwelling_living_area, unheated_dwelling_conservatory, heated_dwelling_conservatory_scotland, internal_void
setpoint_type	addressable, standalone
wattage_unit	constant, variable, two_value
	wm2, w

6.1.43 VESankey

Sankey generation API. VESankey contains functionality to define a network of nodes and their connections, and generate a Sankey diagram based on the input data.

Usage:

```
sankey = iesve.VESankey('depth')
sankey.add_column_names(col_titles)
sankey.set_title('Sankey Diagram')
sankey.add_node_columns(nodes_cols)
sankey.add_connections(connections)
sankey.generate_sankey()
sankey.save_svg(svg_file)
```

6.1.43.1 Methods Defined Here

__init__(ordering)

Constructor function for VESankey. If the string 'depth' is passed to the constructor, then the nodes in the Sankey diagram will be sorted based on a depth-first traversal of the network. This can be useful to reduce cross-over of connections.

add_connections([(source, destination, weight)])

Adds connections between nodes in the network. Each entry in the *connections* list should be a tuple containing the source node name (string), destination node name (string) and the connection weight (real).

source -> string

destination -> string

weight -> real

Nodes should already have been added through either **add_node_names_in_order** or

add_node_columns

add_column_names ([columnNames])

Adds names for the columns in the diagram. Used for labelling the diagram.

add_node_colours ({colours})

Sets the colour each node should be generated with. *colours* is a dictionary of {string : tuple}, where the string is the name of the node, and the tuple holds the RGB values for the colour of that node.

add_node_columns ({nodeColumns})

Adds nodes to the diagram, in the specified column index.

nodeColumns is a dictionary, where each entry is {node name (string) : column index (number)}.

add_node_names_in_order ([names])

Adds names for the nodes to be used as labels in the diagram. Python Dictionaries do not retain ordering of items; this function can be used to ensure nodes are added to the diagram in the order they appear in the list. Can be useful to control positions of nodes in the final diagram.

add_node_columns still needs to be called, regardless of whether **add_node_names_in_order** is called or not.

clear()

Clears all added data from the sankey generator. Necessary if you wish to reuse the same VESankey object to generate multiple different sankey diagrams.

generate_sankey()

Once all nodes and connections have been added, calling **generate_sankey** will generate an in-memory representation of the sankey diagram, ready for exporting.

recommended_ratio () -> real

Returns the recommended ratio for the connection weight:line width ratios that will be used if no ratio is specified through **set_weight_width_ratio**.

save_png(filename)

After **generate_sankey** has been called, *save_png* will export the sankey diagram as a .png file.

save_svg(filename)

After **generate_sankey** has been called, *save_svg* will export the sankey diagram as a .svg file.

set_title(title)

Sets the title of the diagram.

set_units_str (unit)

Sets the string to be used as a legend for representing the units the connection weights are in.

set_weight_width_ratio (ratio)

Sets the weight:line width ratio the diagram should use. Can be used to somewhat control how thick the connection lines should be with vastly differing weights.

6.1.44 VESuncast

Interface for Suncast

6.1.44.1 Methods Defined Here

run(from_month, to_month, design_day, use_diffuse)

Run a Suncast simulation.

get_results(shading_filename, body_index, surface_index, surface_id, opening_num) -> list

Get Suncast insolation data. Returns a list of lists of dictionaries.

Returns data for the surface unless an opening number of 1 or greater is provided (optional).

Each list represents a month of the year.

The sub-list contains a dictionary of data for each hour of the design day.

The dictionary contains the following entries:

external

external_pc

internal

internal_pc

get_sun_up_down() -> list

Get sun up/down times as a list containing a dictionary of data for each day of the year.

The dictionary contains the following entries:

date

down_azi

down_time

up_azi

up_time

get_solar_altitudes(shading_filename, design_day) -> list

Get solar altitude data.

Returns a list of lists for each month containing the altitude for each hour of the design day

6.1.45 VESurface

Surface data. Returned by [VEBody](#) method `get_surfaces()`

6.1.45.1 Notes on Surfaces

A VE body is physically represented by a number of surfaces.

A surface can contain any number of openings (holes, windows, doors) and adjacencies (internal building surfaces that join 2 rooms). Note that openings can not overlap, and openings can not span surfaces. It is perfectly valid for a surface to have an area of 0 where it consists entirely of opening(s) or adjacencies (or any combination thereof).

A surface type is derived from its tilt angle and the VE setting for Vertical-horizontal transition angle (VE Model Settings dialog). Similarly, whether a section of a surface is an adjacency is controlled by the Adjacency separation distance.

6.1.45.2 Methods Defined Here

get_adjacencies() -> list

Returns a list of [VEAdjacency](#) objects for the surface.

get_areas() -> dictionary

Returns the various area values for the surface (all in m² units)

<i>area type</i>	total_gross, total_net, total_window, total_door, total_hole, total_gross_openings, internal_gross, internal_net, internal_window, internal_door, internal_hole, internal_gross_openings, external_gross, external_net, external_window, external_door, external_hole, external_gross_openings
------------------	--

get_constructions() -> list

Returns a list of construction IDs (string) used in the surface (including the constructions assigned to any openings in the surface).

get_openings() -> [openings]

Returns a list of openings ([VEGeometry](#) objects)

get_opening_by_id(opening_id) -> opening

Returns the opening associated to the id ([VEGeometry](#) object)

get_opening_totals() -> dictionary

Returns the number of openings of each of the following types: openings, doors, holes, windows, external_doors, external_holes, external_windows.

get_properties() -> dictionary (property)

Returns assorted properties of the surface

<i>aps_handle</i>	(int, id) the aps handle
<i>area</i>	(float, m ²) surface area
<i>id</i>	(string) Surface ID
<i>orientation</i>	(degrees from north) angle of orientation, not adjusted for site angle
<i>thickness</i>	(meters) the fixed inner volume offset if it has been set, else construction thickness
<i>tilt</i>	(degrees from horizontal) tilt angle, where 0 is straight up, 90 = vertical.
<i>type</i>	(string) representation of type attribute

move(distance) -> list
 Moves the surface by the distance provided.

id() -> str
 Gets the ID of the surface.

6.1.45.3 Attributes Defined Here

<i>index</i>	(int) Index of Surface within parent body
<i>type</i>	(VESurface_type enum) The surface type

6.1.45.4 Enums Defined Here

<i>VESurface_type</i>	floor, ceiling, ext_wall, int_glazing, ext_glazing, int_door, ext_door, ground_floor, roof, roof_glazing, int_wall, hole
-----------------------	--

6.1.46 VEThermalTemplate

Thermal template.

Basic usage:

```
proj = iesve.VEProject.get_current_project()
templates = proj.thermal_templates(True)
```

6.1.46.1 Methods Defined Here

add_air_exchange(airExchange)

Add an [AirExchange](#) object to the template.

add_gain(casualGain)

Add a [CasualGain](#) object to the template.

apply_changes()

Apply any changes made to the template to the model. Note: if this is not called, changes to the template will not be reflected in the model. This includes changes to gains and air exchanges for the template.

Note – this operation may take a significant amount of time on large models.

get() -> (roomConditions, apSystems, casualGains, airExchanges)

Returns a tuple containing all the details for the thermal template.

See the individual get_xxx functions below for information on the returned data.

get_air_exchanges() -> list

Returns a list of [AirExchange](#) objects.

get_apache_systems() -> dictionary

Dictionary containing Apache Systems settings. The following fields are returned:

<code>aux_vent_system</code>	(string) Aux vent system ID
<code>aux_vent_system_same</code>	(Bool) True if aux vent system is same as HVAC system
<code>cooling_capacity_units</code>	(int) see below
<code>cooling_capacity_unlimited</code>	(Bool)
<code>cooling_capacity_value</code>	(float) see cooling_capacity_units for units
<code>cooling_plant_radiant_fraction</code>	(float)
<code>dhw_system</code>	(string) DHW System ID
<code>dhw_system_same</code>	(Bool) True if DHW system is same as HVAC system
<code>heating_capacity_units</code>	(int) see below
<code>heating_capacity_unlimited</code>	(Bool)
<code>heating_capacity_value</code>	(float) see cooling_capacity_units for units
<code>heating_plant_radiant_fraction</code>	(float)
<code>HVAC_system</code>	(string) HVAC System ID
<code>system_air_free_cooling</code>	(float) see system_air_free_cooling_units for units
<code>system_air_free_cooling_units</code>	(int) see below for airflow units
<code>system_air_minimum_flowrate</code>	(float) see system_air_minimum_flowrate_units for units
<code>system_air_minimum_flowrate_units</code>	(int) see below for airflow units
<code>system_air_variation_profile</code>	(string) System air variation profile ID

Valid values for capacity units:

-1 = unlimited

0 = kW

1 = W/m²

Valid values for airflow units:

- 0 = ach
- 1 = l/s
- 2 = l/s/m²
- 3 = l/s/person

get_casual_gains() -> list

Returns a list of [InternalGain](#) objects.

get_room_conditions() -> dictionary

Dictionary containing Room Conditions settings. The following fields are returned:

cooling_profile	(string) profile ID
cooling_setpoint	(float, °C) cooling setpoint, only valid if cooling_setpoint_type is constant
cooling_setpoint_profile	(string) only valid if cooling_setpoint_constant is False
cooling_setpoint_twovalue_main_setpoint	(float, °C) Two-value main setpoint
cooling_setpoint_twovalue_profile	(string) Two-value profile ID
cooling_setpoint_twovalue_setback	(float, °C) Two-value setback setpoint
cooling_setpoint_type	(setpoint_type) Setpoint type
dhw	(float) DHW value linked to dhw_units
dhw_profile	(string) '-' if linked to occupancy, else profile ID of DHW profile
dhw_units	(string) one of: l/(h·pers) or l/h
heating_profile	(string), profile ID
heating_setpoint	(float, °C) cooling setpoint, only valid if heating_setpoint_type is True
heating_setpoint_profile	(string) only valid if heating_setpoint_constant is False
heating_setpoint_twovalue_main_setpoint	(float, °C) Two-value main setpoint
heating_setpoint_twovalue_profile	(string) Two-value profile ID
heating_setpoint_twovalue_setback	(float, °C) Two-value setback setpoint
heating_setpoint_type	(setpoint_type) Setpoint type
plant_profile	(string) profile ID
plant_profile_type	(int) 0 = heating Profile, 1 = cooling profile, 2 = independent profile
sat_perc_lower	(float, %)
sat_perc_upper	(float, %)

remove_air_exchange(airExchange)

Remove air exchange from template. Will throw exception if air exchange is not associated with template.

remove_gain(casualGain)

Remove casual gain from template. Will throw exception if gain is not associated with template.

set(room_conditions, system_data)

Set both System Data and Room Conditions settings by passing in dictionaries of settings.

See [set_apache_systems](#) and [set_room_conditions](#) for dictionary keys.

set_apache_systems(system_data)

Set System Data settings by passing in a dictionary of settings.

aux_vent_system, HVAC_system and DWH_system keys require a valid system ID.

Dictionary keys are:

aux_vent_system

aux_vent_system_same
 conditioned
 cooling_capacity_units
 cooling_capacity_value
 cooling_plant_radiant_fraction
 dhw_system
 dhw_system_same
 heating_capacity_units
 heating_capacity_value
 heating_plant_radiant_fraction
 HVAC_methodology
 HVAC_system
 system_air_free_cooling
 system_air_free_cooling_units
 system_air_minimum_flowrate
 system_air_minimum_flowrate_units
 system_air_variation_profile

set_room_conditions(room_conditions)

Set Room Conditions settings by passing in a dictionary of settings.

Dictionary keys are:

cooling_profile
 cooling_setpoint
 cooling_setpoint_profile
 cooling_setpoint_twovalue_main_setpoint
 cooling_setpoint_twovalue_profile
 cooling_setpoint_twovalue_setback
 cooling_setpoint_typeconstant
 dhw
 dhw_profile
 dhw_units
 heating_profile
 heating_setpoint
 heating_setpoint_constant
 heating_setpoint_twovalue_main_setpoint
 heating_setpoint_twovalue_profile
 heating_setpoint_twovalue_setback
 heating_setpoint_typeprofile
 plant_profile
 plant_profile_type
 sat_perc_lower
 sat_perc_upper

6.1.46.2 Attributes Defined Here

name	(string) Template description
standard	(VEThermalTemplate_standard enum) Template standard

6.1.46.3 Enums Defined Here

<code>VEThermalTemplate_standard</code>	generic, NCM, PRM_FloridaECB, NECB, t24
<code>heating_cooling_capacity_unit</code>	unlimited, kilowatts, watts_per_metre_squared

6.1.47 WeatherFileReader

Interface for working with weather files (fwt and epw formats).

Basic usage:

```
wea_file = iesve.WeatherFileReader()
result = wea_file.open_weather_file( 'WashingtonTMY2.fwt' )
assert result > 0, 'Error opening weather file.'
```

6.1.47.1 Methods Defined Here

open_weather_file(filename) -> int

Load a weather file indicated by filename. Returns > 0 on success, <= 0 on error.

close()

Close weather file.

get_results(variable, start_day, end_day) -> Numpy array of floats

Read a variable out of the weather file. Pass variable by integer from the table below or use the weather_variable enum defined below. start_day and end_day are the first and last day to return data for, ranging from 1 to 365 (inclusive).

The available variables (and their data units) are:

1	Cloud cover	Oktas
2	Wind direction [E of N]	Degrees
3	Dry bulb temperature	°C
4	Wet bulb temperature	°C
5	Direct normal radiation	W/m ²
6	Diffuse horizontal radiation	W/m ²
7	Solar altitude	Degrees
8	Solar azimuth	Degrees
9	Atmospheric pressure	Pa
10	Wind speed	m/s
11	Relative humidity	%
12	Humidity ratio (moisture content)	kg/kg
13	Global radiation	W/m ²
14	Dew point temperature	°C
15	Running-mean temperature	°C
16	Max-adaptive temperature	°C

6.1.47.2 Attributes Defined Here

<i>feb29</i>	(bool) True if year has February 29th (leap year), false otherwise
<i>lat</i>	(float) Weather site latitude
<i>long</i>	(float) Weather site longitude
<i>site</i>	(string) Weather site description
<i>solar_rad_convention</i>	(string) Solar rad convention
<i>start_weekday</i>	(string) First day of the year
<i>time_convention</i>	(string) Time convention
<i>time_zone</i>	(float) Weather site time zone (hours ahead of GMT)
<i>year</i>	(int) Weather data start year

6.1.47.3 Enums Defined Here

<i>weather_variable</i>	cloud_cover, wind_direction, dry_bulb_temperature, wet_bulb_temperature, direct_normal_radiation, diffuse_horizontal_radiation, solar_altitude, solar_azimuth, atmospheric_pressure, wind_speed, relative_humidity, humidity_ratio, global_radiation, dew_point_temperature, running_mean_temperature, max_adaptive_temperature
-------------------------	---

6.1.48 RefModelUserEdits

Interface for reference model user edits data

6.1.48.1 Methods Defined Here

get() -> dictionary

Get reference model user edits data.

Returns a dictionary with the following keys:

geometry

spaces

envelope

interior_lighting

hvac_systems

service_hot_water

miscellaneous

set(data)

Set reference model user edits data using a dictionary of data.

Dictionary can contain the following keys:

geometry

spaces

envelope

interior_lighting

hvac_systems

service_hot_water

miscellaneous